

Programmazione Avanzata in Java e C

Dispensa del Corso – Anno Accademico 2025/2026

Raccolta dei Capitoli e Compendio Teorico-Pratico

Dalle lezioni del Prof. Massimiliano Redolfi

con la collaborazione del Prof. Ing. Giorgio Bontempi

Rielaborazione a cura degli studenti

Indice

1 Warm-up: Fondamenti di C e Java a Confronto	5
1.1 Entry Point ed Esecuzione: Java vs C	5
1.1.1 Implementazione in C	5
1.1.2 Implementazione in Java	5
1.1.3 Architettura di Esecuzione della JVM: Bytecode, Stack Machine e JIT HotSpot	6
1.1.4 Aree di Memoria Runtime: Stack vs Heap e Passaggio dei Parametri per Valore	7
1.2 Primo Tempo: Gestione dei Dati (Rettangoli)	8
1.2.1 L'Approccio Procedurale in C: Strutture e Typedef	8
1.2.2 L'Approccio a Oggetti in Java: Classi e Costruttori	8
1.3 Secondo Tempo: Quadrati e Polimorfismo di Overloading	9
1.3.1 Il Limite Architetture del C: Assenza di Overloading	9
1.3.2 La Soluzione in Java: Overloading dei Metodi	10
1.4 Ereditarietà, Gerarchia e Catena di Costruzione	10
1.4.1 La Parola Chiave extends e la Construction Chain	10
1.4.2 La Classe Radice <code>java.lang.Object</code>	11
1.4.3 Il Contratto Matematico di <code>equals()</code> e <code>hashCode()</code> e l'Architettura delle Tabelle Hash	11
1.5 Metodi di Istanza e Override	13
1.5.1 Spostamento dei Metodi all'Interno della Classe	13
1.5.2 Override dei Metodi (<code>@Override</code>)	13
1.6 Terzo Tempo: Rombi, Forme Varie e Interfacce	14
1.6.1 La Crisi dell'Ereditarietà Singola	14
1.6.2 L'Interfaccia <code>Shape2d</code>	14
1.6.3 L'Arrivo delle Astronavi e la Segregazione dei Protocolli	14
1.7 Determinazione dei Tipi: Compile-Time vs Run-Time	15
1.7.1 Evoluzione del Costrutto Switch: Switch Expression, Arrow Syntax, yield ed Esaustività	16
1.8 Il Java Collections Framework	17
1.9 Java I/O: Accesso al File System e Flussi di Dati	18
1.9.1 La Classe <code>java.io.File</code>	18
1.9.2 Architettura degli Stream: Byte vs Caratteri	18
1.9.3 Pattern Robusto di Lettura e Scrittura Testuale	19
1.9.4 I/O Binario vs I/O Testuale: <code>DataInputStream</code> , <code>DataOutputStream</code> ed Endianness	19
1.9.5 Il Meccanismo Try-With-Resources: <code>AutoCloseable</code> , Ordine LIFO e Suppressed Exceptions	20
1.10 Tipi Enumerativi (enum) e Macchine a Stati Finiti	21
1.10.1 Struttura Avanzata di un enum	21
1.10.2 Implementazione di una Macchina a Stati Finiti (FSM)	21
1.11 Studio di Caso d'Aula: Il Censimento del Bimondo in Stile Procedurale	23
2 Dal Nome alla Funzione: Classi Anonime, Lambda, Method Reference e Nuove Interfacce	25
2.1 Il Problema delle Classi Usa-e-Getta e del Boilerplate	25
2.2 Classi Anonime: Sintassi, Uso e Limiti	26
2.2.1 Caratteristiche e Limitazioni delle Classi Anonime	26
2.2.2 Il Riferimento this nelle Classi Anonime	27
2.3 Closure e il Modello di Memoria: La Regola di "Effectively Final"	27
2.3.1 Cosa Succede in Memoria: La Copia per Valore	28
2.3.2 Perché le Variabili Devono Essere "Effectively Final"	28
2.4 Interfacce Funzionali e l'Annotazione <code>@FunctionalInterface</code>	29
2.5 Lambda Expression: Sintassi, Target Typing ed Efficienza	29
2.5.1 Guida Rapida alla Sintassi delle Lambda	30
2.5.2 Come il Compilatore Capisce il Tipo: Il Target Typing	30

2.5.3	Differenze Sostanziali tra Lambda e Classi Anonime	31
2.6	Method Reference: Le Quattro Categorie Canoniche	31
2.6.1	Le Quattro Forme di Method Reference	31
2.7	I Quattro Livelli di Astrazione a Confronto	32
2.8	Iniezione di Comportamento e Inversione del Controllo	33
2.9	Il Package <code>java.util.function</code> e la Composizione	34
2.9.1	I Cinque Ruoli Fondamentali	34
2.9.2	Tipi Primitivi e Zero-Boxing Overhead	34
2.9.3	Comporre Funzioni e Predicati	35
2.10	Il Motore della Calcolatrice Riscritto e Completo	35
2.11	L'Evoluzione delle Interfacce: Metodi Default, Static e Private	36
2.11.1	Perché Sono Nati i Metodi Default	36
2.12	Risoluzione dei Conflitti tra Metodi Default	36
2.12.1	Le Tre Regole della JVM per Risolvere i Conflitti	37
2.12.2	Metodi Statici e Metodi Privati nelle Interfacce (Java 8 e Java 9)	37
2.13	Contratti, Classi Astratte e i Due Volti dell'Ordinamento	38
2.13.1	Interfaccia o Classe Astratta? Differenze e Scelte Architettureali	38
2.13.2	Due Modi di Ordinare: Comparable vs Comparator	39
2.13.3	Comparator Moderno: Metodi Statici, Combinatori e Null-Safety	40
2.14	Gerarchie Chiuse: Tipi Sigillati (Sealed) e Switch sui Tipi	41
2.14.1	Pattern Matching e Switch Esaustivo (Java 21)	41
2.14.2	Confronto Sintetico: Gerarchie Aperte vs Gerarchie Chiuse	42
3	Stream Computazionali: Pipeline, Valutazione Pigra e Collettori	44
3.1	Il Problema del Paradigma Procedurale e della Modifica Concorrente	44
3.1.1	L'Anti-Pattern delle Liste Ausiliarie	44
3.1.2	La Trappola della Modifica durante l'Iterazione	45
3.2	Il Paradigma degli Stream come Flusso Computazionale	45
3.2.1	L'Analogia delle Palline nel Tubo	45
3.3	Anatomia di una Pipeline e Valutazione Pigra (Laziness)	46
3.3.1	Dimostrazione Sperimentale della Laziness	46
3.4	Ordine di Valutazione: Elaborazione Verticale vs Orizzontale	47
3.4.1	Tracciare l'Ordine Reale di Esecuzione	47
3.4.2	Operazioni Stateful e la Rottura della Verticalità	48
3.5	Il Vincolo di Monouso e la Soluzione con <code>Supplier<Stream<T></code>	48
3.5.1	Perché lo Stream non è Riavvolgibile?	48
3.5.2	La Soluzione Architettureale: <code>Supplier<Stream<T></code>	49
3.6	Creare Stream da Varie Sorgenti	49
3.6.1	Da Collezioni e Array	49
3.6.2	Stream Primitivi Specializzati: <code>IntStream</code> , <code>LongStream</code> , <code>DoubleStream</code>	49
3.6.3	Stream Infiniti: <code>Stream.iterate</code> e <code>Stream.generate</code>	50
3.7	Riduzioni Avanzate: Le Tre Forme di <code>reduce()</code>	50
3.8	Raccogliere Risultati: <code>collect()</code> e la Classe <code>Collectors</code>	51
3.8.1	Collettori Base e Concatenazione Testuale	51
3.8.2	Raggruppamento e Partizionamento Avanzato	52
3.8.3	Lo Studio del Caso d'Aula: Il Censimento del Bimondo in Una Riga	52
3.9	Dentro la Libreria: Architettura Interna, <code>Splitter</code> , <code>Sink</code> e <code>PipelineHelper</code>	53
3.9.1	Lo <code>Splitter<T></code> come Motore di Scorrimento e Decomposizione	53
3.9.2	La Catena di Stadi: <code>PipelineHelper</code> e <code>ReferencePipeline</code>	54
3.9.3	Il Protocollo di Push: I <code>Sink<T></code>	54
3.9.4	Prestazioni Parallele e la Regola di Lea ($N \times Q$)	54
4	Interfacce Grafiche Swing, Architettura ad Eventi ed Event Dispatch Thread	55
4.1	Dal Modello a Riga di Comando alla Programmazione ad Eventi	55
4.1.1	Paradigmi a Confronto: Esecuzione Lineare vs Reattiva	55

4.1.2	Il Dialogo Continuo tra Sistema Operativo e Applicazione	55
4.1.3	Eventi Hardware a Basso Livello vs Eventi Semantici di Alto Livello	56
4.2	Il Modello Architetture: Observer Pattern e Delegation Event Model	56
4.2.1	Multicast Events: Perché <code>add</code> e non <code>set</code> ?	56
4.3	Il Thread della Grafica: Event Dispatch Thread (EDT)	57
4.3.1	Perché Swing non è Thread-Safe?	57
4.3.2	La Chiamata Obbligatoria: <code>EventQueue.invokeLater</code>	57
4.4	Gerarchia dei Componenti Swing e Contenitori	58
4.4.1	I Livelli di Contenimento	58
4.5	Gestione dello Spazio: I Layout Manager	58
4.5.1	Perché il Posizionamento Assoluto (<code>setLayout(null)</code>) è Bandito	58
4.5.2	Confronto Sinottico tra i Principali Layout Manager	59
4.5.3	La Tecnica dei Pannelli Annidati (Nested Panels)	59
4.6	L'Evoluzione dei Listener: Da Classi Esterne a Method Reference	60
4.6.1	1. Classe Esterna Separata	60
4.6.2	2. Classe Interna (Inner Class)	60
4.6.3	3. Classe Anonima Inline (Pre-Java 8)	60
4.6.4	4. Lambda Expression (Java 8+)	60
4.6.5	5. Method Reference su Metodo Dedicato	60
4.7	Progettazione di Componenti Custom: Il Caso del Tastierino Numerico (<code>PnlDigits</code>)	61
4.7.1	Il Principio di Incapsulamento Rigoroso	61
4.7.2	Il Componente come Proxy di Eventi	61
4.7.3	Integrazione del Componente nel Frame Applicativo	62
4.8	Concorrenza in Swing: L'Event Dispatch Thread (EDT) e il Ponte verso i Background Thread	63
5	Thread, Concorrenza, Executor e Virtual Threads	65
5.1	Processi, Thread e il Modello di Memoria di Java	65
5.2	Creazione e Ciclo di Vita dei Thread: <code>Thread</code> vs <code>Runnable</code>	66
5.2.1	I Due Meccanismi di Creazione	66
5.2.2	Gli Stati del Ciclo di Vita di un Thread (<code>Thread.State</code>)	66
5.2.3	Attesa di Terminazione (<code>join()</code>) e Sospensione (<code>sleep()</code>)	67
5.2.4	La Politica Cooperativa di Interruzione (<code>interrupt()</code>)	67
5.3	Il Framework Executor: Disaccoppiamento tra Compito ed Esecuzione	68
5.3.1	Tipi di Thread Pool Standard (<code>java.util.concurrent.Executors</code>)	68
5.3.2	Ciclo di Vita e Chiusura Ordinata (<code>shutdown()</code>)	68
5.4	Calcolo Asincrono con Valore: <code>Callable<V></code> e <code>Future<V></code>	69
5.4.1	Caso di Studio d'Aula: Conteggio Parallelo dei Numeri Primi	69
5.5	Problemi di Sincronizzazione: Race Conditions e Visibilità	70
5.5.1	L'Anatomia di una Race Condition a Livello di Bytecode	70
5.5.2	Il Problema della Visibilità e la Keyword <code>volatile</code>	71
5.6	Sincronizzazione Intrinseca: La Parola Chiave <code>synchronized</code>	71
5.6.1	Le Tre Modalità di Sincronizzazione	71
5.6.2	Coordinamento Avanzato con <code>wait()</code> , <code>notify()</code> e <code>notifyAll()</code>	72
5.7	Deadlock e le Quattro Condizioni di Coffman	72
5.8	Utilità Concorrenti Avanzate del Package <code>java.util.concurrent</code>	73
5.8.1	Primitive Atomiche Lock-Free (<code>java.util.concurrent.atomic</code>)	73
5.8.2	Lock Espliciti <code>ReentrantLock</code> e Interfacce <code>Condition</code>	73
5.8.3	Il Pattern Produttore-Consumatore con <code>BlockingQueue</code>	73
5.9	Concorrenza in Ambienti Grafici: Swing, EDT e <code>SwingWorker</code>	74
5.10	La Nuova Era di Java: Virtual Threads (Java 21, Project Loom)	75
5.10.1	Il Limite Strutturale dei Platform Threads (1:1 con l'OS)	75
5.10.2	L'Architettura dei Virtual Threads (M:N User-Mode)	75
A	Eserciziario e Tracce di Laboratorio	77
A.1	Capitolo 1: Warm-up e Fondamenti della Programmazione a Oggetti	77

A.2	Capitolo 2: Classi Anonime, Lambda, Interfacce Moderne e Sealed	82
A.3	Capitolo 3: Stream Computazionali, Pipeline e Collettori	86
A.4	Capitolo 4: Interfacce Grafiche Swing ed Architettura ad Eventi	91
A.5	Capitolo 5: Thread, Concorrenza, Executor e Virtual Threads	97
B	Compendio e Formulario Tecnico di Java	107
B.1	PARTE I: SCHEMI CONCETTUALI E PATTERN ARCHITETTURALI	107
B.1.1	Notazione UML e Relazioni tra Classi	108
B.1.2	Principi Guida di Ingegneria del Software	108
B.1.3	Ereditarietà, Polimorfismo e Classi Astratte	109
B.1.4	Gestione delle Eccezioni	109
B.1.5	Complessità Computazionale delle Strutture Dati	110
B.2	PARTE II: FORMULARIO OPERATIVO DELLE API DI JAVA	110

Capitolo 1

Warm-up: Fondamenti di C e Java a Confronto

1.1 Entry Point ed Esecuzione: Java vs C

Ogni programma informatico richiede un punto di ingresso convenzionale (*entry point*) a partire dal quale il sistema operativo o il runtime avvia il thread di esecuzione primario. Il confronto tra la classica implementazione di “Hello World” nei due linguaggi mette in luce differenze architetturali profonde.

1.1.1 Implementazione in C

Nel linguaggio C, l'entry point è rappresentato dalla funzione libera globale `main`:

Hello World canonico in C (main.c)

```
1  include <stdio.h>
2
3  /*
4   * Entry point del programma in C.
5   * 'void' specifica esplicitamente che la funzione non accetta argomenti.
6   * Restituisce un intero (0 indica terminazione con successo al S.O.).
7   */
8  int main(void) {
9      printf("Hello World from C\n");
10     return 0;
11 }
```

In C, le funzioni risiedono nello spazio globale dei simboli (o limitate al file tramite il qualificatore `static`). Non esiste l'obbligo di incapsulare il codice all'interno di classi.

1.1.2 Implementazione in Java

In Java, ogni frammento di codice deve risiedere tassativamente all'interno di una classe:

Hello World canonico in Java (HelloWorld.java)

```
1  package it.unibs.eps;
2
3  public class HelloWorld {
4      /*
5       * Entry point universale per la Java Virtual Machine.
6       */
7      public static void main(String[] args) {
8          System.out.printf("Hello World from JAVA!\n");
9      }
10 }
```

Disamina Accademica della Firma di `main()` in Java

Durante la lezione, il docente ha analizzato punto per punto i vincoli della riga `public static void main(String[] args)`:

1. **public**: il metodo deve possedere visibilità pubblica globale per consentire al runtime della JVM (che risiede all'esterno del package dell'applicazione) di invocare il punto di ingresso.
2. **static**: è un metodo di classe e **non un metodo di istanza**. All'avvio del programma, nello heap *non esiste ancora alcun oggetto*. Se il metodo non fosse statico, la JVM si troverebbe in una situazione di stallo logico: non potrebbe invocare il metodo senza prima istanziare un oggetto, ma non saprebbe come costruirlo né quali parametri passare al costruttore. Il qualificatore **static**

svincola il metodo dall'esistenza di istanze nello heap.

3. **void**: il metodo non restituisce alcun valore nello stack di chiamata; eventuali codici di stato per il sistema operativo possono essere trasmessi esplicitamente tramite `System.exit(int)`.
4. **String[] args**: raccoglie i parametri passati all'applicazione tramite riga di comando sotto forma di array di stringhe.

1.1.3 Architettura di Esecuzione della JVM: Bytecode, Stack Machine e JIT HotSpot

A differenza dei programmi C, che vengono compilati direttamente in istruzioni macchina native specifiche per la combinazione di architettura hardware (es. x86_64, ARM64) e sistema operativo (Linux ELF, Windows PE), Java adotta una filosofia a due stadi sintetizzata dal paradigma storico “*Write Once, Run Anywhere*” (WORA).

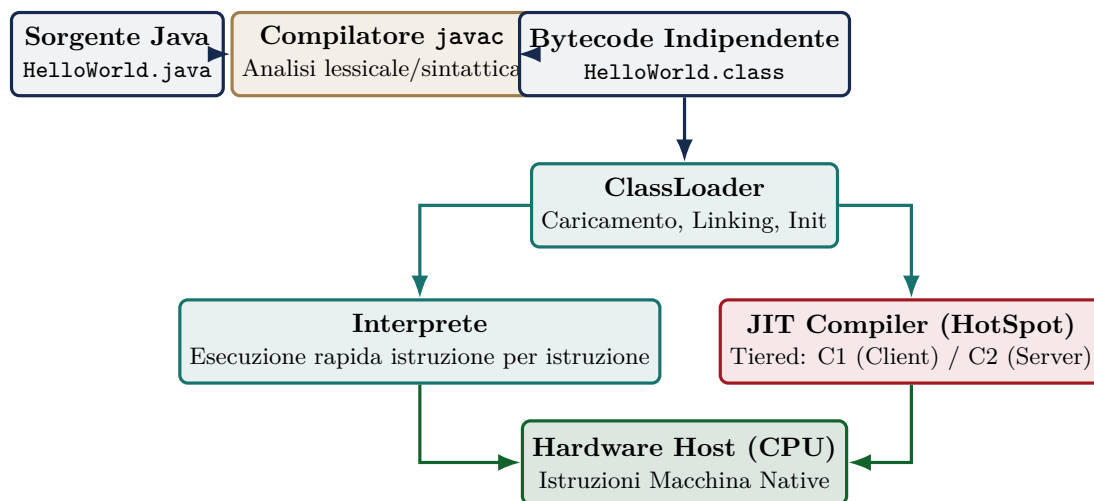


Figura 1.1: Pipeline di compilazione ed esecuzione: dal sorgente al bytecode e compilazione JIT HotSpot.

Il flusso operativo si articola nelle seguenti fasi:

1. **Compilazione Statica (javac)**: il compilatore verifica la correttezza formale del codice e lo traduce in **Bytecode**, una sequenza di istruzioni a basso livello contenuta in file binari con estensione `.class`. Il bytecode non fa alcuna assunzione sull'hardware reale, ma fa riferimento alle specifiche astratte della Java Virtual Machine.
2. **Macchina Virtuale Basata su Stack (Stack Machine)**: mentre le CPU fisiche moderne sono basate su registri (architettura *Register-based*), la JVM implementa un modello architetturale a **Stack di Operandi** (*Stack-based VM*). Le istruzioni di bytecode caricano dati nello stack (`iload`, `ldc`), prelevano operandi per eseguire operazioni aritmetiche (`iadd`, `imul`) e memorizzano il risultato nelle variabili locali (`istore`). Questo modello garantisce estrema compattezza del formato del bytecode e portabilità assoluta, svincolando la semantica del linguaggio dal numero e dalla dimensione dei registri della macchina fisica.
3. **Sottosistema ClassLoader**: quando la JVM richiede una classe, il ClassLoader ne individua il file `.class`, lo carica in memoria, esegue la verifica di sicurezza e integrità del bytecode (*Bytecode Verifier*), alloca la memoria per le variabili statiche e risolve i riferimenti simbolici in indirizzi diretti.
4. **Esecuzione Ibrida e JIT HotSpot (Tiered Compilation)**: all'avvio, l'esecuzione è affidata a un **interprete**, che esegue il bytecode istruzione per istruzione garantendo tempi di bootstrap immediati. Contemporaneamente, il motore di telemetria e profiling della JVM (**HotSpot**) monitora l'applicazione per individuare i *punti caldi* (*hot spots*), ovvero i metodi e i loop eseguiti con maggior frequenza. I blocchi caldi vengono compilati dinamicamente a runtime dal **Just-In-Time (JIT) Compiler** in codice macchina nativo ultra-ottimizzato. Con la *Tiered Compilation*:
 - Il compilatore **C1 (Client)** genera rapidamente codice macchina con ottimizzazioni base;
 - Il compilatore **C2 (Server)** interviene sulle routine più critiche applicando ottimizzazioni scalari avanzate: inlining aggressivo dei metodi, srotolamento dei loop (*loop unrolling*), eliminazione delle sincronizzazioni ridondanti e analisi di fuga (*escape analysis*), che consente di allocare oggetti nello stack anziché nello heap quando l'oggetto non sopravvive alla vita del metodo.

1.1.4 Aree di Memoria Runtime: Stack vs Heap e Passaggio dei Parametri per Valore

Per comprendere a fondo il comportamento di qualsiasi programma Java, è indispensabile padroneggiare la suddivisione fondamentale della memoria gestita dalla JVM durante l'esecuzione:

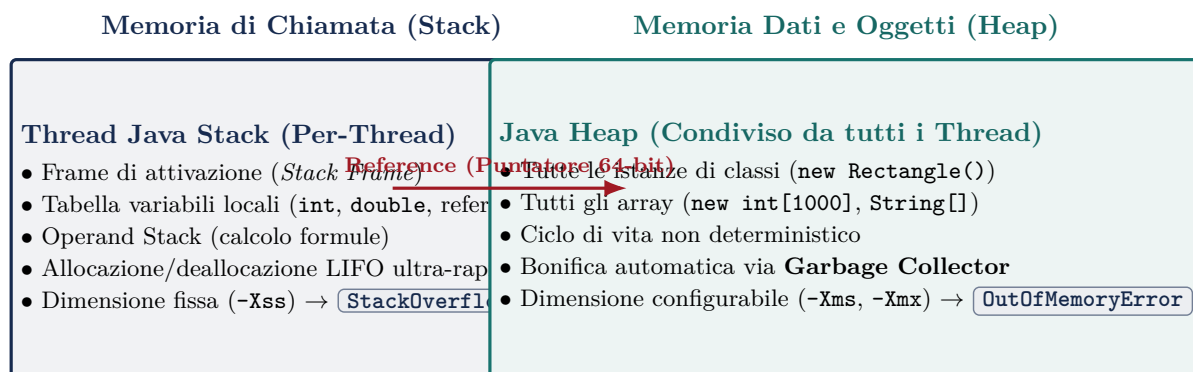


Figura 1.2: Disaccoppiamento strutturale della memoria Java: Stack privato dei thread vs Heap condiviso.

Principio Architetture: Il Dogma Assoluto: Java Passa TASSATIVAMENTE Tutto per Valore

Nel linguaggio Java **non esiste il passaggio per riferimento del C++ o dei puntatori C**. Ogni passaggio di parametri nei metodi avviene **esclusivamente per valore** (*pass-by-value*):

- **Tipi Primitivi** (`int`, `double`, `boolean`, ...): il parametro formale del metodo riceve una copia binaria pura del valore. Qualsiasi modifica apportata alla variabile all'interno del metodo agisce unicamente sullo stack frame del metodo stesso e non ha alcun effetto sulla variabile originale nel chiamante.
- **Tipi Riferimento** (`Object`, `Array`): il parametro formale riceve una copia del **valore del riferimento** (il puntatore all'oggetto nello heap). Di conseguenza:
 1. Se il metodo invoca mutatori sullo stato dell'oggetto dereferenziato (es. `r.width = 20`), la modifica si riflette sullo stato fisico dell'oggetto condiviso nello heap ed è visibile a tutti i riferimenti;
 2. Se il metodo riassegna il parametro a una nuova istanza (es. `r = new Rectangle(1, 1)`), viene modificata unicamente la copia locale del puntatore residente nello stack frame corrente. Al ritorno dal metodo, il riferimento del chiamante continua a puntare all'oggetto originale, inalterato.

Dimostrazione Empirica del Passaggio dei Parametri per Valore

```

1 public class ParameterPassDemo {
2     public static void testSwap(Rectangle r, int num) {
3         num = 999;                // Modifica locale al frame: non altera il chiamante!
4         r.width = 42;             // Modifica lo stato dell'oggetto nello Heap:
        VISIBILE!
5         r = new Rectangle(0, 0); // Riassegna la copia locale del puntatore: NESSUN
        EFFETTO nel chiamante!
6     }
7
8     public static void main(String[] args) {
9         int x = 10;
10        Rectangle rett = new Rectangle(8, 5);
11        testSwap(rett, x);
12
13        System.out.printf("x dopo il metodo: %d\n", x);           // Stampa: 10
        (inalterato)
14        System.out.printf("rett.width: %d\n", rett.width);        // Stampa: 42
        (modificato nello heap)
15        System.out.printf("rett.height: %d\n", rett.height);      // Stampa: 5 (non
        e' stato azzerato!)
16    }
17 }
```

1.2 Primo Tempo: Gestione dei Dati (Rettangoli)

Immaginiamo di trovarci in un universo geometrico bidimensionale (il “Bimondo”) e di dover modellare e catalogare gli oggetti che lo popolano, partendo dalle forme più semplici: i **rettangoli**.

1.2.1 L’Approccio Procedurale in C: Strutture e Typedef

In C, i dati composti vengono raggruppati tramite il costrutto **struct**. Le funzioni che operano su tali dati sono entità separate ed esterne:

Modellazione procedurale di un rettangolo in C (main.c)

```

1  include <stdio.h>
2
3  /*
4   * Definizione del tipo struttura per il rettangolo.
5   * 'Rectangle_ST' e' il tag di struttura nel namespace struct.
6   * 'Rectangle_T' e' il nuovo tipo di dato definito mediante typedef.
7   */
8  typedef struct Rectangle_ST {
9      int width;
10     int height;
11 } Rectangle_T;
12
13 /* Funzione procedurale esterna per il perimetro */
14 int perimeter(Rectangle_T r) {
15     return (r.width + r.height) * 2;
16 }
17
18 /* Funzione procedurale esterna per l'area */
19 int area(Rectangle_T r) {
20     return r.width * r.height;
21 }
22
23 int main(void) {
24     Rectangle_T r;
25     r.width = 8;
26     r.height = 5;
27
28     printf("Perimetro rettangolo: %d\n", perimeter(r));
29     printf("Area rettangolo: %d\n", area(r));
30     return 0;
31 }
```

Dibattito d’Aula: Rectangle_T è un Tipo o una Variabile?

Il docente ha posto all’aula un quesito fondamentale di sintassi C: “*Nel codice sopra, Rectangle_T è una variabile o un tipo?*”.

La risposta corretta è che si tratta di un **tipo di dato**:

- In C standard, dichiarando `struct Nome { ... }`, ogni successiva istanziazione richiederebbe la sintassi verbosa `struct Nome r;`.
- L’istruzione `typedef struct Rectangle_ST { ... } Rectangle_T;` definisce un alias: `Rectangle_ST` è l’identificatore associato alla struct (*struct tag*), mentre `Rectangle_T` diventa a tutti gli effetti un nuovo identificatore di tipo, permettendo di dichiarare variabili con `Rectangle_T r;`.
- Non viene allocata memoria fino a quando una variabile di tale tipo non viene formalmente dichiarata o allocata con `malloc`.

1.2.2 L’Approccio a Oggetti in Java: Classi e Costruttori

In Java, la separazione tra dati e funzioni comincia a dissolversi. In una prima versione elementare, definiamo la classe `Rectangle` come mero aggregato di campi:

Classe Rectangle elementare in Java (Rectangle.java)

```

1 package it.unibs.eps;
2
3 public class Rectangle {
4     public int width;
5     public int height;
6
7     /*
8      * Costruttore esplicito con inizializzazione dello stato.
9      * La parola chiave 'this' disambigua gli attributi di istanza dai parametri.
10    */
11    public Rectangle(int width, int height) {
12        this.width = width;
13        this.height = height;
14    }
15
16    /* Costruttore predefinito senza argomenti */
17    public Rectangle() {
18        this.width = 0;
19        this.height = 0;
20    }
21 }

```

Definizione: Operatore e Costruttore

In Java, l'operatore **new** è responsabile dell'allocazione fisica di un nuovo blocco di memoria nello **Heap**. Esso restituisce il riferimento (*reference*) all'istanza creata. Subito dopo l'allocazione, viene invocato il **costruttore**, un metodo speciale (privo di tipo di ritorno e con nome identico alla classe) avente il compito di inizializzare lo stato iniziale dell'oggetto.

Main con istanziazione di Rectangle (Main.java)

```

1 package it.unibs.eps;
2
3 public class Main {
4     /* Metodi procedurali statici definiti nel Main */
5     public static int perimeter(Rectangle r) {
6         return (r.height + r.width) * 2;
7     }
8
9     public static int area(Rectangle r) {
10        return r.height * r.width;
11    }
12
13    public static void main(String[] args) {
14        // Creazione dell'istanza nello heap mediante costruttore esplicito
15        Rectangle r = new Rectangle(8, 5);
16
17        System.out.printf("Perimetro: %d\n", perimeter(r));
18        System.out.printf("Area: %d\n", area(r));
19    }
20 }

```

1.3 Secondo Tempo: Quadrati e Polimorfismo di Overloading

Il dominio applicativo si espande: dobbiamo introdurre i **quadrati** (**Square**).

1.3.1 Il Limite Architettureale del C: Assenza di Overloading

Nel linguaggio C, tutti i simboli di funzione condividono un unico spazio globale dei nomi. Non è ammessa la presenza di due funzioni con lo stesso nome, anche qualora accettassero parametri differenti. L'aggiunta del quadrato costringe a duplicare le funzioni:

Duplicazione forzata di funzioni in C per il tipo Square

```

1  typedef struct Square_ST {
2      int width;
3      int height;
4  } Square_T;
5
6  /* In C dobbiamo inventare un nome differente! */
7  int perimeter_sq(Square_T s) {
8      return (s.width + s.height) * 2;
9  }
10
11 int area_sq(Square_T s) {
12     return s.width * s.height;
13 }
    
```

Questo vincolo degrada la scalabilità del software procedurale: all'aumentare delle forme, il numero di funzioni specifiche cresce linearmente (`perimeter_triangle`, `perimeter_circle`, ecc.).

1.3.2 La Soluzione in Java: Overloading dei Metodi

Java supporta nativamente il **polimorfismo di sovraccarico** (*method overloading*):

Overloading dei metodi statici in Java

```

1  public class Main {
2      // Versione per Rettangolo
3      public static int perimeter(Rectangle r) {
4          return (r.height + r.width) * 2;
5      }
6
7      // Versione sovraccaricata per Quadrato (stesso nome, firma diversa)
8      public static int perimeter(Square s) {
9          return (s.height + s.width) * 2;
10     }
11 }
    
```

Definizione: Method Overloading (Polimorfismo a Tempo di Compilazione)

Il polimorfismo di sovraccarico consente a più metodi appartenenti alla stessa classe di condividere il medesimo nome, purché presentino una **firma differente** per numero o tipo dei parametri. La risoluzione della versione da invocare avviene staticamente a tempo di compilazione (*early binding*) in base al tipo statico della variabile passata.

1.4 Ereditarietà, Gerarchia e Catena di Costruzione

Dal punto di vista della teoria geometrica, un quadrato non è un'entità slegata dal rettangolo: un quadrato è un rettangolo speciale in cui i lati hanno la medesima lunghezza ($width = height$). In programmazione orientata agli oggetti, questa relazione concettuale è formalizzata dal principio di **ereditarietà** (*IS-A relationship*).

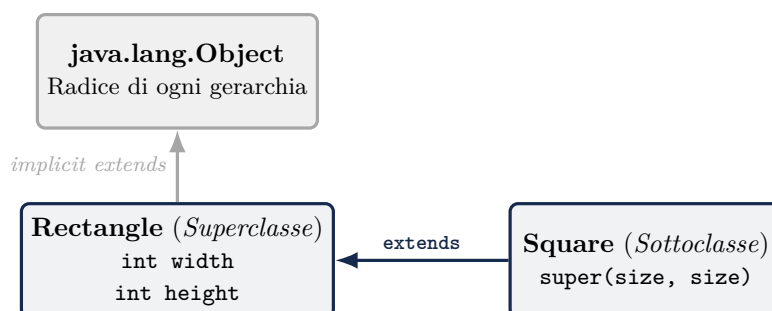


Figura 1.3: Gerarchia di ereditarietà tra `Square` e `Rectangle`, con derivazione da `Object`.

1.4.1 La Parola Chiave `extends` e la Construction Chain

Dichiarando `Square extends Rectangle`, la classe `Square` eredita automaticamente tutti gli attributi e i metodi accessibili della superclasse:

Definizione di Square tramite ereditarietà (Square.java)

```

1 package it.unibs.eps;
2
3 public class Square extends Rectangle {
4     /*
5      * Costruttore di Square.
6      * Invocando super(size, size) richiamiamo esplicitamente
7      * il costruttore della superclasse Rectangle.
8      */
9     public Square(int size) {
10         super(size, size);
11     }
12 }

```

Principio Architetture: Regola Aurea della Construction Chain

Ogni istanza di una sottoclasse racchiude al proprio interno una valida istanza della superclasse. Di conseguenza, il costruttore della classe derivata **deve tassativamente** invocare il costruttore della classe base (tramite `super(...)`) come primissima istruzione. Se non specificato, il compilatore inserisce automaticamente una chiamata a `super()` a vuoto; se la superclasse non possiede un costruttore senza parametri, si verifica un errore di compilazione.

1.4.2 La Classe Radice `java.lang.Object`

In Java, qualsiasi classe che non specifichi una clausola `extends` estende implicitamente la classe `Object`. Questa gerarchia unificata garantisce che ogni entità nello heap possenga un insieme garantito di comportamenti, tra cui:

- `toString()`: rappresentazione testuale dell'oggetto.
- `equals(Object)`: confronto semantico di uguaglianza tra contenuti.
- `hashCode()`: valore hash intero per l'indicizzazione nelle collezioni.
- `getClass()`: metadati di riflessione sul tipo a run-time.

1.4.3 Il Contratto Matematico di `equals()` e `hashCode()` e l'Architettura delle Tabelle Hash

Uno dei punti critici del design a oggetti in Java, fonte inesauribile di bug subdoli negli esami e nella pratica industriale, è la corretta implementazione della coppia di metodi `equals()` e `hashCode()`.

Definizione: Uguaglianza di Identità (==) vs Uguaglianza Semantica (`equals()`)

- L'operatore `==` verifica l'**uguaglianza referenziale**: restituisce `true` se e solo se le due variabili contengono lo stesso identico indirizzo di memoria, cioè puntano alla medesima istanza fisica nello heap (r_1 e r_2 sono lo stesso oggetto).
- Il metodo `equals(Object o)` definisce l'**equivalenza di contenuto** (semantica): stabilisce se due oggetti distinti nello heap debbano essere considerati equivalenti in base ai valori dei loro attributi significativi.

Principio Architetture: Le Cinque Proprietà Assiomatiche del Contratto di `equals()`

La specifica formale della JVM impone che qualsiasi implementazione di `equals(Object)` realizzi una relazione di equivalenza matematica tra riferimenti non nulli:

1. **Riflessività**: $\forall x \neq \text{null}$, l'espressione $x.\text{equals}(x)$ deve restituire `true`.
2. **Simmetria**: $\forall x, y \neq \text{null}$, $x.\text{equals}(y)$ deve restituire `true` se e solo se $y.\text{equals}(x)$ restituisce `true`.
3. **Transitività**: $\forall x, y, z \neq \text{null}$, se $x.\text{equals}(y)$ restituisce `true` e $y.\text{equals}(z)$ restituisce `true`, allora $x.\text{equals}(z)$ deve restituire `true`.
4. **Consistenza**: per qualsiasi riferimento $x, y \neq \text{null}$, molteplici invocazioni successive di

$x.\text{equals}(y)$ devono restituire costantemente lo stesso valore booleano, a condizione che nessun campo informativo impiegato nel confronto sia stato mutato.

5. **Non-nullità:** $\forall x \neq \text{null}$, la verifica $x.\text{equals}(\text{null})$ deve restituire tassativamente **false** (senza mai sollevare `NullPointerException`).

Principio Architetture: Il Contratto Invariante tra `equals()` e `hashCode()`

La regola aurea che governa l'integrità del Java Collections Framework (`HashSet`, `HashMap`) stabilisce:

Se due oggetti sono uguali secondo il metodo `equals(Object)`, essi DEVONO TASSATIVAMENTE produrre lo stesso valore intero da `hashCode()`:

$$x.\text{equals}(y) == \text{true} \implies x.\text{hashCode}() == y.\text{hashCode}()$$

Viceversa, se due oggetti producono lo stesso codice hash, **non è richiesto** che siano uguali (fenomeno inevitabile delle *collisioni hash*, per il principio dei cassetti o *pigeonhole principle*). Tuttavia, una funzione hash di alta qualità deve distribuire uniformemente gli elementi non uguali sull'intero spazio a 32 bit per garantire tempi di accesso $O(1)$.

Il Disastro del Hash Code Dimenticato nei Secchi (*Buckets*)

Cosa accade se uno sviluppatore sovrascrive `equals()` ma dimentica di sovrascrivere `hashCode()`?

1. In una `HashMap` o `HashSet`, la posizione di memorizzazione (*bucket index*) viene calcolata a partire dal valore restituito da `hashCode()`:

$$\text{index} = (h \oplus (h \gg 16)) \& (\text{capacità} - 1)$$

2. Se non sovrascritto, l'oggetto eredita l'`hashCode()` predefinito di `Object`, che calcola il valore in base alla locazione fisica nello heap.
3. Due istanze distinte con gli stessi identici valori (es. `new Rectangle(8, 5)` e `new Rectangle(8, 5)`) produrranno due hash totalmente differenti e finiranno in due secchi (*bucket*) diversi della tabella.
4. Quando si tenta di recuperare l'oggetto con `map.get(new Rectangle(8, 5))`, la mappa interroga il secchio sbagliato e restituisce `null`, rendendo l'oggetto fantasma e violando l'integrità dell'intera struttura dati.

Inoltre, da Java 8, se troppi elementi con hash collidenti si accumulano nel medesimo secchio superando la soglia di 8 elementi (`TREEIFY_THRESHOLD`), la JVM converte internamente la lista di trabocco in un albero rosso-nero bilanciato (`TreeNode`), garantendo un tempo di accesso $O(\log n)$ nel caso peggiore anziché $O(n)$.

Implementazione Canonica e Robusta di `equals()` e `hashCode()`

```

1 package it.unibs.eps;
2
3 import java.util.Objects;
4
5 public class Rectangle {
6     private int width;
7     private int height;
8
9     public Rectangle(int width, int height) {
10         this.width = width;
11         this.height = height;
12     }
13
14     @Override
15     public boolean equals(Object other) {
16         // 1. Controllo di identità referenziale (ottimizzazione rapida)
17         if (this == other) return true;
18         // 2. Controllo di non-nullità e corrispondenza esatta delle classi di tipo

```

```

19         if (other == null || this.getClass() != other.getClass()) return false;
20         // 3. Cast sicuro al tipo concreto
21         Rectangle that = (Rectangle) other;
22         // 4. Confronto campo per campo
23         return this.width == that.width && this.height == that.height;
24     }
25
26     @Override
27     public int hashCode() {
28         // Calcolo hash composito deterministico conforme al contratto
29         return Objects.hash(this.width, this.height);
30     }
31 }

```

1.5 Metodi di Istanza e Override

Fino a questo punto abbiamo trattato le classi come semplici contenitori di campi, delegando il calcolo a metodi statici esterni. Questo approccio è procedurale. Il vero paradigma ad oggetti richiede che i dati siano saldamente uniti ai metodi che li manipolano (**incapsulamento**).

1.5.1 Spostamento dei Metodi all'Interno della Classe

Trasformiamo `perimeter()` e `area()` in **metodi di istanza**:

Rectangle con metodi di istanza completi (Rectangle.java)

```

1 package it.unibs.eps;
2
3 public class Rectangle {
4     public int width;
5     public int height;
6
7     public Rectangle(int width, int height) {
8         this.width = width;
9         this.height = height;
10    }
11
12    /* Metodo di istanza: opera sullo stato dell'oggetto chiamante */
13    public int perimeter() {
14        return (this.width + this.height) * 2;
15    }
16
17    public int area() {
18        return this.width * this.height;
19    }
20 }

```

Quando eseguiamo `Square s = new Square(9); int p = s.perimeter();`, la classe `Square` eredita direttamente i metodi di `Rectangle`. Il perimetro restituito sarà corretto ($(9 + 9) \times 2 = 36$).

1.5.2 Override dei Metodi (@Override)

Tuttavia, per un quadrato, il calcolo del perimetro può essere ottimizzato o espresso in modo più specifico come lato moltiplicato per quattro ($4 \times l$). È possibile **sovrascrivere** l'implementazione ereditata:

Override del metodo perimeter in Square (Square.java)

```

1 package it.unibs.eps;
2
3 public class Square extends Rectangle {
4     public Square(int size) {
5         super(size, size);
6     }
7
8     @Override
9     public int perimeter() {

```

```

10      // Calcolo specializzato per il quadrato
11      return this.height * 4;
12  }
13 }

```

Definizione: Method Overriding (Polimorfismo Dinamico)

L'**override** avviene quando una sottoclasse ridefinisce un metodo ereditato dalla superclasse mantenendo **identica la firma** (nome, parametri e tipo di ritorno compatibile). Durante l'esecuzione, la JVM adotta il meccanismo del *late binding* (o dynamic dispatch): viene sempre eseguita l'implementazione corrispondente al tipo dinamico effettivo dell'oggetto nello heap, indipendentemente dal tipo statico della variabile di riferimento.

1.6 Terzo Tempo: Rombi, Forme Varie e Interfacce

L'espansione del Bimondo richiede ora di introdurre i **Rombi** (**Diamond**), le **Ellissi** (**Ellipse**) e i **Cerchi** (**Circle**).

1.6.1 La Crisi dell'Ereditarietà Singola

Un rombo possiede una diagonale maggiore (**diagonaleMagg**) e una diagonale minore (**diagonaleMin**). Il calcolo dell'area è dato da $(diagM \times diagM)/2$. Possiamo far derivare **Diamond** da **Rectangle**? **Assolutamente no!** Un rombo non possiede base e altezza ortogonali, e tentare di forzare tale derivazione violerebbe il principio di coerenza e sostituibilità (Principio di Liskov).

Allo stesso modo, un cerchio è un'ellisse con i semiassi coincidenti ($xAxis = yAxis = radius$). Come possiamo gestire un catalogo eterogeneo di figure geometriche, calcolarne il perimetro e disegnarle senza creare gerarchie innaturali?

1.6.2 L'Interfaccia **Shape2d**

La soluzione consiste nel definire un'astrazione pura mediante un'interfaccia:

Interfaccia **Shape2d** come contratto geometrico puro (**Shape2d.java**)

```

1 package it.unibs.eps;
2
3 public interface Shape2d {
4     int perimeter();
5     int area();
6     void draw();
7 }

```

Tutte le forme geometriche implementano l'interfaccia:

Snippet Java

```

1 public class Rectangle implements Shape2d { ... }
2 public class Diamond implements Shape2d { ... }
3 public class Ellipse implements Shape2d { ... }

```

1.6.3 L'Arrivo delle Astronavi e la Segregazione dei Protocolli

Supponiamo ora di dover inserire nel Bimondo un'astronave spaziale (**Spaceship**). Un'astronave possiede una velocità (**speed**), un livello di corazzatura (**armor**) e deve poter essere disegnata sullo schermo (**draw()**). Possiamo far implementare **Shape2d** all'astronave?

Perché un'Astronave NON è una **Shape2d**

Come rimarcato con enfasi dal docente durante la lezione: “Le astronavi NON sono forme geometriche, sono astronavi!”. Un'astronave non possiede un perimetro geometrico né un'area analitica. Se implementasse **Shape2d**, sarebbe costretta a definire metodi fittizi per **area()** e **perimeter()**, introducendo un anti-pattern distruttivo.

La soluzione architetturale corretta consiste nel definire un'interfaccia a grana più fine che descriva esclusivamente il protocollo di rendering:

Interfaccia comportamentale Drawable (Drawable.java)

```
1 package it.unibs.eps;
2
3 public interface Drawable {
4     void draw();
5 }
```

L'interfaccia `Shape2d` estenderà a sua volta `Drawable`:

Snippet Java

```
1 public interface Shape2d extends Drawable {
2     int perimeter();
3     int area();
4 }
```

La classe `Spaceship` implementerà unicamente `Drawable`. Il motore grafico potrà così iterare su una collezione eterogenea di oggetti `Drawable` e invocarne il metodo `draw()` in modo uniforme e semanticamente ineccepibile.

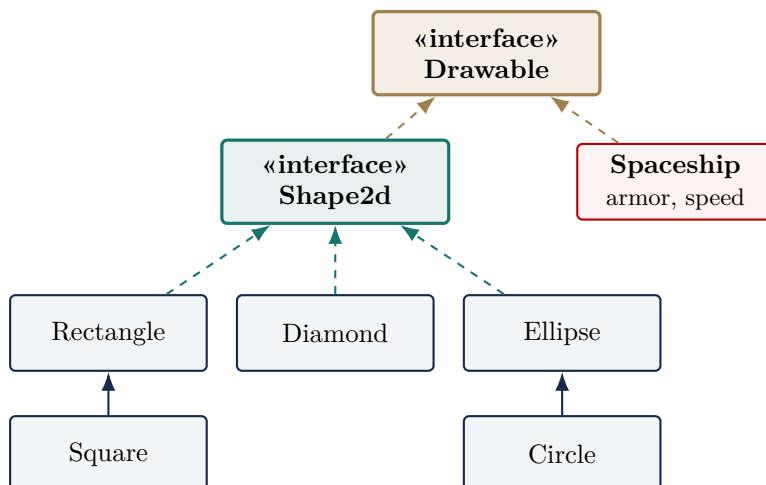


Figura 1.4: Diagramma architetturale completo: separazione semantica tra forme geometriche (`Shape2d`) e protocollo visivo (`Drawable`).

1.7 Determinazione dei Tipi: Compile-Time vs Run-Time

In un sistema a oggetti polimorfo, una variabile può possedere un tipo statico (dichiarato) più generale rispetto al tipo dinamico dell'istanza contenuta:

Controllo dei tipi con instanceof e Reflection

```
1 // Istanza reale: Square, Riferimento statico: Shape2d
2 Shape2d s = new Square(7);
3
4 /* 1. Controllo a tempo di compilazione tramite operatore instanceof */
5 if (s instanceof Shape2d) {
6     int p = ((Shape2d) s).perimeter();
7     System.out.printf("Perimetro calcolato: %d\n", p);
8 }
9
10 /* 2. Controllo dinamico a run-time tramite Reflection */
11 public boolean areObjectsAssignable(Object src, Object dest) {
12     return dest.getClass().isInstance(src);
13 }
```

1.7.1 Evoluzione del Costrutto Switch: Switch Expression, Arrow Syntax, yield ed Esaustività

Lo **switch** tradizionale, ereditato direttamente dal C e presente in Java fin dalla versione 1.0, era concepito unicamente come uno *statement* (istruzione procedurale di controllo di flusso) e soffriva di gravi difetti strutturali:

1. **Fall-through accidentale**: in assenza di un'istruzione esplicita **break**, l'esecuzione scivola automaticamente nei rami successivi. Questa caratteristica, pensata in origine per il micro-assembly, è stata la causa primaria di innumerevoli bug del software nei decenni.
2. **Incapacità di produrre valori**: non essendo un'espressione, per assegnare una variabile in base a un caso occorre dichiarare una variabile mutabile all'esterno e assegnarla all'interno di ciascun ramo, disperdendo la trasparenza referenziale.
3. **Scoping ambiguo**: tutte le variabili dichiarate all'interno dello switch condividevano lo stesso namespace locale.

A partire da Java 14, la piattaforma ha formalizzato le ****Switch Expression****, affiancate dalla sintassi a freccia (**->**):

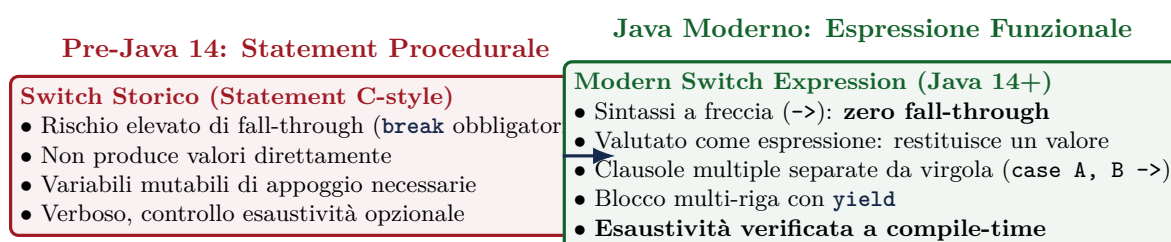


Figura 1.5: Confronto architetturale tra switch statement storico e moderna switch expression.

Esempio di Switch Expression Moderna con yield (CalcolatoreGiorni.java)

```

1 package it.unibs.eps;
2
3 public class CalcolatoreGiorni {
4     public enum Mese { GEN, FEB, MAR, APR, MAG, GIU, LUG, AGO, SET, OTT, NOV, DIC }
5
6     public static int giorniNelMese(Mese m, int anno) {
7         // Switch Expression impiegata come valore restituito dal return:
8         return switch (m) {
9             case GEN, MAR, MAG, LUG, AGO, OTT, DIC -> 31;
10            case APR, GIU, SET, NOV -> 30;
11            case FEB -> {
12                // Ramo complesso racchiuso in blocco graffe: uso obbligatorio di
13                yield
14                boolean bisestile = (anno % 4 == 0 && anno % 100 != 0) || (anno % 400
15                == 0);
16                System.out.printf("Controllo bisestile per l'anno %d: %b\n", anno,
17                bisestile);
18                yield bisestile ? 29 : 28;
19            }
20        }; // Il punto e virgola e' OBBLIGATORIO perche' chiude l'istruzione return
21    }
22 }
    
```

Principio Architetturale: Regola di Esaustività del Compilatore

Quando il costrutto **switch** viene impiegato come **espressione** (cioè produce un valore che viene assegnato a una variabile o restituito da un metodo), il compilatore impone tassativamente l'**esaustività** (*exhaustiveness*):

- Se lo switch opera su un tipo **enum**, devono essere coperte tutte le costanti dichiarate. Se una costante viene omessa e non è presente una clausola **default**, il programma rifiuta di compilare.
- Se opera su tipi aperti come stringhe o interi, la presenza della clausola **default** è richiesta obbligatoriamente dal compilatore.

1.8 Il Java Collections Framework

Nei sistemi applicativi complessi, l'utilizzo degli array primitivi a dimensione fissa (`T[]`) risulta eccessivamente rigido. Java mette a disposizione il **Java Collections Framework** nel package `java.util`, una gerarchia unificata di interfacce e classi di supporto.

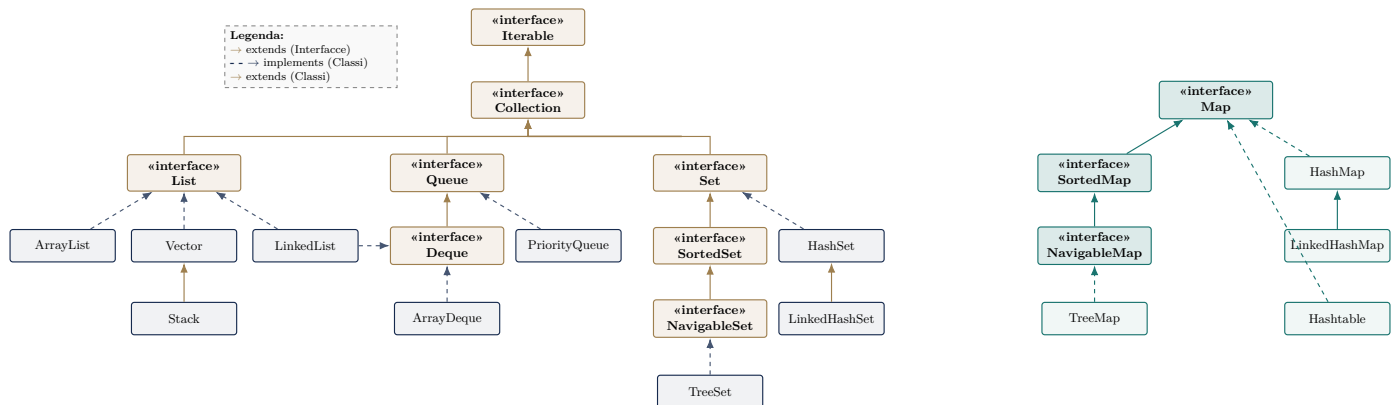


Figura 1.6: Tassonomia fondamentale del Java Collections Framework (le Mappe costituiscono una gerarchia indipendente da `Collection`).

- **List**: elenchi ordinati di elementi che ammettono duplicati. L'accesso posizionale è garantito dall'indice.
 - **ArrayList**: basato su array dinamico ridimensionabile. Ottimo per accessi casuali $O(1)$, più lento negli inserimenti centrali $O(n)$. Non sincronizzato.
 - **Vector**: analogo ad **ArrayList**, ma con metodi sincronizzati (thread-safe). Presenta un sovraccarico prestazionale nei contesti single-thread.
- **Set**: collezioni matematiche di elementi distinti (duplicati non ammessi).
 - **HashSet**: tabella hash, tempo di inserimento e lookup medio $O(1)$, nessun ordinamento garantito.
 - **TreeSet**: albero rosso-nero, mantiene gli elementi rigidamente ordinati secondo l'ordinamento naturale (**Comparable**) o un **Comparator**. Complessità $O(\log n)$.
- **Map**: associazioni chiave-valore con chiavi univoche. Non estende `Collection`.
 - **HashMap**: tabella hash altamente efficiente.
 - **TreeMap**: albero ordinato per chiave.

Catalogazione con `HashMap`: Istogramma delle Forme

Il docente ha mostrato un pattern canonico per contare quante istanze di ciascuna classe siano presenti all'interno di una collezione eterogenea di **Shape2d**:

Conteggio e catalogazione tramite HashMap (MainCatalog.java)

```

1  package it.unibs.eps;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.List;
6  import java.util.Map;
7
8  public class MainCatalog {
9      public static void main(String[] args) {
10         List<Shape2d> listaOggetti = new ArrayList<>();
11         listaOggetti.add(new Rectangle(5, 4));
12         listaOggetti.add(new Square(6));
13         listaOggetti.add(new Rectangle(2, 3));
14         listaOggetti.add(new Square(9));
15         listaOggetti.add(new Square(4));
16
17         // Mappa per il conteggio: NomeClasse -> Frequenza
18         Map<String, Integer> mappaOggetti = new HashMap<>();
19
20         for (Shape2d s : listaOggetti) {
21             String key = s.getClass().getSimpleName();
22             if (mappaOggetti.containsKey(key)) {
23                 int counter = mappaOggetti.get(key);
24                 mappaOggetti.put(key, counter + 1);
25             } else {
26                 mappaOggetti.put(key, 1);
27             }
28         }
29
30         // Iterazione elegante sulle coppie chiave-valore
31         for (Map.Entry<String, Integer> kv : mappaOggetti.entrySet()) {
32             System.out.printf("%s: %d istanze registrate\n", kv.getKey(),
33                 kv.getValue());
34         }
35     }
36 }

```

1.9 Java I/O: Accesso al File System e Flussi di Dati

La gestione dell'Input/Output in Java è articolata su due livelli distinti: la gestione delle entità del file system (metadati) e il trasferimento effettivo dei byte/caratteri (stream).

1.9.1 La Classe `java.io.File`

La classe `File` rappresenta un'astrazione del percorso di file e cartelle. **Punto critico:** la classe `File` permette di interrogare gli attributi del file system (`exists()`, `isDirectory()`, `length()`, `mkdir()`, `delete()`), ma **non consente di leggere o scrivere il contenuto del file**.

1.9.2 Architettura degli Stream: Byte vs Caratteri

Per accedere al contenuto, Java adotta il pattern architetturale a flussi (*Stream*):

- **Stream Orientati ai Byte:** operano su sequenze grezze di dati a 8 bit.
 - `InputStream` / `OutputStream`: classi base (specializzate in `FileInputStream` e `FileOutputStream`).
 - `BufferedInputStream` / `BufferedOutputStream`: buffer di memoria intermedia per minimizzare le costose system call al disco.
 - `DataInputStream` / `DataOutputStream`: consentono la lettura/scrittura binaria diretta di tipi primitivi Java (`writeInt`, `readDouble`).
- **Stream Orientati ai Caratteri (Reader e Writer):** operano su caratteri Unicode a 16 bit, gestendo automaticamente le tabelle di decodifica (es. UTF-8).
 - `FileReader` / `FileWriter`: aggancio ai file di testo.

- `BufferedReader` / `BufferedWriter`: lettura ad alte prestazioni con buffer; `BufferedReader` introduce il metodo fondamentale `readLine()`.
- `PrintWriter`: scrittura formattata ad alto livello (`printf`, `println`).

1.9.3 Pattern Robusto di Lettura e Scrittura Testuale

Metodi robusti per lettura e scrittura di file di testo (TextFileUtils.java)

```

1 package it.unibs.eps;
2
3 import java.io.BufferedReader;
4 import java.io.BufferedWriter;
5 import java.io.FileReader;
6 import java.io.FileWriter;
7 import java.io.IOException;
8
9 public class TextFileUtils {
10
11     /* Lettura completa riga per riga con numerazione */
12     public static String read(String fname) {
13         StringBuilder sb = new StringBuilder();
14         // Idioma moderno try-with-resources per chiusura automatica
15         try (BufferedReader in = new BufferedReader(new FileReader(fname))) {
16             int lcount = 1;
17             String s;
18             while ((s = in.readLine()) != null) {
19                 sb.append(String.format("%3d| %s\n", lcount++, s));
20             }
21             return sb.toString();
22         } catch (IOException ex) {
23             throw new RuntimeException("Errore durante la lettura del file: " +
24                                     fname, ex);
25         }
26     }
27
28     /* Scrittura di una stringa su file di testo */
29     public static void write(String fname, String body) {
30         try (BufferedWriter out = new BufferedWriter(new FileWriter(fname))) {
31             out.write(body);
32         } catch (IOException ex) {
33             throw new RuntimeException("Errore durante la scrittura del file: " +
34                                     fname, ex);
35         }
36     }
37 }
    
```

1.9.4 I/O Binario vs I/O Testuale: DataInputStream, DataOutputStream ed Endianness

La distinzione tra file di testo e file binari riflette due filosofie di persistenza profondamente diverse:

- **File di Testo:** memorizzano informazioni destinate alla lettura umana mediante sequenze di caratteri codificati (tipicamente UTF-8). Un numero intero pari a 12345678 richiede 8 byte di testo (i caratteri ASCII '1', '2', ..., '8'). L'accesso richiede parsing tramite `Scanner` o `Integer.parseInt()`, con elevato overhead di CPU e consumo di spazio.
- **File Binari:** salvano direttamente la rappresentazione binaria compatta dei tipi di dato primitivi: un `int` a 32 bit occupa **sempre e solo 4 byte**, un `double` occupa esattamente 8 byte, e un valore booleano occupa 1 singolo byte.

Per la gestione dei dati binari strutturati, il package `java.io` fornisce le classi filtro `DataOutputStream` e `DataInputStream`:



Figura 1.7: Composizione a strati dei filtri stream per I/O binario ad alte prestazioni.

Principio Architetture: Portabilità del Byte Order: Lo Standard Big-Endian della JVM

Nei linguaggi a basso livello come C, la scrittura binaria di un intero a 32 bit con `fwrite(&val, sizeof(int), 1, fp)` riversa i byte nell'ordine imposto dall'architettura della CPU ospitante (generalmente *Little-Endian* sui processori x86/ARM, dove il byte meno significativo precede quello più significativo). Questo rende il file illeggibile o corrotto se trasferito su un'architettura differente.

In Java, la specifica della JVM impone che `DataOutputStream` e `DataInputStream` operino **tassativamente in formato Big-Endian** (*Network Byte Order*: il byte più significativo viene scritto per primo), indipendentemente dalla CPU fisica su cui gira il programma. Questo garantisce la totale portabilità binaria dei dati tra qualsiasi sistema operativo e architettura.

Scrittura e Lettura Binaria Deterministica (BinaryDataDemo.java)

```
1 package it.unibs.eps;
2
3 import java.io.*;
4
5 public class BinaryDataDemo {
6     public static void salvaDati(String path, int id, double quota, String sigla)
7     throws IOException {
8         try (DataOutputStream out = new DataOutputStream(
9             new BufferedOutputStream(new FileOutputStream(path)))) {
10             out.writeInt(id);           // 4 byte esatti
11             out.writeDouble(quota);     // 8 byte esatti
12             out.writeUTF(sigla);       // Stringa binaria con prefisso di lunghezza a
13             // 2 byte
14         }
15     }
16
17     public static void caricaDati(String path) throws IOException {
18         try (DataInputStream in = new DataInputStream(
19             new BufferedInputStream(new FileInputStream(path)))) {
20             int id = in.readInt();
21             double quota = in.readDouble();
22             String sigla = in.readUTF();
23             System.out.printf("Letto record: ID=%d, Quota=%.2f, Sigla=%s\n", id,
24                 quota, sigla);
25         }
26     }
27 }
```

1.9.5 Il Meccanismo Try-With-Resources: AutoCloseable, Ordine LIFO e Suppressed Exceptions

Introdotta in Java 7, il costrutto **try-with-resources** costituisce il pilastro per la gestione sicura e deterministica delle risorse del sistema operativo (socket di rete, descrittori di file, lock fisici, connessioni database).

Definizione: Il Contratto dell'Interfaccia `java.lang.AutoCloseable`

Qualsiasi classe che implementi l'interfaccia `AutoCloseable` definisce un unico metodo formale:

```
void close() throws Exception;
```

La JVM garantisce che per qualsiasi risorsa istanziata nell'intestazione del **try**, il metodo `close()` verrà invocato automaticamente al termine del blocco protetto, sia che il blocco termini regolarmente, sia che si verifichi un'eccezione o un'istruzione di interruzione (**return**, **break**).

Principio Architetture: Regola LIFO di Chiusura delle Risorse Annidate

Quando nell'intestazione del **try** vengono dichiarate più risorse separate da punto e virgola, la JVM garantisce che la loro chiusura avvenga in ****ordine rigorosamente inverso rispetto alla dichiarazione**** (*Last-In, First-Out*):

```
try (FileInputStream fis = new FileInputStream(file);           // 1. Aperto per primo
```

```

    BufferedInputStream bis = new BufferedInputStream(fis); // 2. Aperto per secondo
    DataInputStream dis = new DataInputStream(bis)) {        // 3. Aperto per terzo
    // Elaborazione dati...
} // Chiusura automatica garantita in ordine: dis.close() -> bis.close() -> fis.close()

```

Questo ordine previene errori di corruzione: lo stream più esterno esegue il flush del buffer prima che lo stream di basso livello sottostante venga chiuso.

Il Drama Pre-Java 7 delle Eccezioni Soppresse (*Suppressed Exceptions*)

Nel codice legacy pre-Java 7, la chiusura delle risorse era affidata al blocco **finally**. Questo pattern nascondeva un vizio logico mortale:

1. Se il codice nel blocco **try** sollevava un'eccezione primaria (es. `DatiCorrottiException`);
2. E la successiva chiamata manuale a `in.close()` nel blocco **finally** sollevava a sua volta un'eccezione secondaria (es. `DiskFailureException`);
3. L'eccezione del **finally** sovrascriveva e cancellava **totalmente** l'eccezione primaria del blocco **try**, disperdendo per sempre la vera causa scatenante dell'errore.

Con il **try-with-resources**, questo problema è architetturalmente risolto: l'eccezione primaria del blocco **try** viene preservata e propagata, mentre l'eventuale eccezione sollevata durante la chiusura automatica di `close()` viene allegata come ***eccezione soppressa***. Essa rimane accessibile tramite il metodo `Throwable.getSuppressed()`, consentendo diagnosi forensi ineccepibili.

1.10 Tipi Enumerativi (enum) e Macchine a Stati Finiti

Nel linguaggio C tradizionale, la keyword **enum** definisce semplici costanti intere associate a etichette mnemoniche, prive di sicurezza di tipo (un enum in C può essere assegnato a un qualunque intero).

In Java, gli **enum** sono veri e propri tipi ad oggetti di prima classe (*type-safe enum pattern*). Estendono implicitamente la classe `java.lang.Enum`, possono possedere costruttori, campi privati, metodi di istanza e interfacce.

1.10.1 Struttura Avanzata di un enum

Enum Java con stato interno e metodi (Colore.java)

```

1 package it.unibs.eps;
2
3 public enum Colore {
4     RED("rosso brillante"),
5     GREEN("verde foresta"),
6     BLUE("blu notte");
7
8     private final String descrizione;
9
10    // Il costruttore di un enum e' sempre implicitamente privato
11    Colore(String descrizione) {
12        this.descrizione = descrizione;
13    }
14
15    public String getDescrizione() {
16        return this.descrizione;
17    }
18 }

```

1.10.2 Implementazione di una Macchina a Stati Finiti (FSM)

L'apice dell'espressività degli enumerativi in Java risiede nella possibilità di dichiarare un **metodo astratto** all'interno dell'enum e far sì che ogni singola costante ne fornisca un'implementazione specifica mediante una classe interna anonima. Questo consente di modellare **Macchine a Stati Finiti** in modo deterministico e compatto:

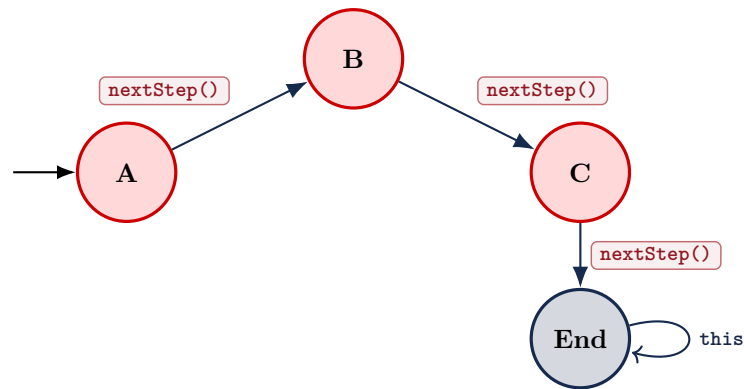


Figura 1.8: Diagramma di transizione degli stati della Macchina a Stati Finiti modellata con enum Java.

Macchina a stati finiti con pattern enum polimorfo (MacchinaStati.java)

```

1 package it.unibs.eps;
2
3 public enum MacchinaStati {
4     A("Stato Iniziale - Alfa") {
5         @Override
6         public MacchinaStati nextStep() {
7             return B;
8         }
9     },
10    B("Stato di Elaborazione - Beta") {
11        @Override
12        public MacchinaStati nextStep() {
13            return C;
14        }
15    },
16    C("Stato di Verifica - Gamma") {
17        @Override
18        public MacchinaStati nextStep() {
19            return End;
20        }
21    },
22    End("Stato Finale - Terminato") {
23        @Override
24        public MacchinaStati nextStep() {
25            return this; // Rimane nello stato terminale
26        }
27    };
28
29    private final String desc;
30
31    MacchinaStati(String desc) {
32        this.desc = desc;
33    }
34
35    public String getDesc() {
36        return this.desc;
37    }
38
39    /* Metodo astratto obbligatoriamente implementato da ogni stato */
40    public abstract MacchinaStati nextStep();
41 }

```

Esecuzione del ciclo della macchina a stati (MainFSM.java)

```

1 package it.unibs.eps;
2
3 public class MainFSM {
4     public static void main(String[] args) {
5         MacchinaStati statoCorrente = MacchinaStati.A;

```

```

6      System.out.println("== Avvio Macchina a Stati ==");
7      while (statoCorrente != MacchinaStati.End) {
8          System.out.printf("Attivo: %s\n", statoCorrente.getDesc());
9          statoCorrente = statoCorrente.nextStep();
10     }
11     System.out.printf("Raggiunto: %s\n", statoCorrente.getDesc());
12     System.out.println("== Esecuzione Conclusa ==");
13 }
14 }
15 }

```

Principio Architetture: Vantaggi Architetture del Pattern FSM con Enum

Modellare una macchina a stati tramite enum garantisce:

- **Type Safety assoluta:** è impossibile transitare verso uno stato non contemplato dal sistema a tipi.
- **Incapsulamento della logica di transizione:** ogni stato è responsabile esclusivo della determinazione del proprio successore.
- **Assenza di switch/case complessi:** si elimina la proliferazione di costrutti **switch-case** procedurali, delegando il comportamento al polimorfismo interno delle costanti enumerative.

1.11 Studio di Caso d'Aula: Il Censimento del Bimondo in Stile Procedurale

A chiusura del modulo di Warm-up, il docente ha proposto una sintesi applicativa fondamentale: la catalogazione statistica di un universo popolato da forme geometriche eterogenee (il *Bimondo*).

Immaginiamo di disporre di una collezione di figure bidimensionali che implementano il contratto `Shape2d` e di voler determinare la frequenza di ciascuna classe concreta presente nel mondo:

Censimento del Bimondo: Approccio Procedurale con HashMap (BimondoProcedurale.java)

```

1  package it.unibs.eps;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.List;
6  import java.util.Map;
7
8  public class BimondoProcedurale {
9      public static void main(String[] args) {
10         // Popolamento del mondo geometrico eterogeneo
11         List<Shape2d> mondo = new ArrayList<>();
12         mondo.add(new Rectangle(4, 5));
13         mondo.add(new Square(6));
14         mondo.add(new Rectangle(2, 8));
15         mondo.add(new Diamond(10, 6));
16         mondo.add(new Square(3));
17         mondo.add(new Square(7));
18
19         // Mappa per il censimento delle frequenze: NomeClasse -> Conteggio
20         Map<String, Integer> frequenze = new HashMap<>();
21         double areaTotale = 0.0;
22
23         for (Shape2d s : mondo) {
24             String nomeClasse = s.getClass().getSimpleName();
25
26             // Aggiornamento procedurale con getOrDefault
27             int conteggioAttuale = frequenze.getOrDefault(nomeClasse, 0);
28             frequenze.put(nomeClasse, conteggioAttuale + 1);
29
30             // Accumulo procedurale dell'area complessiva
31             areaTotale += s.area();

```

```

32     }
33
34     System.out.println("=== CENSIMENTO STATISTICO DEL BIMONDO ===");
35     for (Map.Entry<String, Integer> entry : frequenze.entrySet()) {
36         System.out.printf("Entita': %-12s | Rilevati: %d\n", entry.getKey(),
37             entry.getValue());
38     }
39     System.out.printf("Area Totale del Mondo: %.2f\n", areaTotale);
40 }

```

Il Punto di Partenza per la Rivoluzione Funzionale

Questo algoritmo procedurale, sebbene corretto, richiede la dichiarazione di variabili di stato mutevoli (`areaTotale`), una mappa esplicita e controlli manuali sulle chiavi. Nel Capitolo 3 vedremo come questa intera elaborazione statistica verrà compressa in un'unica ed elegante pipeline dichiarativa tramite le Stream API e i collettori `Collectors.groupingBy`.

Consultazione Esercizi di Laboratorio sul Modulo 1

Per consolidare l'architettura delle classi, la gestione della memoria Stack vs Heap, il contratto di `equals()` ed `hashCode()`, la manipolazione dei file binari e le macchine a stati finiti, si raccomanda di consultare e svolgere gli esercizi dettagliati nell'**Appendice A**:

- **Esercizio A.1 (StringMagic)**: incapsulamento, manipolazione stringhe e metodologia TDD;
- **Esercizio A.1 (FileUtil)**: I/O a caratteri e hex dump a byte con `try-with-resources`;
- **Esercizio A.1 (CollectionStats)**: statistiche descrittive su collezioni generiche;
- **Esercizio A.1 (Gerarchia Geometrica)**: separazione tra `Shape2d` e protocollo visivo `Drawable`;
- **Esercizio A.1 (Macchina a Stati Finiti)**: implementazione pattern State mediante enum polimorfi;
- **Esercizio A.1 (Polinomio Incapsulato)**: algebra simbolica e rispetto del contratto `equals`/`hashCode`;
- **Esercizio A.1 (MiniFileSystemSimulator)**: gerarchia composita multi-file (Composite Pattern).

Capitolo 2

Dal Nome alla Funzione: Classi Anonime, Lambda, Method Reference e Nuove Interfacce

In questo capitolo affrontiamo un problema classico della programmazione a oggetti: quando un programma cresce, spesso ci troviamo a creare decine di piccole classi che servono una sola volta (*classi usa-e-getta*). Questo riempie il progetto di file minuscoli e codice ripetitivo (*boilerplate code*).

L'obiettivo didattico è imparare a trattare i **comportamenti come entità di prima classe**: non vogliamo creare per forza un oggetto intero solo per eseguire un'azione; vogliamo poter passare direttamente una funzione a un metodo, memorizzarla in una mappa o restituirla come risultato di un calcolo.

2.1 Il Problema delle Classi Usa-e-Getta e del Boilerplate

Immaginiamo di progettare il motore (*engine*) di una calcolatrice per numeri interi. La calcolatrice riceve due numeri interi e un simbolo di operazione (ad esempio "+", "-", "*", "^"), e deve calcolare il risultato corrispondente.

Nell'approccio classico a oggetti, separiamo nettamente la definizione dell'interfaccia dalle implementazioni concrete:

1. Definiamo un'interfaccia con il contratto per gli operatori binari;
2. Creiamo una classe separata per ogni singola operazione aritmetica.

Interfaccia di Contratto: BinaryOperator.java

```
1 public interface BinaryOperator {
2     int eval(int a, int b);
3 }
```

Implementazione Tradizionale: Somma.java

```
1 public class Somma implements BinaryOperator {
2     @Override
3     public int eval(int a, int b) {
4         return a + b;
5     }
6 }
```

Allo stesso modo dovremmo creare i file `Sottrazione.java`, `Moltiplicazione.java` e `Potenza.java`. Poi, nel motore della calcolatrice, colleghiamo i simboli alle rispettive operazioni usando una `Map`:

Configurazione del Motore con Mappa di Operatori

```
1 Map<String, BinaryOperator> operatori = new HashMap<>();
2 operatori.put("+", new Somma());
3 operatori.put("-", new Sottrazione());
4 operatori.put("*", new Moltiplicazione());
5 operatori.put("^", new Potenza());
6
7 BinaryOperator op = operatori.get("+");
8 System.out.println(op.eval(3, 4)); // Stampa: 7
```

Vantaggi e Difetti di Questo Approccio

Dal punto di vista dell'architettura software, questo schema è formalmente corretto: rispetta il principio *Open/Closed* perché per aggiungere un nuovo operatore (ad esempio il modulo "%") non dobbiamo modificare il codice della calcolatrice, ma solo registrare una nuova coppia chiave-valore.

Tuttavia, il costo di scrittura e manutenzione è davvero esagerato:

- Dobbiamo creare un file sorgente separato per ogni singola operazione di una riga;
- Dobbiamo ripetere ogni volta la struttura `public class ... implements BinaryOperator`;
- Di ogni classe creiamo **un solo oggetto in tutto il ciclo di vita del programma**, che registriamo nella mappa e di cui non useremo mai più il nome specifico;
- Il nome della classe (`Somma`) non serve a nulla nel resto del codice, perché la calcolatrice interagisce solo tramite l'interfaccia `BinaryOperator` e il metodo `eval(int, int)`.

Questo è un chiaro esempio di **boilerplate code**: codice cerimoniale richiesto dalla sintassi del linguaggio, che però non aggiunge alcun reale valore logico.

2.2 Classi Anonime: Sintassi, Uso e Limiti

Per evitare di creare un file separato per ogni classe usata una sola volta, Java 1.1 ha introdotto le **classi anonime** (*anonymous inner classes*).

Definizione: Classe Anonima

Una classe anonima è una classe priva di nome che viene dichiarata e istanziata nello stesso punto del codice in cui serve. La sintassi fonde l'allocazione tramite `new` con la definizione del corpo della classe:

`new Tipo(argomenti) { corpo della classe }`

dove `Tipo` è l'interfaccia da implementare oppure la classe da estendere.

Applicando questo costrutto alla nostra calcolatrice, possiamo scrivere l'operazione di somma direttamente sul posto:

Dichiarazione e Istanziamento di una Classe Anonima

```
1 BinaryOperator somma = new BinaryOperator() {
2     @Override
3     public int eval(int a, int b) {
4         return a + b;
5     }
6 };
```

Come Leggere Questa Sintassi

Questa istruzione può sembrare insolita a prima vista per tre motivi:

1. `new BinaryOperator()`: sembra che stiamo istanziando un'interfaccia (cosa vietata in Java). In realtà stiamo creando un'istanza di una nuova classe anonima che *implementa* quell'interfaccia. Le parentesi tonde vuote indicano che stiamo chiamando il costruttore predefinito di `Object`.
2. Le parentesi graffe `{ ... }` contengono il corpo della classe, dove possiamo implementare il metodo `eval(int, int)`.
3. Il punto e virgola finale `;` chiude l'istruzione di assegnamento: l'intera espressione produce il riferimento all'oggetto appena creato nello heap.

2.2.1 Caratteristiche e Limitazioni delle Classi Anonime

Una classe anonima è una vera e propria classe Java, ma per il fatto di non avere un nome ha regole ben precise:

Cosa può fare	Spiegazione
Dichiarare campi e metodi ausiliari	Può avere campi d'istanza e metodi privati di supporto, oltre a quelli richiesti dall'interfaccia.
Inizializzare lo stato	Può usare blocchi di inizializzazione (<code>{ ... }</code>) al posto del costruttore.
Accedere al contesto esterno	Può accedere a tutti i membri della classe contenitore e alle variabili locali del metodo circostante.
Cosa NON può fare	Motivazione
Nessun costruttore esplicito	In Java il costruttore deve avere lo stesso nome della classe; senza nome, non si può dichiarare un costruttore.
Nessun riutilizzo	Non avendo un nome, non possiamo invocare di nuovo new per creare un'altra istanza altrove.
Una sola interfaccia o classe base	Non può implementare più interfacce contemporaneamente, né estendere una classe e implementare un'interfaccia insieme.

Tabella 2.1: Capacità e limitazioni pratiche delle classi anonime in Java.

Cosa Succede nel Bytecode

Anche se nel codice sorgente non hanno nome, a livello di file compilati (`.class`) il compilatore assegna alle classi anonime un nome numerato progressivo: `Calcolatrice$1.class`, `Calcolatrice$2.class`, ecc. Questo meccanismo genera file fisici su disco e carica le classi a runtime tramite classloader.

2.2.2 Il Riferimento `this` nelle Classi Anonime

Un dettaglio importante riguarda la parola chiave `this`:

- **Nelle classi anonime:** `this` si riferisce all'istanza della classe anonima stessa. Se vogliamo riferirci all'oggetto della classe esterna, dobbiamo scrivere in modo qualificato: `ClasseEsterna.this`.
- **Nelle lambda expression** (come vedremo a breve): la lambda non crea un nuovo ambito per `this`; all'interno di una lambda, `this` indica sempre l'oggetto della classe esterna in cui ci troviamo.

2.3 Closure e il Modello di Memoria: La Regola di “Effectively Final”

Uno dei punti di forza delle classi definite all'interno di un metodo è la possibilità di **catturare le variabili locali** del metodo stesso (*closure*).

Consideriamo questo metodo che crea un operatore moltiplicato per un fattore di scala k :

Metodo Factory che Cattura una Variabile Locale

```

1 public static BinaryOperator sommaScalata(int k) {
2     return new BinaryOperator() {
3         @Override
4         public int eval(int a, int b) {
5             return k * (a + b); // k e' il parametro del metodo esterno!
6         }
7     };
8 }

```

Se usiamo questo metodo memorizzando l'operatore ottenuto:

Uso Differito dell'Operatore Creato

```

1 BinaryOperator operatoreDoppio = sommaScalata(2);
2 System.out.println(operatoreDoppio.eval(3, 4)); // Calcola: 2 * (3 + 4) = 14

```

Definizione: Closure (Chiusura)

Una **closure** è una funzione (o blocco di codice) che porta con sé l'ambiente in cui è stata creata, conservando l'accesso alle variabili locali del metodo circostante anche dopo che quel metodo ha terminato l'esecuzione.

2.3.1 Cosa Succede in Memoria: La Copia per Valore

Se osserviamo come viene eseguito il codice, notiamo una situazione particolare:

1. La variabile k risiedeva sullo **stack** della chiamata `sommaScalata(2)`;
2. Quando il metodo esegue il **return**, il suo stack frame viene deallocato e distrutto;
3. Più tardi, quando chiamiamo `eval(3, 4)`, il codice riesce comunque a leggere $k = 2$ e calcolare 14. Come fa?

La JVM risolve questo dilemma con una **copia per valore**: durante la creazione dell'oggetto nello heap, il compilatore crea un campo privato nascosto (ad esempio `final int val$k`) all'interno dell'istanza e vi copia dentro il valore di k .

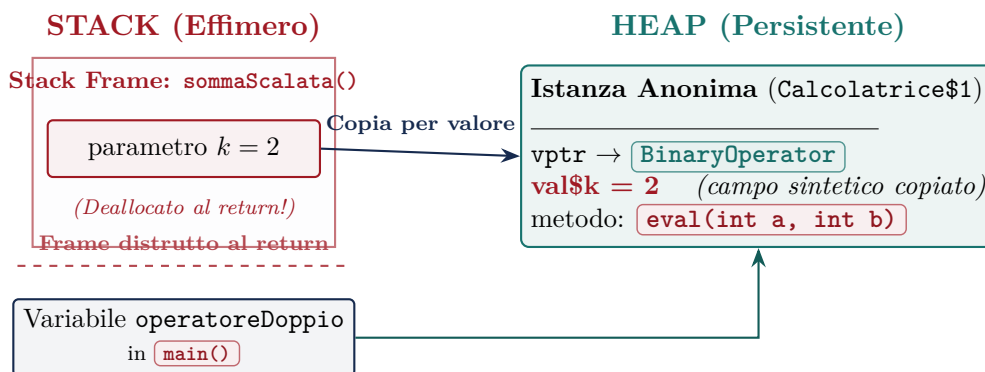


Figura 2.1: Modello di memoria della Closure: la variabile locale dello stack viene copiata in un campo privato dell'oggetto nello heap.

2.3.2 Perché le Variabili Devono Essere “Effectively Final”

Dato che la JVM lavora su una copia, capiamo subito il perché di una regola fondamentale di Java:

Definizione: Variabile Effectively Final

Una variabile locale (o parametro) si dice **effectively final** se dopo la sua inizializzazione non viene mai più modificata. In pratica, se aggiungessimo la parola chiave **final** prima del suo tipo, il programma continuerebbe a compilare senza errori.

Principio Architetture: Regola sulle Variabili Catturate

Una classe anonima o una lambda expression può accedere alle variabili locali del metodo circostante solo se queste sono dichiarate **final** oppure sono **effectively final**.

Cosa succederebbe se Java permettesse di modificare la variabile? Supponiamo per assurdo che Java consentisse di modificare una variabile locale dopo averla catturata:

Esempio Errato: Violazione di Effectively Final

```

1  int contatore = 0;
2  Runnable r = new Runnable() {
3      @Override
4      public void run() {
5          contatore++; // ERRORE DI COMPILAZIONE: contatore non e' effectively final!
6      }
7  };
8  contatore = 10;
    
```

Se questo codice funzionasse, esisterebbero **due copie separate di contatore**:

- La variabile locale originale sullo stack del metodo;
- Il campo clonato `val$contatore` dentro l'oggetto nello heap.

Se il metodo scrivesse `contatore = 10`, la copia nello heap rimarrebbe ferma a 0. Se la lambda facesse `contatore++`, aggiornerebbe solo la copia nello heap lasciando lo stack invariato. I due valori si di-

sallineerebbero immediatamente. Per evitare queste incongruenze, Java impone che il dato catturato sia immutabile.

Come Condividere uno Stato Modificabile

Se abbiamo davvero bisogno di modificare un valore all'interno di una lambda o classe anonima, possiamo separare il contenitore dal contenuto tramite due pattern comuni:

1. **Array di un solo elemento:** scriviamo `final int[] contatore = {0};` e poi facciamo `contatore[0]++;`. Il riferimento all'array resta immutabile (**final**), mentre la cella al suo interno può cambiare valore liberamente.
2. **Classi atomiche:** usiamo classi dedicate della libreria concorrente come `AtomicInteger` (con il metodo `incrementAndGet()`), che garantiscono mutabilità sicura anche con più thread.

2.4 Interfacce Funzionali e l'Annotazione `@FunctionalInterface`

Prima di passare alla sintassi delle lambda, chiariamo il requisito necessario affinché Java capisca come interpretarle:

Definizione: Interfaccia Funzionale (SAM – Single Abstract Method)

Un'interfaccia si definisce **funzionale** se dichiara **esattamente un unico metodo astratto**. Nel calcolo di questo conteggio:

- I metodi con implementazione **default** non contano;
- I metodi **static** non contano;
- Eventuali dichiarazioni di metodi pubblici di `Object` (come `equals`) non contano, perché ogni classe li eredita già.

Perché serve un solo metodo astratto? Se l'interfaccia ha un solo metodo astratto da implementare, scrivere il codice della classe equivale semplicemente a **definire il corpo di quell'unico metodo**. Il compilatore conosce già il nome del metodo, i parametri attesi e il tipo di ritorno: non serve riscriverli ogni volta.

Interfaccia Funzionale Annotata

```
1 @FunctionalInterface
2 public interface BinaryOperator {
3     int eval(int a, int b);
4 }
```

L'Utilità di `@FunctionalInterface`

L'annotazione non cambia il comportamento del codice a runtime, ma svolge due ruoli protettivi importanti:

1. **Documentazione chiara:** comunica subito a chi legge che l'interfaccia è pensata per essere usata con le lambda.
2. **Controllo del compilatore:** se in futuro qualcuno prova ad aggiungere per sbaglio un secondo metodo astratto all'interfaccia, il compilatore segnala subito un errore, impedendo di rompere le lambda già scritte nel progetto.

2.5 Lambda Expression: Sintassi, Target Typing ed Efficienza

Riprendiamo la classe anonima vista in precedenza:

La Classe Anonima Tradizionale

```
1 BinaryOperator somma = new BinaryOperator() {
2     public int eval(int a, int b) {
3         return a + b;
4     }
5 };
```

Se analizziamo questo codice togliendo tutto ciò che è scontato:

- Il tipo `BinaryOperator` è già scritto a sinistra dell'uguale;
- Il metodo da implementare è per forza `eval`, l'unico disponibile;
- I tipi dei parametri (`int`, `int`) sono già definiti nel contratto;
- Le uniche due cose davvero nuove scritte dallo sviluppatore sono i **nomi dei due argomenti** (`a`, `b`) e la **formula di calcolo** (`a + b`).

Definizione: Lambda Expression

Una **lambda expression** è una funzione anonima scritta in forma compatta:

(parametri) -> corpo

che a runtime produce un'istanza di un'interfaccia funzionale.

Possiamo semplificare la classe anonima in due passaggi:

I Due Formati delle Lambda

```
1 // 1. Forma con blocco (Block Lambda con graffe e return esplicito):
2 BinaryOperator sommaBlocco = (a, b) -> { return a + b; };
3
4 // 2. Forma a espressione singola (Expression Lambda, senza graffe ne' return):
5 BinaryOperator somma = (a, b) -> a + b;
```

2.5.1 Guida Rapida alla Sintassi delle Lambda

In base al numero di parametri e alle istruzioni da eseguire, la sintassi si adatta per essere il più leggibile possibile:

- **Nessun parametro:** usiamo parentesi tonde vuote obbligatorie: `() -> 42`;
- **Un solo parametro:** possiamo omettere le parentesi tonde: `x -> x * 2`;
- **Due o più parametri:** le parentesi tonde sono obbligatorie: `(x, y) -> x + y`;
- **Tipi espliciti:** se vogliamo indicare i tipi per chiarezza, dobbiamo specificarli per tutti i parametri: `(int x, int y) -> x + y`;
- **Più istruzioni:** se il corpo contiene più righe, usiamo graffe e `return` esplicito: `x -> { log(x); return x * 2; }`.

2.5.2 Come il Compilatore Capisce il Tipo: Il Target Typing

Se scriviamo semplicemente l'espressione isolata `(a, b) -> a + b`, che tipo ha in Java?

Principio Architeturale: Principio di Target Typing

In Java, una lambda expression **non ha un tipo proprio autonomo**. Il suo tipo viene dedotto dal contesto in cui si trova, cioè dal tipo dell'interfaccia funzionale attesa in quel punto (**Target Typing**).

La stessa identica espressione `(a, b) -> a + b` può assumere tipi diversi a seconda dell'assegnamento:

Stessa Lambda, Tipi Bersaglio Differenti

```
1 // Assume il tipo della nostra interfaccia custom (metodo astratto: eval)
2 BinaryOperator op1 = (a, b) -> a + b;
3
4 // Assume il tipo standard di java.util.function (metodo astratto: applyAsInt)
5 IntBinaryOperator op2 = (a, b) -> a + b;
```

Da questo principio derivano due conseguenze pratiche:

1. **Non possiamo usare `var` da solo con una lambda:** Scrivere `var f = (a, b) -> a + b`; genera un **errore di compilazione**. Con `var` chiediamo al compilatore di dedurre il tipo dall'espressione a destra; ma l'espressione a destra non ha un tipo proprio e aspetta di riceverlo da sinistra. Senza un punto di riferimento, la compilazione fallisce.

2. **Serve un metodo astratto univoco:** Se un'interfaccia avesse due metodi astratti con la stessa firma, il compilatore non saprebbe quale dei due stiamo implementando con la lambda.

2.5.3 Differenze Sostanziali tra Lambda e Classi Anonime

Una lambda non è solo una classe anonima con una sintassi più corta:

1. **Comportamento di `this`:** Dentro una lambda, `this` fa riferimento all'oggetto della classe esterna in cui ci troviamo. Non crea un nuovo livello di scoping.
2. **Nessun mascheramento di variabili:** I parametri della lambda non possono avere lo stesso nome di variabili locali già presenti nel metodo (altrimenti il compilatore segnala un errore di nome duplicato). Una classe anonima, invece, crea un vero scope separato e può mascherare i nomi esterni.
3. **Prestazioni ed efficienza (Bytecode):** Le classi anonime creano un file `.class` su disco per ogni classe e allocano sempre nuovi oggetti nello heap. Le lambda expression, invece, usano l'istruzione speciale di bytecode `invokedynamic` (introdotta da Java 7/8): la classe di supporto viene generata a runtime solo al primo utilizzo in modo molto più leggero ed efficiente.

2.6 Method Reference: Le Quattro Categorie Canoniche

Molto spesso il corpo di una lambda si limita a chiamare un metodo già esistente, passando semplicemente gli argomenti ricevuti:

Lambda di Semplice Inoltro

```
1 Consumer<String> stampa = s -> System.out.println(s);
```

In questi casi la lambda non fa nessun calcolo: fa solo da intermediario. Con i **Method Reference** possiamo fare a meno di riscrivere i parametri, indicando direttamente il nome del metodo preesistente.

Definizione: Method Reference

Un **method reference** è una sintassi compatta nella forma `qualificatore::nomeMetodo` che punta a un metodo o costruttore esistente. Quando la funzione viene eseguita, i parametri attesi dall'interfaccia funzionale vengono inoltrati automaticamente al metodo indicato.

La riga precedente diventa quindi:

Sostituzione con Method Reference

```
1 Consumer<String> stampa = System.out::println;
```

2.6.1 Le Quattro Forme di Method Reference

I method reference si dividono in quattro categorie a seconda di cosa scriviamo a sinistra dei doppi punti (`::`):

- **1. Metodo d'istanza su un oggetto specifico (`oggetto::metodo`)**

L'oggetto su cui chiamare il metodo è già noto e fissato in anticipo. Tutti i parametri della lambda diventano gli argomenti del metodo.

Esempio: `System.out::println` $\equiv$ `x -> System.out.println(x)`

- **2. Metodo statico di una classe (`Classe::metodoStatico`)**

Non c'è nessun oggetto su cui chiamare il metodo; i parametri della lambda vengono passati direttamente come argomenti al metodo statico.

Esempi: `Math::max` $\equiv$ `(a, b) -> Math.max(a, b);`

`Integer::sum` $\equiv$ `(a, b) -> Integer.sum(a, b)`

- **3. Metodo d'istanza su un oggetto arbitrario (`Classe::metodoDiIstanza`)**

A sinistra scriviamo il nome della classe, ma il metodo referenziato è un metodo d'istanza.

Come funziona: il **primo parametro** della lambda funge da bersaglio (*receiver*) su cui viene chiamato il metodo, mentre gli altri parametri diventano gli argomenti della chiamata.

Esempio con un parametro: `String::trim` $\equiv$ `s -> s.trim()`

Esempio con due parametri: `String::compareTo` $\equiv$ `(s, t) -> s.compareTo(t)`

• 4. Riferimento a costruttore (`Classe::new`)

Punta al costruttore della classe per creare una nuova istanza al momento opportuno.

Esempio: `ArrayList::new` $\equiv$ `() -> new ArrayList<>()`

Consultazione Rapida delle Sintassi

Per un promemoria rapido di tutte le quattro categorie con le relative equivalenze lambda prima dell'esame o del laboratorio, consulta la **Scheda dei Method Reference in Appendice B.2**.

Capire a Fondo la Categoria 3: Il Primo Parametro come Ricevente

La terza categoria è utilissima ogni volta che dobbiamo elaborare oggetti di una collezione. Ad esempio, quando dobbiamo confrontare due stringhe in un `Comparator<String>`, il comparatore riceve due stringhe (s, t). Scrivendo semplicemente:

`String::compareTo`

il compilatore capisce che la prima stringa s è l'oggetto su cui chiamare il metodo, e la seconda stringa t è il parametro da passargli: (s, t) $\rightarrow$ `s.compareTo(t)`. Il codice diventa pulitissimo e si legge in modo naturale.

2.7 I Quattro Livelli di Astrazione a Confronto

Mettiamo ora a confronto i quattro modi che Java ci offre per realizzare la stessa identica operazione di somma all'interno del motore della calcolatrice:

La Scala Evolutiva: Quattro Modi per Scrivere la Stessa Azione

```

1 // 1. Classe concreta separata (Java 1.0)
2 operatori.put("+", new Somma());
3
4 // 2. Classe anonima scritta sul posto (Java 1.1)
5 operatori.put("+", new BinaryOperator() {
6     @Override
7     public int eval(int a, int b) { return a + b; }
8 });
9
10 // 3. Lambda expression (Java 8)
11 operatori.put("+", (a, b) -> a + b);
12
13 // 4. Method reference (Java 8)
14 operatori.put("+", Integer::sum);
    
```

OOP 1.0

1. Classe con Nome (`new Somma()`):

File sorgente separato, utile se la classe serve in molti punti del sistema.

Java 1.1

2. Classe Anonima (`new BinaryOperator() { ... }`):

Dichiarazione sul posto, per un'unica istanza che deve mantenere uno stato complesso.

Java 8

3. Lambda Expression (`(a, b) -> a + b`):

Elimina il codice ripetitivo: ideale per formule, filtri e calcoli veloci.

Java 8+

4. Method Reference (`Integer::sum`):

Massima chiarezza: punta direttamente a un metodo già pronto e testato.

Meno Boilerplate
Più Chiarezza
e Leggibilità

Figura 2.2: I quattro livelli di astrazione per implementare un comportamento in Java.

Principio Architeturale: Quale Forma Scegliere nella Pratica?

Per orientarsi nella scrittura del codice quotidiano:

1. **Scegli il Method Reference** quando il codice deve solo chiamare un metodo già esistente senza logica in più;
2. **Scegli la Lambda Expression** (meglio se su una sola riga senza graffe) quando devi fare un calcolo, combinare valori o applicare un piccolo filtro;
3. **Scegli la Classe Anonima** quando l'interfaccia ha più di un metodo astratto (non è SAM), oppure se devi gestire campi interni mutabili o ridefinire metodi di `Object`;
4. **Scegli una Classe Ordinaria con Nome** quando la classe deve essere riutilizzata in più parti del programma o fa parte dell'architettura pubblica dell'applicazione.

2.8 Iniezione di Comportamento e Inversione del Controllo

Oltre a scrivere meno codice, il vero salto di qualità delle lambda è l'**Iniezione di Comportamento** (*Behavior Parameterization*): la possibilità di passare un algoritmo o un'azione come parametro a un altro metodo.

Guardiamo questi due metodi:

Due Metodi con la Stessa Struttura di Ciclo

```

1 static void stampaInteri(int n) {
2     for (int i = 0; i < n; i++)
3         System.out.println(i);
4 }
5
6 static void stampaPari(int n) {
7     for (int i = 0; i < n; i++)
8         if (i % 2 == 0) System.out.println("pari: " + i);
9 }

```

La struttura del ciclo è identica: contiamo da 0 a $n - 1$. L'unica cosa che cambia è *cosa facciamo dentro al ciclo ad ogni passaggio*. Nel codice tradizionale non potevamo unificare questi due metodi, perché potevamo passare come argomenti solo **dati**, non **azioni**.

Con un'interfaccia funzionale possiamo invece passare l'azione come argomento:

Unico Metodo che Riceve l'Azione da Eseguire

```

1 static void ripeti(int n, IntConsumer azione) {
2     for (int i = 0; i < n; i++)
3         azione.accept(i);
4 }

```

Ora i due metodi precedenti si scrivono in una riga:

Iniezione Diretta dell'Azione

```

1 ripeti(5, i -> System.out.println(i));
2 ripeti(5, i -> { if (i % 2 == 0) System.out.println("pari: " + i); });

```

I Cinque Scenari Tipici dell'Esecuzione Differita

Passare una funzione a un metodo non significa eseguirla all'istante: significa consegnare un blocco di codice a qualcun altro perché lo esegua **al momento giusto, in futuro e potenzialmente più volte**. Questo meccanismo è alla base di tutto il Java moderno:

1. **Esecuzione ripetuta**: metodi come `ripeti(n, azione)`;
2. **Collezioni e Stream**: passare un criterio di ordinamento a `list.sort()`, o un filtro a una pipeline di Stream;
3. **Eventi grafici o di rete**: collegare un'azione a un bottone (`bottone.addActionListener(e -> ...)`) che scatterà solo quando l'utente fa click;

4. **Valutazione ritardata (*Lazy Evaluation*)**: calcolare un messaggio di log pesante solo se il livello di log (es. `DEBUG`) è effettivamente attivo;
5. **Programmazione concorrente**: avviare un task asincrono in background su un nuovo thread con `new Thread(() -> calcolo()).start()`.

2.9 Il Package `java.util.function` e la Composizione

Per evitare che ogni sviluppatore debba creare da zero una propria interfaccia per ogni combinazione di parametri, la libreria standard di Java offre nel package `java.util.function` un insieme di interfacce pronte all'uso.

2.9.1 I Cinque Ruoli Fondamentali

Le interfacce standard si organizzano attorno a cinque concetti base:

- **`Supplier<T>` (Fornitore / Produttore)**
Non riceve nessun parametro e produce un valore di tipo `T` tramite il metodo `get()`.
Uso tipico: fabbriche di oggetti, generatori casuali: `() -> new Random()`.
- **`Consumer<T>` (Consumatore)**
Riceve un elemento di tipo `T`, compie un'azione (effetto collaterale) e non restituisce nulla (`void`) tramite il metodo `accept(t)`.
Uso tipico: operazioni di stampa o salvataggio su database: `s -> System.out.println(s)`.
- **`Function<T, R>` (Trasformatore / Mappatore)**
Riceve un valore di tipo `T` e lo converte in un valore di tipo `R` tramite il metodo `apply(t)`.
Uso tipico: estrazione di proprietà, conversioni di tipo: `s -> s.length()`.
- **`Predicate<T>` (Verificatore / Filtro)**
Riceve un elemento di tipo `T` e restituisce un booleano tramite il metodo `test(t)`.
Uso tipico: condizioni di ricerca o filtri su liste: `s -> s.isEmpty()`.
- **`UnaryOperator<T>` e `BinaryOperator<T>` (Operatori Omogenei)**
Casi particolari di funzioni in cui i parametri e il valore di ritorno appartengono tutti allo stesso tipo `T`:
 - **`UnaryOperator<T>`**: prende un `T` e restituisce un `T` (`s -> s.toUpperCase()`);
 - **`BinaryOperator<T>`**: prende due elementi (`T, T`) e restituisce un `T` (`(a, b) -> a + b`).

Esistono inoltre le versioni a due argomenti: `BiFunction<T, U, R>`, `BiConsumer<T, U>` e `BiPredicate<T, U>`.

Scheda di Consultazione Completa del Formulario

La lista completa di tutte le interfacce del package, comprese le specializzazioni per tipi primitivi e tutti i metodi default di composizione, è disponibile in **Appendice B.2 (Catalogo Completo di `java.util.function`)**.

2.9.2 Tipi Primitivi e Zero-Boxing Overhead

In Java i generici funzionano solo con le classi (`Object`). Se usassimo `Function<Integer, Integer>`, la JVM dovrebbe continuamente trasformare i tipi da `int` a `Integer` (*autoboxing*) e viceversa (*unboxing*), creando decine di piccoli oggetti temporanei nello heap e rallentando il programma.

Per questo motivo, la libreria fornisce versioni dedicate ai tipi primitivi (`int`, `long`, `double`):

- `IntConsumer`, `LongConsumer`, `DoubleConsumer`;
- `IntPredicate`, `LongPredicate`, `DoublePredicate`;
- `IntFunction<R>`, `ToIntFunction<T>`;
- `IntUnaryOperator`, `IntBinaryOperator`.

La nostra interfaccia didattica con firma `int eval(int a, int b)` corrisponde alla classe standard di libreria `IntBinaryOperator`.

Attenzione ai Nomi Duplicati

Se nel codice importiamo tutto il package con `import java.util.function.*;`, il tipo standard `BinaryOperator` entra in conflitto con la nostra interfaccia didattica con lo stesso nome. Nei progetti reali basta usare il nome completo oppure usare direttamente `IntBinaryOperator`.

2.9.3 Comporre Funzioni e Predicati

Le interfacce funzionali standard non si limitano al loro metodo principale: mettono a disposizione comodi metodi per collegare più funzioni insieme in una catena logica:

Composizione di Predicati e Funzioni

```

1 // 1. Composizione di Predicati (AND, OR, NOT)
2 Predicate<String> nonVuota = ((Predicate<String>) String::isEmpty).negate();
3 Predicate<String> corta = s -> s.length() < 5;
4 Predicate<String> valida = nonVuota.and(corta); // Deve essere non vuota E corta
5
6 // 2. Concatenazione di Funzioni con andThen
7 Function<String, String> pulisciSpazi = String::trim;
8 Function<String, Integer> contaLettere = String::length;
9 Function<String, Integer> elabora = pulisciSpazi.andThen(contaLettere);
10
11 System.out.println(elabora.apply("    ciao mondo    ")); // Stampa: 10

```

Con `andThen` la seconda operazione viene eseguita dopo la prima, consentendo di costruire passaggi di trasformazione lineari e chiari.

2.10 Il Motore della Calcolatrice Riscritto e Completo

Tornando al problema iniziale della calcolatrice, vediamo come diventa compatto ed elegante il codice combinando lambda e method reference:

Motore della Calcolatrice: Motore.java

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class Motore {
5     private final Map<String, BinaryOperator> operatori = new HashMap<>();
6
7     public Motore() {
8         // Registrazione semplice e immediata degli operatori
9         operatori.put("+", (a, b) -> a + b);
10        operatori.put("-", (a, b) -> a - b);
11        operatori.put("*", (a, b) -> a * b);
12        operatori.put("^", (a, b) -> (int) Math.pow(a, b));
13        operatori.put("max", Math::max); // Method reference statico!
14    }
15
16    public int calcola(String simbolo, int a, int b) {
17        BinaryOperator op = operatori.get(simbolo);
18        if (op == null) {
19            throw new IllegalArgumentException("Operatore sconosciuto: " + simbolo);
20        }
21        return op.eval(a, b);
22    }
23 }

```

Abbiamo mantenuto tutti i vantaggi architetturali della mappa di operatori, ma abbiamo azzerato il codice inutile: non ci sono più file sparsi per ogni operazione, e aggiungere una nuova funzione richiede una sola riga nel costruttore.

2.11 L'Evoluzione delle Interfacce: Metodi Default, Static e Private

Fino a Java 7, un'interfaccia poteva contenere solo firme di metodi astratti (senza corpo) e costanti. A partire da Java 8 e 9, le interfacce si sono arricchite con tre nuovi strumenti:

Elemento	Introdotta	Scopo Principale	Regola
Metodi Astratti	Da sempre	Definiscono il contratto che le classi devono rispettare.	Obbligatori da implementare.
Metodi default	Java 8	Forniscono un'implementazione già pronta ereditabile dalle classi.	Ereditati automaticamente, sovrascrivibili.
Metodi static	Java 8	Metodi di supporto o factory legati al tipo dell'interfaccia.	Chiamati tramite <code>Interfaccia.metodo()</code> .
Metodi private	Java 9	Condividono codice comune tra più metodi default interni.	Invisibili all'esterno dell'interfaccia.

Tabella 2.2: Gli elementi di un'interfaccia in Java moderno.

2.11.1 Perché Sono Nati i Metodi Default

I metodi **default** sono stati introdotti principalmente per garantire la **retrocompatibilità** delle librerie esistenti.

Quando con Java 8 è nata la libreria degli Stream, gli ingegneri volevano che tutte le collezioni potessero usare il metodo `forEach(Consumer)`. Tuttavia, `forEach` doveva essere aggiunto all'interfaccia base `Iterable`. Se fosse stato aggiunto come normale metodo astratto, **milioni di classi già scritte in tutto il mondo che implementavano `Iterable` avrebbero smesso istantaneamente di compilare**. Grazie alla parola chiave **default**, l'interfaccia ha potuto fornire un'implementazione standard con un normale ciclo `for`; tutte le classi esistenti hanno così ereditato il nuovo metodo senza dover modificare una sola riga di codice.

Esempio Completo di Interfaccia con Default, Static e Private

```

1 public interface Shape2d {
2     double area();
3     double perimeter();
4
5     // Metodo statico: factory per creare un quadrato (Java 8)
6     static Shape2d square(double side) {
7         return new Rectangle(side, side);
8     }
9
10    // Metodo default: ereditato automaticamente da tutte le forme (Java 8)
11    default String describe() {
12        return name() + " | area=" + area() + " | perimetro=" + perimeter();
13    }
14
15    // Metodo privato: supporto interno condiviso (Java 9)
16    private String name() {
17        return getClass().getSimpleName();
18    }
19 }
```

Qualsiasi classe che implementa `Shape2d` (come `Rectangle` o `Circle`) riceve subito il metodo `describe()` senza sforzo, pur mantenendo la libertà di farne l'override se vuole personalizzarlo.

2.12 Risoluzione dei Conflitti tra Metodi Default

Dato che una classe Java può implementare più interfacce, e ciascuna interfaccia può contenere metodi **default**, sorge spontanea una domanda: *cosa succede se due interfacce hanno un metodo default con lo stesso nome e gli stessi parametri?*

Due Interfacce con lo Stesso Metodo Default

```

1 interface Drawable {
2     default String describe() { return "Entità Disegnabile"; }
3 }
```

```

4
5 interface Shape2d {
6     default String describe() { return "Forma Geometrica 2D"; }
7 }
    
```

Se una classe le implementa entrambe:

```

1 class Circle implements Shape2d, Drawable {
2     // ERRORE DI COMPILAZIONE SE NON SCEGLIAMO QUALE USARE!
3 }
    
```

Conflitto da Risolvere

2.12.1 Le Tre Regole della JVM per Risolvere i Conflitti

Per evitare ogni ambiguità, il compilatore Java applica tre regole in ordine di priorità:

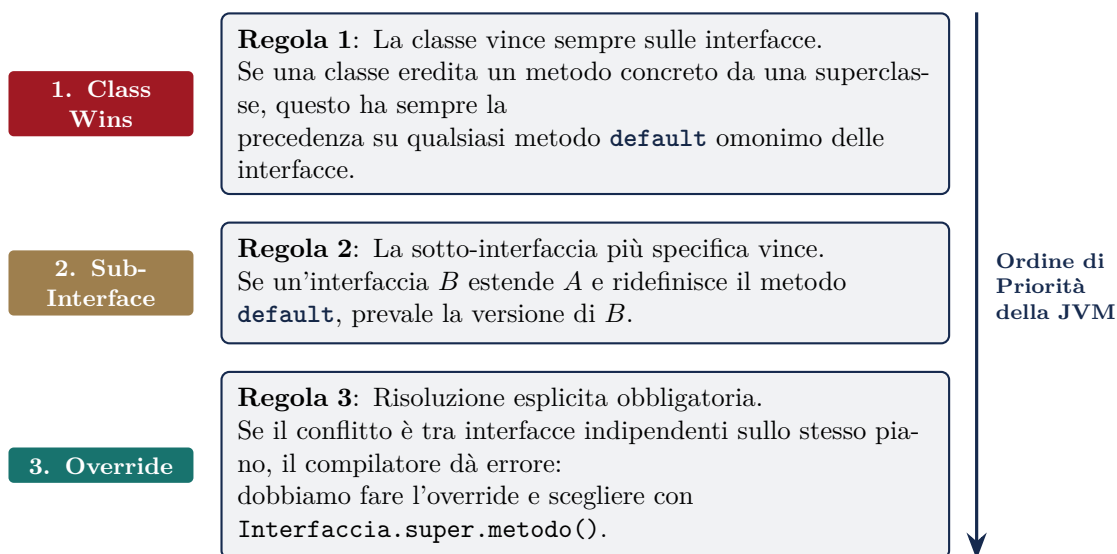


Figura 2.3: Le tre regole per risolvere i conflitti di metodi default in Java.

Nel nostro esempio con `Circle`, le due interfacce sono indipendenti (Regola 3). Dobbiamo quindi scrivere noi l'override:

```

1 class Circle implements Shape2d, Drawable {
2     @Override
3     public String describe() {
4         // Possiamo scegliere una delle due, oppure combinarle entrambe
5         return Shape2d.super.describe() + " (anche " + Drawable.super.describe() +
6         ")";
7     }
8 }
    
```

Risoluzione Esplicita nel Cerchio

2.12.2 Metodi Statici e Metodi Privati nelle Interfacce (Java 8 e Java 9)

L'evoluzione delle interfacce verso componenti modulari sempre più ricchi è stata completata da due ulteriori introduzioni:

- **Metodi Statici (da Java 8):** consentono di definire metodi di utilità e *factory method* direttamente all'interno dell'interfaccia correlata, senza dover istanziare classi helper esterne con costruttore privato (come la classica classe `Collections` affiancata a `Collection`). Un metodo statico in un'interfaccia:
 - Non fa parte dell'istanza e **non viene ereditato** dalle classi che implementano l'interfaccia;
 - Deve essere invocato usando esplicitamente il nome dell'interfaccia: `Shape2d.crea Rettangolo(w, h)`.
- **Metodi Privati (da Java 9):** quando più metodi **default** all'interno della stessa interfaccia necessitano di condividere frammenti complessi di logica algoritmica o controlli di validazione degli argomenti, si

creava in passato il dilemma di dover duplicare il codice oppure esporre un metodo ausiliario come pubblico nell'API contrattuale. Da Java 9, un'interfaccia può dichiarare metodi **private** (e **private static**), visibili unicamente all'interno dell'interfaccia stessa.

Metodi Default, Statici e Privati in un'Interfaccia (OperatoreNumerico.java)

```

1 package it.unibs.eps;
2
3 public interface OperatoreNumerico {
4     double calcola(double x);
5
6     // Metodo default pubblico che sfrutta un metodo privato
7     default double calcolaConLog(double x) {
8         validaInput(x);
9         double res = calcola(x);
10        logOperazione("calcola", x, res);
11        return res;
12    }
13
14    // Metodo statico factory pubblico
15    static OperatoreNumerico identita() {
16        return x -> x;
17    }
18
19    // Metodo privato di istanza (Java 9): fattorizza la validazione
20    private void validaInput(double x) {
21        if (Double.isNaN(x) || Double.isInfinite(x)) {
22            throw new IllegalArgumentException("Input non valido: " + x);
23        }
24    }
25
26    // Metodo privato statico (Java 9): ausiliario non esposto all'esterno
27    private static void logOperazione(String op, double in, double out) {
28        System.out.printf("[LOG %s] in=%.2f -> out=%.2f\n", op, in, out);
29    }
30 }
    
```

2.13 Contratti, Classi Astratte e i Due Volti dell'Ordinamento

2.13.1 Interfaccia o Classe Astratta? Differenze e Scelte Architettureali

Nel capitolo precedente abbiamo definito l'interfaccia `Shape2d` per modellare figure geometriche aventi un'area e un perimetro. Un'interfaccia, nella sua forma fondamentale, rappresenta un **contratto puro**: un elenco formale di metodi che una classe promette di implementare. Chi utilizza un riferimento a `Shape2d` ignora volutamente quale classe concreta abbia davanti: è proprio questa *ignoranza voluta* che permette di aggiungere nuove forme al sistema senza modificare una singola riga del codice che le manipola (principio *Open/Closed*). Tuttavia, sorge spontanea una domanda classica della progettazione ad oggetti: *quando conviene usare un'interfaccia e quando invece una classe astratta?*

Classe Astratta con Stato Condiviso: AbstractShape.java

```

1 public abstract class AbstractShape implements Shape2d {
2     private final String name; // Stato condiviso (campo privato)
3
4     protected AbstractShape(String name) {
5         this.name = name;
6     }
7
8     public String name() {
9         return name; // Implementazione concreta condivisa
10    }
11
12    // area() e perimeter() rimangono astratti: ogni forma li calcola a modo suo
13    @Override public abstract double area();
14    @Override public abstract double perimeter();
    
```

```
5 }
```

Principio Architetture: Criterio di Scelta: Contratto vs Stato Condiviso

I due costrutti rispondono a esigenze architetture profondamente distinte:

- **La Classe Astratta** serve quando più classi derivate devono condividere **stato** (campi di istanza) e una radice comune di implementazione (costruttori, metodi di servizio protetti). Poiché in Java l'ereditarietà delle classi è rigorosamente **singola**, una classe può estendere una sola classe astratta.
- **L'Interfaccia** serve per definire una **capacità o protocollo comportamentale** che classi tassonomicamente lontanissime possono possedere (es. `Drawable`, `Comparable`, `AutoCloseable`). Una classe può implementare un numero arbitrario di interfacce.

Regola Aurea di Progettazione: Si definisca *sempre* il contratto fondamentale come interfaccia. Se utile, si affianchi a parte una classe astratta che ne fornisca un'implementazione parziale di supporto (come fa la Java Standard Library con `List` e `AbstractList`).

L'Anti-Pattern delle Costanti in Interfaccia

In Java le variabili dichiarate in un'interfaccia sono implicitamente **public static final**. In passato si usava creare interfacce prive di metodi al solo scopo di raggruppare costanti, per poi "implementarle" nelle classi applicative e avere i nomi a portata di mano. Questo schema (*Constant Interface Pattern*) è oggi considerato un grave anti-pattern architetture: inquina l'API pubblica della classe ed espone dettagli implementativi interni. Le costanti devono risiedere nella classe che le utilizza, in una classe di utilità non istanziabile con costruttore privato, oppure all'interno di un **enum**.

2.13.2 Due Modi di Ordinare: Comparable vs Comparator

Ordinare una lista di forme geometriche richiede di stabilire quando un elemento "viene prima" di un altro. La libreria standard di Java offre due contratti distinti per formalizzare questo concetto:

I Due Contratti Fondamentali di Ordinamento

```
1 // 1. Ordinamento Naturale: implementato dalla classe stessa
2 public interface Comparable<T> {
3     int compareTo(T other); // < 0 se this precede other, 0 se pari, > 0 se this segue
4 }
5
6 // 2. Ordinatore Estrinseco: oggetto esterno che confronta due istanze
7 public interface Comparator<T> {
8     int compare(T a, T b); // < 0 se a precede b, 0 se pari, > 0 se a segue b
9 }
```

- `Comparable<T>` definisce l'**ordinamento naturale** della classe, unico e intrinseco all'oggetto stesso. Una classe che implementa `Comparable<Rectangle>` dichiara qual è il suo criterio primario di confronto (ad esempio l'area):

Implementazione di Comparable

```
1 public class Rectangle implements Shape2d, Comparable<Rectangle> {
2     private final double w, h;
3     public Rectangle(double w, double h) { this.w = w; this.h = h; }
4     public double width() { return w; }
5     public double height() { return h; }
6     @Override public double area() { return w * h; }
7     @Override public double perimeter() { return 2 * (w + h); }
8
9     @Override
10    public int compareTo(Rectangle other) {
11        return Double.compare(this.area(), other.area()); // Ordinamento naturale
12        per area
13    }
14 }
```

- `Comparator<T>` è invece un **oggetto esterno di confronto**: possono esserne quanti se ne vuole (per area crescente, per perimetro decrescente, per larghezza, per nome alfabetico), e vengono scelti al momento dell'invocazione di `sort()`:

Uso di Comparable e Comparator

```

1 List<Rectangle> rs = new ArrayList<> (List.of(new Rectangle(3, 4), new Rectangle(1,
    10)));
2
3 Collections.sort(rs); // Sfrutta l'ordinamento naturale compareTo() (per area)
4
5 // Ordinamento ad-hoc per perimetro tramite lambda:
6 rs.sort((a, b) -> Double.compare(a.perimeter(), b.perimeter()));

```

Definizione: Il Contratto Matematico di un Ordinamento

Qualsiasi implementazione di `compareTo` o `compare` deve soddisfare tassativamente le proprietà matematiche di un ordine totale:

1. **Antisimmetria**: $\text{sgn}(\text{compare}(a, b)) == -\text{sgn}(\text{compare}(b, a))$.
2. **Transitività**: se $\text{compare}(a, b) > 0$ e $\text{compare}(b, c) > 0$, allora deve valere $\text{compare}(a, c) > 0$.
3. **Riflessività / Coerenza**: $\text{compare}(a, a) == 0$ per ogni a .

Coerenza con equals: È vivamente raccomandato che $(\text{compare}(a, b) == 0) \iff a.\text{equals}(b)$. Se questa coerenza viene violata, le collezioni ordinate (`TreeSet`, `TreeMap`), che basano l'unicità degli elementi esclusivamente sul comparatore e ignorano del tutto `equals()`, mostreranno comportamenti anomali rispetto alle collezioni standard (`HashSet`, `HashMap`). Ad esempio, in un `TreeSet` ordinato solo per area, due rettangoli distinti 2×6 e 3×4 verrebbero considerati duplicati e il secondo verrebbe scartato!

Principio Architeturale: L'Errore Fatale della Sottrazione nei Comparatori

Un errore comune tra i programmatori alle prime armi è implementare il confronto tramite sottrazione numerica:

```
return (int) (a.area() - b.area()); // ERRORE GRAVE!
```

Questo approccio fallisce in due modi disastrosi:

1. **Troncamento Floating-Point**: se $a.\text{area}() = 0.5$ e $b.\text{area}() = 0.2$, la differenza 0.3 convertita a `int` viene troncata a 0 , dichiarando i due oggetti uguali pur avendo aree differenti.
2. **Overflow Numerico su Interi**: per i numeri interi, se $a = -2 \cdot 10^9$ e $b = 2 \cdot 10^9$, l'operazione $a - b$ va in underflow positivo, invertendo il segno del confronto e violando l'antisimmetria.

Soluzione Standard: Si utilizzi sempre e solo `Double.compare(a, b)` per numeri reali e `Integer.compare(a, b)` per numeri interi.

2.13.3 Comparator Moderno: Metodi Statici, Combinatori e Null-Safety

Grazie alle lambda e ai metodi default introdotti in Java 8, `Comparator<T>` è diventato il modello di riferimento per le API fluide e componibili della piattaforma Java:

- **Factory Statica** `Comparator.comparing(Function<T, U> keyExtractor)`: estrae la chiave di confronto e costruisce il comparatore senza scrivere lambda complesse;
- **Inversione** `reversed()`: inverte l'ordine corrente in modo dichiarativo;
- **Concatenazione** `thenComparing(Comparator)`: gestisce i criteri secondari a parità del primo;
- **Gestione dei Null**: `Comparator.nullsFirst(comp)` e `Comparator.nullsLast(comp)` per trattare elementi nulli senza sollevare `NullPointerException`.

Esempio Avanzato: Ordinamento Multi-Livello Fluido

```

1 List<Shape2d> forme = new ArrayList<>();
2
3 // Ordinamento su tre livelli:
4 // 1. Prima per area crescente;

```

```

5 // 2. A parita' di area, per perimetro decrescente;
6 // 3. A parita' di perimetro, per nome di classe alfabetico.
7 forme.sort(
8     Comparator.comparing(Shape2d::area)
9                 .thenComparing(Comparator.comparing(Shape2d::perimeter).reversed())
10                .thenComparing(s -> s.getClass().getSimpleName())
11 );

```

2.14 Gerarchie Chiuse: Tipi Sigillati (Sealed) e Switch sui Tipi

Nella programmazione a oggetti classica, le gerarchie sono aperte: chiunque può creare una nuova sottoclasse di una classe non **final**.

Tuttavia, in molti casi reali il numero di varianti possibili è limitato e noto fin dall'inizio (ad esempio i messaggi di un protocollo di rete, le figure geometriche gestite o gli stati di una macchina a stati). Prima di Java 17, l'unico modo per avere un insieme chiuso era usare una **enum**, che però vincola a semplici istanze costanti e non permette alle singole varianti di avere campi e dati differenti.

Con Java 17 arrivano le **classi e interfacce sigillate (sealed)**:

Definizione: Interfaccia Sigillata (Sealed Interface)

Un'interfaccia sigillata dichiara esplicitamente, tramite la parola chiave **permits**, l'elenco tassativo delle sole classi che hanno il permesso di implementarla.

Gerarchia Sigillata con permits

```

1 public sealed interface Shape2d permits Rectangle, Circle, Diamond {
2     double area();
3     double perimeter();
4 }
5
6 // Ogni sottoclasse permessa DEVE dichiararsi in uno di questi tre modi:
7 // 1) final (non puo' essere ulteriormente estesa)
8 // 2) sealed (limita a sua volta le sue sottoclassi)
9 // 3) non-sealed (permette a chiunque di estenderla)
10
11 public final class Rectangle implements Shape2d {
12     private final double w, h;
13     public Rectangle(double w, double h) { this.w = w; this.h = h; }
14     public double width() { return w; }
15     public double height() { return h; }
16     @Override public double area() { return w * h; }
17     @Override public double perimeter() { return 2 * (w + h); }
18 }
19
20 public final class Circle implements Shape2d {
21     private final double r;
22     public Circle(double r) { this.r = r; }
23     public double radius() { return r; }
24     @Override public double area() { return Math.PI * r * r; }
25     @Override public double perimeter() { return 2 * Math.PI * r; }
26 }
27
28 public final class Diamond implements Shape2d {
29     private final double d1, d2;
30     public Diamond(double d1, double d2) { this.d1 = d1; this.d2 = d2; }
31     public double d1() { return d1; }
32     public double d2() { return d2; }
33     @Override public double area() { return (d1 * d2) / 2.0; }
34     @Override public double perimeter() { return 2.0 * Math.hypot(d1, d2); }
35 }

```

2.14.1 Pattern Matching e Switch Esaustivo (Java 21)

Il grande vantaggio dei tipi sigillati si apprezza nello **switch sui tipi** introdotto in Java 21.

Poiché il compilatore conosce in anticipo tutte le implementazioni ammesse, può verificare che il nostro `switch` le gestisca tutte (**esaustività**):

Switch sui Tipi Senza Ramo Default

```
1 public static String formattaFigura(Shape2d s) {
2     return switch (s) {
3         case Rectangle r -> "Rettangolo " + r.width() + "x" + r.height();
4         case Circle c      -> "Cerchio di raggio " + c.radius();
5         case Diamond d     -> "Rombo con diagonali " + d.d1() + "x" + d.d2();
6     }; // NESSUN DEFAULT NECESSARIO: copre tutti i casi possibili!
7 }
```

Pattern Matching con Guardie Condizionali (when) in Java 21

```
1 public static String formattaFiguraConGuardia(Shape2d s) {
2     return switch (s) {
3         // La clausola when raffina il caso senza rompere l'esaustività:
4         case Rectangle r when r.width() == r.height() ->
5             "Quadrato perfetto speciale di lato " + r.width();
6         case Rectangle r ->
7             "Rettangolo ordinario " + r.width() + "x" + r.height();
8         case Circle c when c.radius() > 10.0 ->
9             "Grande cerchio di raggio " + c.radius();
10        case Circle c ->
11            "Cerchio ordinario di raggio " + c.radius();
12        case Diamond d ->
13            "Rombo con diagonali " + d.d1() + "x" + d.d2();
14    };
15 }
```

Perché Non Mettere il Ramo Default è una Sicurezza

L'assenza della clausola `default` è una garanzia preziosa:

- Se mettessimo un ramo `default`, aggiungendo in futuro una nuova forma (es. `Triangle`), il programma continuerebbe a compilare e la nuova forma finirebbe zitta nel ramo generico, magari con bug invisibili;
- Senza ramo `default`, appena aggiungiamo `Triangle` alla clausola `permits`, il compilatore ci segnala subito tutti i punti dell'applicazione dove dobbiamo gestire la nuova forma, impedendo dimenticanze.

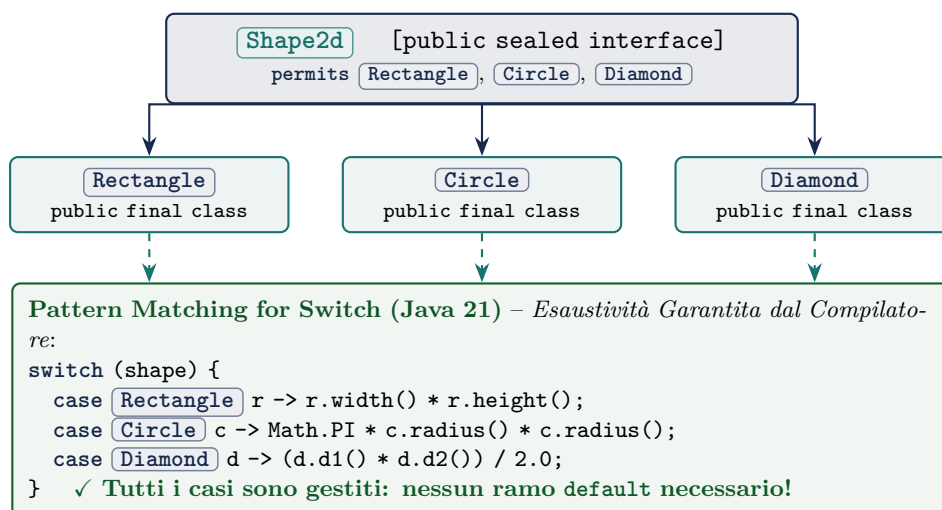


Figura 2.4: Gerarchia sigillata (**sealed**) e pattern matching: il compilatore verifica che tutti i casi siano gestiti senza clausola di fallback.

2.14.2 Confronto Sintetico: Gerarchie Aperte vs Gerarchie Chiuse

Possiamo riassumere le differenze tra i due approcci:

Caratteristica	Gerarchia Aperta (Polimorfismo Tradizionale)	Gerarchia Chiusa (Sealed / Tipi Algebrici)
Chi aggiunge tipi?	Chiunque, in qualsiasi punto o libreria esterna.	Solo chi controlla e dichiara l'interfaccia base con permits .
Aggiungere un'operazione	Richiede di aggiungere il metodo all'interfaccia e modificarne tutte le classi.	Molto facile: basta scrivere una funzione con uno switch sui tipi.
Aggiungere un nuovo tipo	Molto facile: basta implementare l'interfaccia senza toccare le classi esistenti.	Richiede di aggiornare la clausola permits e tutti gli switch dell'applicazione.
Quando usarla?	Per plugin, librerie estensibili e framework aperti.	Per insiemi chiusi: messaggi di rete, comandi, stati di un protocollo, token di un parser.

Tabella 2.3: Confronto pratico tra gerarchie aperte e gerarchie sigillate.

Consultazione Esercizi di Laboratorio sul Modulo 2

Per consolidare l'uso delle classi anonime, il refactoring funzionale verso lambda expression e method reference, la composizione fluida di comparatori null-safe e le gerarchie sigillate con pattern matching, si consultino gli esercizi dettagliati nell'**Appendice A**.

Capitolo 3

Stream Computazionali: Pipeline, Valutazione Pigra e Collettori

In questo capitolo esploriamo una delle innovazioni più radicali introdotte in Java a partire dalla versione 8: le **Stream API** e il paradigma di elaborazione dichiarativo e funzionale delle sequenze di dati.

Fino a Java 7, qualsiasi operazione di filtraggio, trasformazione o aggregazione su collezioni di dati richiedeva un approccio procedurale rigido, basato su cicli espliciti (`for`, `while`, `for-each`), variabili di stato provvisorie e liste ausiliarie. Questo stile non solo produce decine di righe di codice ripetitivo (*boilerplate code*), ma espone il programmatore a gravi trappole semantiche a tempo di esecuzione, prime fra tutte la modifica concorrente di una collezione durante la sua iterazione.

Con le Stream API, il paradigma si capovolge: il programmatore non descrive più *come* iterare fisicamente sui dati passo dopo passo (iterazione esterna), ma dichiara *cosa* vuole ottenere attraverso una sequenza fluida di trasformazioni (iterazione interna), demandando alla libreria sottostante l'ottimizzazione dell'accesso alla memoria, la valutazione differita (*laziness*) e l'eventuale parallelizzazione trasparente del calcolo.

3.1 Il Problema del Paradigma Procedurale e della Modifica Concorrente

Per comprendere il valore fondante degli stream, partiamo da un problema apparentemente banale: data una lista di stringhe che rappresentano nomi di città o parole del vocabolario, vogliamo selezionare solo quelle che iniziano con una determinata lettera (ad esempio la lettera "C"), convertirle in lettere maiuscole e scartare tutte le altre.

3.1.1 L'Anti-Pattern delle Liste Ausiliarie

Nell'approccio procedurale classico, il programmatore istanzia collezioni temporanee e accumula manualmente i risultati intermedi:

Approccio Procedurale con Liste di Appoggio

```
1 List<String> parole = Arrays.asList("como", "milano", "cremona", "brescia", "cantu",  
2   "bergamo");  
3 // 1. Prima lista di accumulo: filtro per iniziale  
4 List<String> filtrate = new ArrayList<>();  
5 for (String p : parole) {  
6     if (p.startsWith("c")) {  
7         filtrate.add(p);  
8     }  
9 }  
10  
11 // 2. Seconda lista di accumulo: trasformazione in maiuscolo  
12 List<String> maiuscole = new ArrayList<>();  
13 for (String p : filtrate) {  
14     maiuscole.add(p.toUpperCase());  
15 }  
16  
17 System.out.println("Risultato: " + maiuscole);
```

Questo codice presenta diversi difetti strutturali:

1. **Prolissità e rumore visivo:** abbiamo impiegato oltre 15 righe per un'operazione concettualmente immediata. L'intento logico ("filtra per C e rendi maiuscolo") è sepolto sotto dettagli di allocazione memoria (`new ArrayList<>()`), chiamate manuali a `add()` e variabili temporanee di ciclo.
2. **Spreco di memoria e cache:** vengono allocate due strutture dati ridondanti (`filtrate` e `maiuscole`) che vivono nell'heap solo per trasportare elementi da un ciclo al successivo, sovraccaricando inutilmente il Garbage Collector.

3. **Mancanza di componibilità**: se volessimo ordinare il risultato o estrarre solo i primi due elementi, dovremmo annidare altri cicli o invocare metodi statici esterni (`Collections.sort()`), frammentando ulteriormente il flusso logico.

3.1.2 La Trappola della Modifica durante l'Iterazione

Una tentazione frequente per evitare la lista di appoggio è quella di rimuovere direttamente gli elementi indesiderati dalla lista originale all'interno di un ciclo `for-each`:

Errore Fatale: Rimozione in Ciclo `for-each`

```
1 List<String> citta = new ArrayList<>(Arrays.asList("como", "milano", "cremona",
2     "brescia"));
3 // TENTATIVO ERRATO: lancia ConcurrentModificationException a runtime!
4 for (String c : citta) {
5     if (!c.startsWith("c")) {
6         citta.remove(c); // ERRORE CRITICO!
7     }
8 }
```

L'Analogia dei Gradini della Scala

Come sottolineato con enfasi a lezione dal docente, il ciclo `for-each` di Java è un costrutto sintattico (*syntactic sugar*) che la JVM compila istanziando un oggetto `Iterator`. L'iteratore mantiene un cursore interno che punta all'indice corrente e un contatore di modifiche attese (`expectedModCount`).

Se invochiamo `citta.remove()` direttamente sulla collezione anziché sull'iteratore, modifichiamo la struttura sottostante all'insaputa dell'iteratore stesso. Alla successiva chiamata di `it.next()`, l'iteratore rileva una discrepanza tra `modCount` ed `expectedModCount` e lancia immediatamente una `ConcurrentModificationException`.

L'analogia didattica d'aula è memorabile: *“Iterare su una collezione mentre la si modifica strutturalmente è come salire una scala mentre qualcuno, gradino dopo gradino, sega i pioli sotto i vostri piedi. Prima o poi mettete il piede nel vuoto e precipitate.”*

Per cancellare elementi in sicurezza nel mondo procedurale pre-Java 8, bisognava recuperare l'iteratore esplicito e chiamare esclusivamente `it.remove()`, raddoppiando ancora una volta il boilerplate.

3.2 Il Paradigma degli Stream come Flusso Computazionale

Definizione: Stream di Dati (`java.util.stream.Stream<T>`)

Uno **Stream** è una sequenza di elementi emessi da una sorgente dati che supporta operazioni aggregate sequenziali o parallele. A differenza di una collezione, uno stream **non memorizza elementi in memoria**, non possiede indici di accesso casuale e non altera la struttura originaria da cui ha origine (*non-interfering*). Uno stream è un canale computazionale effimero attraverso cui i dati fluiscono, vengono elaborati e infine consumati o raccolti.

Principio Architetture: Differenza Ontologica: Collezione vs Stream

- Una **Collezione** è una struttura dati in memoria che **contiene** elementi. Tutti gli elementi sono già presenti contemporaneamente nell'heap prima che la collezione possa essere interrogata.
- Uno **Stream** è una struttura di computazione che **trasporta e trasforma** elementi su richiesta. I singoli elementi possono essere calcolati al volo (*on-demand*) o prelevati da sorgenti potenzialmente infinite.

3.2.1 L'Analogia delle Palline nel Tubo

Per visualizzare mentalmente il funzionamento di una pipeline di stream, il docente propone la metafora meccanica del tubo e delle palline:

- **La Sorgente (Source)**: è una tramoggia o un serbatoio iniziale (una `List`, un array, un file su disco, un generatore numerico) che lascia cadere le palline nel tubo una alla volta.

- **I Setacci e le Pompe (*Intermediate Operations*)**: all'interno del tubo sono collocate griglie di selezione (`filter()`) che fanno passare solo le palline con determinate caratteristiche scartando le altre, oppure meccanismi di verniciatura o deformazione (`map()`) che trasformano il colore o la forma della pallina mentre questa viaggia.
- **Il Bacino di Raccolta (*Terminal Operation*)**: all'estremità inferiore del tubo c'è un secchio o un misuratore finale. Finché non apriamo la valvola terminale o non posizioniamo il secchio, nessuna pallina cade dalla tramoggia iniziale.

Ecco come il problema iniziale delle città viene risolto con un'unica, elegante pipeline di stream:

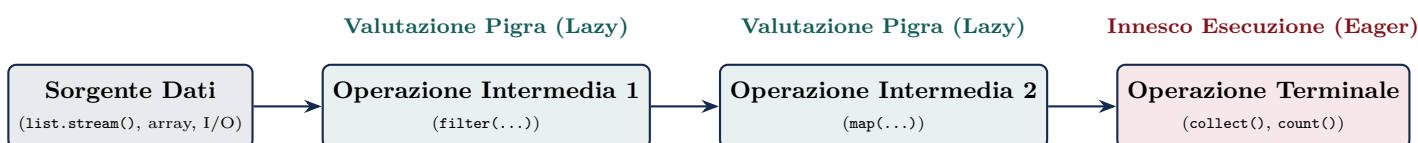
Risoluzione Dichiarativa con le Stream API

```
1 List<String> parole = Arrays.asList("como", "milano", "cremona", "brescia", "cantu",
2     "bergamo");
3 List<String> risultato = parole.stream()
4     .filter(s -> s.startsWith("c"))
5     .map(String::toUpperCase)
6     .toList(); // oppure .collect(Collectors.toList())
7
8 System.out.println("Risultato: " + risultato); // [COMO, CREMONA, CANTU]
```

Non abbiamo creato alcuna lista ausiliaria esplicita, non ci siamo preoccupati di indici o iteratori e il codice si legge esattamente come le specifiche in linguaggio naturale: *“Dalla lista estrai uno stream, filtra gli elementi che iniziano con c, trasformati in maiuscolo e raccoglili in una nuova lista immutabile”*.

3.3 Anatomia di una Pipeline e Valutazione Pigrà (Laziness)

Qualsiasi elaborazione basata su Stream è strutturata rigorosamente come una **pipeline** composta da tre fasi sequenziali:



1. **Sorgente (*Source*)**: fornisce gli elementi iniziali. Può essere finita (una collezione, un array) o teoricamente infinita (generatori algoritmici, stream da socket di rete).
2. **Operazioni Intermedie (*Intermediate Operations*)**: accettano uno stream in ingresso e restituiscono un nuovo stream trasformato. Possono essere concatenate a catena in numero arbitrario (0, 1, 2, ...). Sono sempre **lazy** (pigre): non eseguono alcun calcolo nel momento in cui vengono dichiarate, ma si limitano a costruire un grafo descrittivo della computazione.
3. **Operazione Terminale (*Terminal Operation*)**: chiude la pipeline, innesca la valutazione effettiva dei dati ed emette un risultato scalare (`long`, `boolean`, `Optional`) o una collezione (`List`, `Set`, `Map`), oppure consuma gli elementi per produrre un effetto collaterale (`forEach()`). Una volta invocata l'operazione terminale, lo stream è definitivamente **consumato** e non può più essere utilizzato.

3.3.1 Dimostrazione Sperimentale della Laziness

Per dimostrare l'inerzia assoluta delle operazioni intermedie in assenza di un'operazione terminale, consideriamo il seguente esperimento didattico:

L'Inerzia dello Stream senza Terminale

```
1 List<String> nomi = Arrays.asList("Anna", "Marco", "Giuseppe", "Lucia");
2
3 // Costruzione di una pipeline con filtri e stampe interne
4 nomi.stream()
5     .filter(s -> {
6         System.out.println("FILTRO eseguito su: " + s);
7         return s.length() > 4;
8     })
9     .map(s -> {
```

```

0      System.out.println("MAP eseguita su: " + s);
1      return s.toUpperCase();
2  });
3
4  System.out.println("--- Fine del blocco di codice ---");

```

Cosa stampa a console questo programma? Sorprendentemente, a console comparirà **unicamente**:

```
1 --- Fine del blocco di codice ---
```

Nessuna riga di “FILTRO eseguito” o “MAP eseguita” viene emessa. La JVM ha semplicemente registrato le due funzioni lambda all’interno dei metadati dello stream; poiché nessun consumatore terminale ha richiesto un risultato, l’intera computazione è rimasta inerte. Questo garantisce enormi risparmi prestazionali: se il risultato finale non serve, la CPU non spende un singolo ciclo di clock per elaborare i dati.

3.4 Ordine di Valutazione: Elaborazione Verticale vs Orizzontale

Uno dei fraintendimenti più diffusi tra i programmatori alle prime armi è immaginare che gli stream operino in maniera *orizzontale*: ovvero che prima il filtro venga eseguito su tutti i N elementi della lista creando una collezione filtrata invisibile, e solo dopo la mappatura venga applicata a tutti i superstiti.

In Java, l’elaborazione degli stream avviene di norma in modo rigorosamente **VERTICALE**: ogni singolo elemento prelevato dalla sorgente attraversa *l’intera pipeline in cascata* dall’inizio alla fine prima che il secondo elemento venga estratto dalla sorgente.

3.4.1 Tracciare l’Ordine Reale di Esecuzione

Aggiungiamo ora un’operazione terminale (`findFirst()`) alla nostra pipeline di test ed esaminiamo l’output dei log:

Tracciamento dell’Elaborazione Verticale e Short-Circuit

```

1 List<String> nomi = Arrays.asList("Anna", "Marco", "Giuseppe", "Lucia");
2
3 String trovato = nomi.stream()
4     .filter(s -> {
5         System.out.println("Passo 1: filter su " + s);
6         return s.length() > 4;
7     })
8     .map(s -> {
9         System.out.println("Passo 2: map su " + s);
10        return s.toUpperCase();
11    })
12    .findFirst()
13    .orElse("Nessuno");
14
15 System.out.println("Elemento estratto: " + trovato);

```

L’output generato a console rivela perfettamente il flusso verticale e l’effetto di **short-circuiting** (interruzione anticipata):

```

1 Passo 1: filter su Anna
2 Passo 1: filter su Marco
3 Passo 2: map su Marco
4 Elemento estratto: MARCO

```

Analizziamo cosa è successo:

1. “Anna” entra nel tubo: ha 4 caratteri, il filtro restituisce **false**. “Anna” viene scartata immediatamente e non raggiunge mai la `map()`.
2. “Marco” entra nel tubo: ha 5 caratteri, il filtro restituisce **true**. “Marco” prosegue immediatamente il tragitto verticale ed entra nella `map()`, che lo converte in “MARCO”.
3. “MARCO” raggiunge il terminale `findFirst()`: il terminale cercava il primo elemento valido e lo ha appena trovato.

4. **Chiusura anticipata:** la pipeline si arresta istantaneamente! “Giuseppe” e “Lucia” non vengono nemmeno prelevati dalla lista di partenza.

Principio Architeturale: L'Importanza Critica dell'Ordine dei Metodi Intermedi

Poiché l'elaborazione è verticale, l'ordine di invocazione dei metodi intermedi impatta drasticamente le prestazioni:

- Mettere `filter()` prima di `map()`: la trasformazione costosa viene eseguita *esclusivamente* sugli elementi che soddisfano il predicato.
- Mettere `map()` prima di `filter()`: costringe la CPU a trasformare tutti gli elementi della sorgente, inclusi quelli che verranno buttati via un istante dopo.

3.4.2 Operazioni Stateful e la Rottura della Verticalità

Esistono tuttavia alcune operazioni intermedie speciali che non possono operare in modo puramente verticale. Queste operazioni prendono il nome di **operazioni stateful** (con stato):

- `sorted()`: per ordinare una sequenza di elementi, non è possibile valutare un elemento alla volta isolatamente; è matematicamente necessario accumulare *tutti* gli elementi della sorgente in un buffer interno, confrontarli ed emetterli solo al termine.
- `distinct()`: deve conservare memoria di tutti gli elementi già transitati (solitamente tramite un `HashSet` interno) per scartare i duplicati.

Perché non si può ordinare uno stream infinito

L'operazione intermedia `sorted()` agisce come una diga idroelettrica lungo il tubo: blocca l'intero flusso verticale e attende che l'ultimo elemento sia transitato prima di iniziare a riversare i dati a valle.

Ne consegue una regola insormontabile: **è tassativamente impossibile ordinare uno stream infinito**. Se applichiamo `sorted()` a un generatore infinito (`Stream.iterate()`), la pipeline cercherà di accumulare infiniti elementi nell'heap, esaurendo la memoria della JVM e collassando con un inesorabile `OutOfMemoryError`.

3.5 Il Vincolo di Monouso e la Soluzione con `Supplier<Stream<T>`

Una delle caratteristiche più rigide del modello formale di Java è che **uno Stream può essere consumato una sola volta**:

Definizione: Vincolo di Consumo Unico (*Single-Use Stream*)

Non appena su un'istanza di `Stream` viene invocata un'operazione terminale (`collect()`, `count()`, `forEach()`, `anyMatch()`, ecc.), lo stream viene marcato internamente come chiuso e consumato. Qualsiasi successivo tentativo di invocare operazioni intermedie o terminali sulla medesima istanza scatena immediatamente l'eccezione a tempo di esecuzione:

```
java.lang.IllegalStateException: stream has already been operated upon or closed
```

Errore Fatale: Tentativo di Riuso di uno Stream Consumato

```
1 List<String> lista = Arrays.asList("a", "b", "c");
2 Stream<String> mioStream = lista.stream().filter(s -> !s.isEmpty());
3
4 // 1. Prima operazione terminale: conteggio
5 long conteggio = mioStream.count(); // OK: lo stream viene consumato qui!
6
7 // 2. SECONDA operazione terminale: ERRORE CRITICO A RUNTIME!
8 Optional<String> primo = mioStream.findFirst();
9 // Lancia IllegalStateException!
```

3.5.1 Perché lo Stream non è Riavvolgibile?

A differenza di un nastro magnetico o di una lista indicizzata, lo stream è modellato per consentire il processamento di sorgenti a flusso continuo (come letture da schede di rete, pipeline I/O di file da decine di gigabyte

o generatori stocastici). Mantenere in memoria la cronologia di tutti gli elementi transitati per consentire un eventuale “riavvolgimento” violerebbe il principio di efficienza e causerebbe memory leak massicci.

3.5.2 La Soluzione Architetture: Supplier<Stream<T>

Quando la logica dell'applicazione richiede di rieseguire la medesima pipeline di trasformazione su richiesta (ad esempio per effettuare interrogazioni statistiche multiple o calcoli concorrenti), il pattern raccomandato consiste nell'incapsulare la catena di generazione all'interno di un'interfaccia funzionale `Supplier<Stream<T>`:

Pattern Factory Leggero con Supplier

```
1 List<String> dati = Arrays.asList("Alfa", "Beta", "Gamma", "Delta");
2
3 // Definiamo una factory pura di stream mediante lambda Supplier
4 Supplier<Stream<String>> streamFactory = () -> dati.stream().filter(s -> s.length() >
5     4);
6
7 // 1. Primo utilizzo: calcoliamo il conteggio aprendo un nuovo stream
8 long quanti = streamFactory.get().count();
9 System.out.println("Elementi > 4 caratteri: " + quanti); // 2 (Gamma, Delta)
10
11 // 2. Secondo utilizzo: estraiamo il primo elemento aprendo un ALTRO stream fresco
12 Optional<String> primo = streamFactory.get().findFirst();
13 System.out.println("Primo elemento: " + primo.orElse("Nessuno")); // Gamma
14
15 // 3. Terzo utilizzo: verifichiamo se esiste una stringa con iniziale 'D'
16 boolean esisteD = streamFactory.get().anyMatch(s -> s.startsWith("D"));
17 System.out.println("Esiste iniziale D? " + esisteD); // true
```

Ogni invocazione di `streamFactory.get()` genera una pipeline completamente nuova e vergine a partire dalla sorgente originaria, garantendo piena thread-safety ed eliminando alla radice ogni rischio di `IllegalStateException`.

3.6 Creare Stream da Varie Sorgenti

Java offre una vastissima gamma di metodi factory per istanziare stream a partire da strutture dati preesistenti, tipi primitivi o formule matematiche generatrici.

3.6.1 Da Collezioni e Array

Il punto di partenza naturale per i progetti a oggetti:

- `collezione.stream()`: stream sequenziale da qualsiasi implementazione di `Collection` (`ArrayList`, `HashSet`, ecc.).
- `collezione.parallelStream()`: stream parallelo che partiziona automaticamente il carico sui core della CPU tramite il framework Fork/Join.
- `Arrays.stream(array)`: stream su array di oggetti o tipi primitivi.
- `Stream.of(val1, val2, val3)`: stream a partire da un elenco varargs esplicito di elementi.

3.6.2 Stream Primitivi Specializzati: IntStream, LongStream, DoubleStream

L'utilizzo di `Stream<Integer>` o `Stream<Double>` comporta un sovraccarico considerevole dovuto al fenomeno dell'**autoboxing/unboxing** continuo (conversione tra il tipo primitivo `int` e l'oggetto wrapper `Integer` nell'heap).

Per garantire prestazioni ad altissima efficienza e prevenire allocazioni dinamiche inutili nel percorso critico, Java mette a disposizione tre interfacce primitive specializzate: `IntStream`, `LongStream` e `DoubleStream`.

Utilizzo di IntStream e Statistiche Riassuntive

```
1 // Intervallo di interi da 1 a 100 inclusi
2 int somma = IntStream.rangeClosed(1, 100).sum(); // 5050
3
4 // Calcolo di statistiche complete in un singolo passaggio senza boxing
5 IntSummaryStatistics stats = IntStream.of(12, 45, 68, 2, 99, 34, 51)
6     .summaryStatistics();
7
8 System.out.println("Minimo: " + stats.getMin()); // 2
9 System.out.println("Massimo: " + stats.getMax()); // 99
```

```

0  System.out.println("Media: " + stats.getAverage()); // 44.428...
1  System.out.println("Totale elementi: " + stats.getCount()); // 7
2
3  // Conversione da primitivo a Stream di oggetti (boxing esplicito)
4  List<Integer> listaBoxed = IntStream.range(0, 5)
5      .boxed()
6      .toList();
    
```

3.6.3 Stream Infiniti: Stream.iterate e Stream.generate

Uno stream non deve necessariamente basarsi su dati memorizzati a priori. Possiamo generare sequenze matematiche infinite applicando una funzione unaria iterativa:

Definizione: Metodo Stream.iterate(seed, UnaryOperator)

Genera uno stream ordinato e infinito prodotto dall'applicazione iterativa della funzione a partire da un elemento iniziale (*seed*):

$$\text{Stream} = \{s_0, f(s_0), f(f(s_0)), f(f(f(s_0))), \dots\}$$

Poiché la sequenza è infinita, è obbligatorio troncarla a valle mediante l'operazione intermedia di short-circuit `limit(n)` prima di richiamare un'operazione terminale.

Generazione della Serie di Fibonacci con Stream.iterate

```

1  // Generiamo le coppie consecutive (F_n, F_{n+1}) partendo da (0, 1)
2  // Ogni coppia successiva e': (b, a + b)
3  List<Long> primiFibonacci = Stream.iterate(new long[]{0, 1}, coppia -> new
4      long[]{coppia[1], coppia[0] + coppia[1]})
5      .limit(10) // Limitiamo ai primi 10 elementi
6      .map(coppia -> coppia[0]) // Estraiamo il primo termine della coppia
7      .toList();
8  System.out.println("Primi 10 numeri di Fibonacci: " + primiFibonacci);
9  // Stampa: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    
```

3.7 Riduzioni Avanzate: Le Tre Forme di reduce()

Mentre i metodi speciali come `sum()`, `min()` o `count()` coprono aggregazioni standard, l'operazione terminale generica di aggregazione funzionale è `reduce()` (*fold*). Java fornisce tre versioni sovraccaricate di `reduce()`, ciascuna con specifiche garanzie matematiche:

1. Senza Identità

Forma 1: `Optional<T> reduce(BinaryOperator<T> accumulator)`
Nessun valore identità: se lo stream è vuoto, restituisce `Optional.empty()`.

2. Con Identità

Forma 2: `T reduce(T identity, BinaryOperator<T> accumulator)`
Con elemento neutro *e*: garantisce che per stream vuoto il risultato sia *e*.

3. Con Combiner

Forma 3: `<U> U reduce(U identity, BiFunction<U, ? super T, U> acc, BinaryOperator<U> combiner)`
Cambio di tipo ($T \rightarrow U$): l'accumulatore trasforma gli elementi, il **combiner** fonde i risultati parziali prodotti dai thread concorrenti negli stream paralleli.

Figura 3.1: Le tre forme sovraccaricate del metodo `reduce()` in Java.

Le Tre Forme di reduce() a Confronto (ReduceDemo.java)

```

1  package it.unibs.eps;
2
3  import java.util.List;
4  import java.util.Optional;
5
    
```

```

6 public class ReduceDemo {
7     public static void main(String[] args) {
8         List<Integer> numeri = List.of(2, 3, 4, 5);
9
10        // 1. Forma 1: Prodotto senza identita' -> restituisce Optional
11        Optional<Integer> prodOpt = numeri.stream().reduce((a, b) -> a * b);
12        System.out.println("Prodotto: " + prodOpt.orElse(0)); // 120
13
14        // 2. Forma 2: Somma con identita' (0 e' elemento neutro della somma) ->
15        // restituisce T
16        int somma = numeri.stream().reduce(0, Integer::sum);
17        System.out.println("Somma: " + somma); // 14
18
19        // 3. Forma 3: Cambio di tipo da String (T) a Integer (U) - somma lunghezze
20        List<String> parole = List.of("mela", "pera", "banana");
21        int sommaLunghezze = parole.stream().reduce(
22            0, // Identita' di tipo U
23            (acc, str) -> acc + str.length(), // Accumulator: (U, T) -> U
24            Integer::sum // Combiner: (U, U) -> U
25        );
26        System.out.println("Somma lunghezze: " + sommaLunghezze); // 4 + 4 + 6 = 14
27    }
28 }

```

Principio Architetture: Perché la Terza Forma Richiede Tassativamente il Combiner?

Negli stream sequenziali, il **combiner** non viene mai invocato perché un solo thread elabora gli elementi in serie. Tuttavia, la firma del metodo impone il combiner per garantire la correttezza formale della **parallelizzazione**:

- In uno stream parallelo (`parallelStream()`), il flusso di elementi di tipo T viene suddiviso tra molteplici thread del `ForkJoinPool`;
- Ciascun worker thread produce un accumulatore parziale indipendente di tipo U invocando l'accumulator;
- Al termine, i risultati parziali di tipo U devono essere aggregati a coppie risalendo l'albero di divisione binaria. Il **combiner** è la funzione puramente binaria $(U, U) \rightarrow U$ che unisce due sotto-risultati parziali.

Affinché la riduzione sia corretta, l'accumulatore e il combiner devono essere **associativi** e l'identità deve essere un autentico elemento neutro: `combiner(identity, u) == u`.

3.8 Raccogliere Risultati: `collect()` e la Classe `Collectors`

L'operazione terminale più versatile e potente del framework è `collect()`, che accetta un'istanza dell'interfaccia `Collector` per trasformare gli elementi dello stream in una struttura dati complessa (liste, insiemi, mappe gerarchiche, stringhe formattate). La classe di utilità `Collectors` fornisce decine di factory statiche pronte all'uso.

3.8.1 Collettori Base e Concatenazione Testuale

- `Collectors.toList()` / `toSet()`: accumula gli elementi in una lista o in un insieme mutabile (oppure `toUnmodifiableList()` per collezioni immutabili).
- `Collectors.toCollection(TreeSet::new)`: accumula gli elementi in una specifica implementazione decisa dal programmatore.
- `Collectors.joining(delimitatore, prefisso, suffisso)`: concatena elementi testuali con separatore in modo ultra-efficiente tramite un `StringBuilder` interno:

Esempio di Joining Testuale

```

1 List<String> linguaggi = Arrays.asList("Java", "C", "Rust", "Python");
2
3 String csv = linguaggi.stream()
4     .map(String::toUpperCase)

```

```

5     .collect(Collectors.joining(", ", "[", "]"));
6
7     System.out.println(csv); // [JAVA, C, RUST, PYTHON]

```

3.8.2 Raggruppamento e Partizionamento Avanzato

Il vero potenziale analitico delle Stream API risiede nelle operazioni di aggregazione relazionale, analoghe a clausole GROUP BY del linguaggio SQL:

Definizione: Raggruppamento con Collectors.groupingBy()

Il metodo `groupingBy(Function<T, K> classifier)` suddivide gli elementi dello stream in partizioni in base al valore calcolato dalla funzione di classificazione, restituendo una `Map<K, List<T>`. Inoltre, combinando un **downstream collector** (`groupingBy(classifier, downstream)`), è possibile applicare un'ulteriore riduzione sugli elementi di ciascun gruppo (conteggio, somma, mappatura, trasformazione in set).

Raggruppamenti Downstream e Partizionamento

```

1 List<String> parole = Arrays.asList("albero", "casa", "arco", "sole", "mela", "mare",
2     "sasso");
3
4 // 1. Raggruppamento per lunghezza della stringa: Map<Integer, List<String>>
5 Map<Integer, List<String>> perLunghezza = parole.stream()
6     .collect(Collectors.groupingBy(String::length));
7 // {4=[casa, sole, mela, mare], 5=[sasso], 6=[albero]}
8
9 // 2. Raggruppamento con downstream counting(): Map<Integer, Long>
10 Map<Integer, Long> conteggioPerLunghezza = parole.stream()
11     .collect(Collectors.groupingBy(String::length, Collectors.counting()));
12 // {4=4, 5=1, 6=2}
13
14 // 3. Raggruppamento con downstream mapping(): Map<Integer, Set<String>>
15 Map<Integer, Set<String>> maiuscolePerLunghezza = parole.stream()
16     .collect(Collectors.groupingBy(
17         String::length,
18         Collectors.mapping(String::toUpperCase, Collectors.toSet())
19     ));
20
21 // 4. Raggruppamento con downstream filtering() (Java 9+):
22 Map<Integer, List<String>> soloConA = parole.stream()
23     .collect(Collectors.groupingBy(
24         String::length,
25         Collectors.filtering(s -> s.contains("a"), Collectors.toList())
26     ));
27
28 // 5. Partizionamento binario con predicato booleano: Map<Boolean, List<String>>
29 Map<Boolean, List<String>> iniziaConS = parole.stream()
30     .collect(Collectors.partitioningBy(s -> s.startsWith("s")));
31 // {false=[albero, casa, arco, mela, mare], true=[sole, sasso]}

```

3.8.3 Lo Studio del Caso d'Aula: Il Censimento del Bimondo in Una Riga

Nel Capitolo 1 abbiamo affrontato l'esercizio del *Bimondo Geometrico*, in cui una lista eterogenea di figure bidimensionali doveva essere censita per contare quante istanze di ciascuna classe concreta fossero presenti nella simulazione.

Nell'approccio procedurale tradizionale, il censimento richiedeva la dichiarazione di una `Map<String, Integer>`, un ciclo di scansione, l'estrazione manuale del nome di classe con `s.getClass().getSimpleName()` e una serie di controlli `if (map.containsKey(...))` con relativi incrementi manuali.

Con le Stream API e i collettori avanzati, l'intera operazione si condensa in **un'unica, fulminea riga di codice dichiarativo**:

Il Censimento del Bimondo Risolto in Una Riga

```

1 List<Shape2d> mondo = Arrays.asList(
2     new Cerchio(5.0),
3     new Rettangolo(4.0, 6.0),
4     new Cerchio(2.5),
5     new Triangolo(3.0, 4.0),
6     new Cerchio(1.0)
7 );
8
9 // Censimento istantaneo per tipo concreto con counting downstream:
10 Map<Class<?>, Long> censimento = mondo.stream()
11     .collect(Collectors.groupingBy(Object::getClass, Collectors.counting()));
12
13 censimento.forEach((classe, quanti) ->
14     System.out.println("Classe " + classe.getSimpleName() + " -> Presenti: " +
15         quanti));
    
```

L'output prodotto a console:

```

1 Classe Cerchio -> Presenti: 3
2 Classe Rettangolo -> Presenti: 1
3 Classe Triangolo -> Presenti: 1
    
```

3.9 Dentro la Libreria: Architettura Interna, Splitterator, Sink e PipelineHelper

Come funziona realmente il motore delle Stream API sotto il cofano della JVM? La Sezione 3.6 della dispensa del corso analizza l'architettura interna della libreria, evidenziando il raffinato pattern a catena di montaggio che governa l'esecuzione.

3.9.1 Lo `Splitterator<T>` come Motore di Scorrimento e Decomposizione

Alla radice di ogni sorgente di stream non risiede un semplice `Iterator`, bensì un `Splitterator<T>` (*Splitable Iterator*):



Figura 3.2: Meccanismo di scomposizione binaria ricorsiva di uno `Splitterator`.

I quattro metodi cardine di `Splitterator<T>` definiscono il protocollo:

1. `boolean tryAdvance((Consumer<? super T> action):` avanza di un singolo elemento consumandolo con l'azione fornita. Restituisce `false` se la sorgente è esaurita (l'equivalente unificato di `hasNext()` e `next()`).
2. `Splitterator<T> trySplit():` la gemma architetturale per la concorrenza. Se la sorgente è partizionabile, estrae una porzione consistente di elementi in un nuovo `Splitterator` da affidare a un altro thread del `ForkJoinPool`, trattenendo per sé la parte restante.
3. `long estimateSize():` stima il numero di elementi rimanenti.
4. `int characteristics():` restituisce una maschera di bit con le caratteristiche intrinseche della sorgente:
 - ORDERED: l'ordine di incontro degli elementi è rigorosamente significativo;
 - DISTINCT: gli elementi della sorgente sono matematicamente privi di duplicati;
 - SORTED: gli elementi rispettano già un ordine naturale o un comparatore;
 - SIZED: la dimensione esatta della sorgente è nota a priori prima dell'elaborazione;
 - NONNULL: la sorgente non contiene elementi `null`.

Principio Architeturale: Ottimizzazioni a Zero-Costo Basate sui Flag di Caratteristica

La JVM sfrutta i flag di caratteristica per eliminare interi stadi della pipeline a tempo di compilazione/esecuzione:

Se invochiamo `treeSet.stream().sorted()`, la pipeline rileva che la sorgente possiede già il flag `SORTED` e **rimuove completamente l'operazione `sorted()`**, risparmiando il buffer e l'ordinamento $O(n \log n)$!

3.9.2 La Catena di Stadi: `PipelineHelper` e `ReferencePipeline`

Quando scriviamo `lista.stream().filter(...).map(...)`, non viene eseguito alcun calcolo. La libreria alloca semplicemente una catena di oggetti interni derivati da `ReferencePipeline`:

- **Head**: incapsula la sorgente e lo `Splitterator` originario;
- **StatelessOp**: stadi intermedi privi di stato (`filter`, `map`, `peek`);
- **StatefulOp**: stadi intermedi con stato e barriera di accumulo (`sorted`, `distinct`).

3.9.3 Il Protocollo di Push: I `Sink<T>`

Quando viene invocata l'operazione terminale (es. `findFirst()` o `collect()`), il motore inverte la direzione: risale la catena dei `ReferencePipeline` e costruisce a ritroso una serie annidata di interfacce `Sink<T>` (*imbuti di ricezione*):

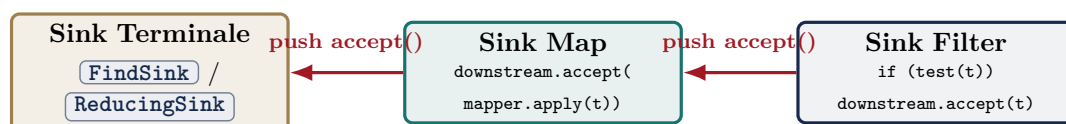


Figura 3.3: Protocollo di propagazione push tra Sink annidati pilotato dallo `Splitterator`.

Il ciclo vitale di un `Sink` espone 4 metodi formali:

- **void `begin`(long size)**: informa lo stadio a valle del numero di elementi in arrivo (o `-1` se sconosciuto);
- **void `accept`(T t)**: consuma il dato e lo trasmette immediatamente al sink successivo (*push model*);
- **boolean `cancellationRequested`()**: interroga il sink per verificare se il calcolo può essere interrotto (*short-circuit*). Appena `findFirst()` riceve il primo dato, restituisce `true` e lo `Splitterator` interrompe l'estrazione;
- **void `end`()**: notifica la conclusione dello streaming, consentendo agli stadi con stato (`sorted`) di emettere gli elementi ordinati.

3.9.4 Prestazioni Parallele e la Regola di Lea ($N \times Q$)

La parallelizzazione con `parallelStream()` non è una scorciatoia a costo zero:

- **Costo di Split e Scheduling**: dividere una struttura ad albero o un array con `trySplit()` e coordinare i task sui thread ha un costo sensibile;
- **Strutture inefficienti**: un `ArrayList` o un range numerico si dividono in $O(1)$ con perfetto bilanciamento. Una `LinkedList` si divide in $O(n)$ con overhead rovinoso;
- **La Regola Euristica di Doug Lea**: la parallelizzazione di uno stream offre benefici concreti rispetto al flusso sequenziale se e solo se:

$$N \times Q > 10.000$$

dove N è il numero di elementi della sorgente e Q è il costo computazionale (tempo di CPU) dell'elaborazione per singolo elemento. Per operazioni banali su piccoli array, lo stream parallelo sarà drasticamente più lento dello stream sequenziale.

Consultazione Esercizi di Laboratorio sugli Stream

Per consolidare l'architettura delle Stream API, verificare empiricamente la lazziness, implementare le tre forme di `reduce()`, progettare collettori avanzati downstream e profilare pipeline parallele, si rimanda agli esercizi dettagliati del **Capitolo A** (dall'Esercizio A.3 all'Esercizio A.3).

Capitolo 4

Interfacce Grafiche Swing, Architettura ad Eventi ed Event Dispatch Thread

In questo capitolo esploriamo la transizione dalla programmazione procedurale a riga di comando (CLI) allo sviluppo di applicazioni grafiche interattive (**GUI**) mediante il framework **Java Swing** e il toolkit fondamentale **Java AWT** (*Abstract Window Toolkit*).

Costruire un'interfaccia grafica non significa soltanto disegnare pulsanti o caselle di testo sullo schermo: significa abbracciare un radicale cambio di paradigma architetturale, noto come **Event-Driven Programming** (programmazione guidata dagli eventi), in cui il flusso di controllo del programma non è più scandito sequenzialmente dal codice dello sviluppatore, ma reagisce in modo asincrono alle azioni imprevedibili dell'utente e del sistema operativo.

4.1 Dal Modello a Riga di Comando alla Programmazione ad Eventi

4.1.1 Paradigmi a Confronto: Esecuzione Lineare vs Reattiva

Nelle applicazioni a riga di comando studiate finora, l'esecuzione del codice segue un flusso sequenziale deterministico governato dal programmatore:

Flusso Sequenziale Sincrono (CLI)

```
1 Scanner scanner = new Scanner(System.in);
2 System.out.print("Inserisci il primo numero: ");
3 int a = scanner.nextInt(); // IL THREAD MAIN SI BLOCCA QUI IN ATTESA
4 System.out.print("Inserisci il secondo numero: ");
5 int b = scanner.nextInt(); // IL THREAD MAIN SI BLOCCA DI NUOVO
6 System.out.println("Somma: " + (a + b));
```

In questo modello:

- Il programma decide *quando* e *cosa* l'utente può inserire;
- L'invocazione di metodi di I/O (come `scanner.nextInt()`) è **bloccante**: il thread della CPU si ferma in attesa che l'utente preme il tasto Invio;
- L'utente non ha alcuna libertà di interagire fuori dall'ordine imposto dallo sviluppatore.

Nelle applicazioni dotate di interfaccia grafica (**GUI**), questo modello è completamente insostenibile:

- Sullo schermo sono presenti contemporaneamente molteplici elementi interattivi (pulsanti, campi di testo, slider, menu a tendina);
- L'utente può compiere un'infinità di azioni imprevedibili in qualsiasi istante: cliccare un pulsante, ridimensionare la finestra, minimizzarla, spostare il cursore del mouse, digitare caratteri o chiudere l'applicazione;
- Il programma non può "bloccarsi in attesa": l'interfaccia deve rimanere fluida, ridisegnarsi costantemente a 60 frame al secondo e reagire istantaneamente alle sollecitazioni.

4.1.2 Il Dialogo Continuo tra Sistema Operativo e Applicazione

A livello architetturale di basso livello, il sistema operativo (Windows, Linux X11/Wayland, macOS) controlla costantemente le periferiche hardware di input (mouse, tastiera, touch screen). Quando l'utente muove il cursore del mouse o preme un tasto:

1. Il driver di periferica solleva un *hardware interrupt*;
2. Il kernel del sistema operativo confeziona un **messaggio di sistema** (ad esempio `WM_LBUTTONDOWN` o `WM_KEYDOWN` nell'architettura Win32) contenente coordinate (x, y) , stato dei tasti e timestamp;
3. Il kernel deposita questo messaggio nella coda degli eventi della finestra su cui si trova il puntatore;

4. La Java Virtual Machine (JVM), attraverso i propri adapter nativi AWT, intercetta il messaggio del sistema operativo e lo trasforma in un'istanza fortemente tipizzata della gerarchia `java.awt.AWTEvent`.

4.1.3 Eventi Hardware a Basso Livello vs Eventi Semantici di Alto Livello

Nel toolkit Java AWT/Swing gli eventi si dividono in due grandi categorie:

Definizione: Tassonomia degli Eventi Java

- **Eventi di Basso Livello (*Low-Level / Component Events*)**: rappresentano le singole interazioni fisiche grezze con l'hardware o con la finestra del sistema operativo. Appartengono a questa categoria: `MouseEvent` (movimento, ingresso, uscita, pressione del tasto del mouse), `KeyEvent` (pressione o rilascio di un singolo tasto fisico), `WindowEvent` (apertura, chiusura, iconificazione del frame) e `FocusEvent`.
- **Eventi Semantici (*Semantic / High-Level Events*)**: incapsulano l'intento logico dell'utente, astruendo completamente dal dispositivo fisico utilizzato per compierlo. Appartengono a questa categoria: `ActionEvent` (attivazione di un comando logico da un pulsante o voce di menu), `ItemEvent` (selezione/deselezione di una checkbox) e `ChangeEvent` (variazione di valore di uno slider).

Principio Architeturale: Astrazione mediante Eventi Semantici

Una buona architettura software per interfacce grafiche si ancora quasi sempre ad **eventi semantici**. Un `JButton` può essere attivato con un click sinistro del mouse, premendo la barra spaziatrice quando ha il focus, premendo il tasto Invio o tramite una combinazione di tasti di scelta rapida (*mnemonic*). Se il programmatore registrasse un ascoltatore grezzo di mouse (`MouseListener`), intercetterebbe solo il click fisico escludendo la tastiera e rompendo l'accessibilità. Registrando invece un `ActionListener` sull'evento semantico `ActionEvent`, l'applicazione risponde all'*intenzione* dell'utente a prescindere dal canale di input fisico.

4.2 Il Modello Architeturale: Observer Pattern e Delegation Event Model

Per gestire gli eventi in modo disaccoppiato, Java adotta il celebre **Delegation Event Model**, derivazione diretta del design pattern GoF **Observer**:



Il modello si fonda su tre attori architeturali distinti:

1. **La Sorgente dell'Evento (*Event Source*)**: è il componente grafico con cui l'utente interagisce (ad esempio un'istanza di `JButton`). La sorgente non sa *cosa* farà il programma quando verrà cliccata; sa soltanto gestire una lista interna di ascoltatori registrati.
2. **L'Objetto Evento (*Event Object*)**: è una classe immutabile (derivata da `java.util.EventObject`) che trasporta tutte le informazioni contestuali dell'accaduto. In particolare, il metodo fondamentale `getSource()` restituisce il riferimento al componente che ha scatenato l'evento, mentre metodi specifici (es. `getActionCommand()`) forniscono dettagli aggiuntivi.
3. **L'Ascoltatore (*Event Listener*)**: è un oggetto che implementa un'interfaccia dedicata derivata da `java.util.EventListener`. Contiene uno o più metodi di callback (come il metodo `actionPerformed()`) che vengono invocati dalla sorgente quando l'evento si verifica.

4.2.1 Multicast Events: Perché add e non set?

Una finezza architeturale evidenziata a lezione riguarda la nomenclatura dei metodi di registrazione dei listener in Java:

La Distinzione tra set e add

Perché in Java scriviamo:

`pulsante.addActionListener(listener);` anziché `pulsante.setActionListener(listener);`?

Il prefisso **set** indicherebbe una relazione 1 : 1: una volta impostato un listener, impostarne un altro sovrascriverebbe e distruggerebbe il precedente.

Il prefisso **add** denota formalmente un'architettura **Multicast Event**: la sorgente mantiene una collezione interna di ascoltatori. Possono registrarsi $0, 1, 2, \dots, N$ listener completamente indipendenti tra loro. Quando l'evento accade, la sorgente itera sulla collezione e notifica *tutti* gli ascoltatori registrati. Allo stesso modo, `removeActionListener()` consente di cancellare la sottoscrizione in qualunque momento.

4.3 Il Thread della Grafica: Event Dispatch Thread (EDT)

La comprensione dell'esecuzione concorrente è il pilastro fondamentale della sicurezza in ambiente Swing:

Principio Architetture: Regola Aurea di Swing: Single-Threaded UI Rule

Il toolkit grafico Swing **NON** è **thread-safe**. I metodi di creazione, configurazione, manipolazione dello stato e visualizzazione dei componenti grafici devono essere invocati **esclusivamente dal thread dedicato alla grafica**, denominato **Event Dispatch Thread (EDT)**.

4.3.1 Perché Swing non è Thread-Safe?

Rendere thread-safe l'intera gerarchia di centinaia di classi di Swing (aggiungendo blocchi **synchronized** su ogni metodo di repaint, calcolo dei font, layout o gestione delle coordinate) avrebbe introdotto un costo computazionale mostruoso e creato un rischio perenne di **deadlock** tra i lock interni del toolkit e i lock applicativi dello sviluppatore.

I progettisti di Java (e di quasi tutti i moderni toolkit grafici, inclusi JavaFX, Android e iOS Cocoa) hanno quindi adottato il modello *Single-Threaded Subsystem*: tutti gli eventi grafici, le richieste di ridisegno (`repaint()`) e le chiamate ai listener vengono elaborate sequenzialmente da un unico thread di sistema, chiamato AWT-EventQueue-0.

4.3.2 La Chiamata Obbligatoria: EventQueue.invokeLater

Quando viene avviata un'applicazione Java, la JVM avvia il thread principale **main**. Se il thread **main** istanzia direttamente la finestra o ne modifica i componenti, si crea una grave **race condition** con l'EDT.

Per rispettare formalmente il contratto architetturale di Swing, l'avvio della GUI deve essere demandato all'EDT tramite il metodo statico `EventQueue.invokeLater(Runnable)`:

Punto di Ingresso Obbligatorio per Applicazioni Swing

```

1  import java.awt.EventQueue;
2  import javax.swing.JFrame;
3
4  public class PrimaFinestraSwing {
5      public static void main(String[] args) {
6          // Demanda l'istanziamento e l'avvio della GUI all'Event Dispatch Thread (EDT)
7          EventQueue.invokeLater(() -> {
8              try {
9                  JFrame finestra = new JFrame("La Mia Prima GUI");
10                 finestra.setSize(400, 300);
11                 finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12                 finestra.setLocationRelativeTo(null); // Centra sullo schermo
13                 finestra.setVisible(true);
14             } catch (Exception e) {
15                 e.printStackTrace();
16             }
17         });
18     }
19 }
```

Il Pericolo del Congelamento della Finestra: Non Bloccare mai l'EDT!

Poiché l'EDT elabora tutti gli eventi sequenzialmente in un unico ciclo (*event loop*), se all'interno del metodo `actionPerformed()` di un listener eseguiamo un'operazione lenta o bloccante (come il download di un file da Internet, una query su database pesante o un ciclo da miliardi di iterazioni):

L'INTERA INTERFACCIA GRAFICA SI CONGELA ISTANTANEAMENTE!

La finestra smette di rispondere ai click, non si ridisegna più, assume un aspetto sbiadito e il sistema operativo mostra il messaggio *"L'applicazione non risponde"*. Le operazioni lunghe devono essere eseguite su un thread worker separato (ad esempio tramite la classe `javax.swing.SwingWorker`) e rientrare sull'EDT solo per aggiornare i componenti visivi.

4.4 Gerarchia dei Componenti Swing e Contenitori

Tutti i componenti visivi di Swing ereditano da una gerarchia di classi con radice in `java.awt.Component`:

- `Component` → classe astratta di AWT che incapsula la geometria bidimensionale (posizione x, y , dimensioni larghezza/altezza, colore di sfondo e font);
- `Container` → sottoclasse di `Component` capace di contenere al proprio interno altri componenti figli e governarne la disposizione tramite un `LayoutManager`;
- `JComponent` → classe base di quasi tutti i componenti Swing (prefisso 'J'). Introduce funzionalità avanzate: rendering a doppio buffer (*double buffering* per eliminare lo sfarfallio), bordi configurabili (`BorderFactory`), tooltip e supporto per il Look and Feel modificabile a runtime.

4.4.1 I Livelli di Contenimento

La progettazione di una finestra si articola su tre livelli gerarchici:

1. **Contenitori Top-Level (Primo Livello):** sono le finestre fornite direttamente dal sistema operativo ospitante. Includono `JFrame` (finestra principale con decorazioni, barra del titolo e pulsanti riduci/chiudi) e `JDialog` (finestre di dialogo modali o non-modali). Una finestra `JFrame` non può essere contenuta in un'altra finestra.
2. **Il ContentPane:** è il pannello invisibile interno al `JFrame` su cui vengono fisicamente ancorati tutti gli elementi visivi. Si recupera tramite `frame.getContentPane()` (oppure chiamando direttamente `frame.add()` che in Java moderno delega in trasparenza al `ContentPane`).
3. **Contenitori Intermedi:** componenti come `JPanel` che non sono finestre autonome, ma fungono da "scatole" per aggregare e raggruppare sotto-componenti, isolandoli visivamente o applicando layout differenti.
4. **Componenti Atomici:** elementi terminali della gerarchia come `JButton`, `JLabel`, `TextField`, `JCheckBox` o `JRadioButton`.

4.5 Gestione dello Spazio: I Layout Manager

4.5.1 Perché il Posizionamento Assoluto (`setLayout(null)`) è Bandito

Nei primi esperimenti con le GUI, molti programmatori sono tentati di posizionare i componenti specificando coordinate fisse in pixel (*Absolute Positioning*):

Anti-Pattern: Posizionamento Assoluto con `setLayout(null)`

```
1 // PRATICA PESSIMA: BANNATA NELLO SVILUPPO PROFESSIONALE!
2 pannello.setLayout(null);
3 JButton btn = new JButton("Premi Qui");
4 btn.setBounds(50, 100, 120, 30); // x=50, y=100, larghezza=120, altezza=30
5 pannello.add(btn);
```

Principio Architetture: Il Fallimento del Posizionamento Assoluto

L'utilizzo di `setLayout(null)` e `setBounds()` produce interfacce **fragili e non portabili**:

- Se l'utente ridimensiona la finestra, i componenti rimangono bloccati nelle loro coordinate fisse in alto a sinistra, lasciando enormi spazi vuoti o finendo tagliati fuori dall'area visibile;

- Se l'applicazione viene eseguita su un display moderno con risoluzione 4K e fattore di scala DPI al 150% o 200%, le etichette con dimensioni fisse in pixel taglieranno il testo interno;
 - Se la lingua del sistema operativo cambia o i font di sistema sono leggermente più larghi, il testo traboccherà dai pulsanti.
- Per questa ragione, Java adotta i **Layout Manager**: oggetti che calcolano dinamicamente la posizione e le dimensioni di ogni componente in base allo spazio disponibile e alle dimensioni preferite (*preferred size*).

4.5.2 Confronto Sinottico tra i Principali Layout Manager

Layout Manager	Logica di Disposizione	Contesto d'Uso Tipico
BorderLayout	Suddivide lo spazio in 5 regioni: NORTH, SOUTH, EAST, WEST, CENTER. Le regioni laterali prendono la loro larghezza/altezza preferita; il CENTER assorbe tutto lo spazio residuo.	Layout predefinito del <code>ContentPane</code> di <code>JFrame</code> . Ideale per la struttura macroscopica della finestra.
FlowLayout	Dispone i componenti in una sequenza orizzontale da sinistra a destra con wrapping a capo automatico, rispettando le dimensioni preferite dei componenti.	Layout predefinito di <code>JPanel</code> . Ottimo per barre di pulsanti o raccolte di checkbox.
GridLayout	Suddivide il contenitore in una griglia regolare di R righe e C colonne di identiche dimensioni, forzando tutti i componenti ad avere la stessa area.	Calcolatrici, tastierini numerici, scacchiere.
BoxLayout	Dispone i componenti lungo un unico asse (orizzontale <code>X_AXIS</code> o verticale <code>Y_AXIS</code>), supportando molle (<i>glue</i>) e distanziatori (<i>strut</i>).	Barre laterali verticali o barre di stato complesse.

Tabella 4.1: Confronto tra i Layout Manager fondamentali in Java Swing.

4.5.3 La Tecnica dei Pannelli Annidati (Nested Panels)

L'approccio professionale per costruire interfacce ricche ed eleganti consiste nell'**annidamento gerarchico di pannelli**: si utilizza un `BorderLayout` per la struttura globale del frame e si inseriscono nei suoi quadranti (NORTH, SOUTH, ecc.) dei `JPanel` secondari dotati di `FlowLayout` o `GridLayout`:

Esempio Architetture di Pannelli Annidati

```
1 JFrame frame = new JFrame("Layout Annidato Professionale");
2 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
3 frame.setSize(500, 350);
4
5 // Il ContentPane di default usa BorderLayout
6 Container cp = frame.getContentPane();
7 cp.setLayout(new BorderLayout(8, 8)); // gap orizzontale e verticale di 8px
8
9 // 1. ZONA CENTRALE: un'area di testo espandibile
10 cp.add(new JScrollPane(new JTextArea("Contenuto principale...")),
11        BorderLayout.CENTER);
12
13 // 2. ZONA SUD: un pannello contenitore per i pulsanti di controllo
14 JPanel pnlSud = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
15 JButton btnAnnulla = new JButton("Annulla");
16 JButton btnConferma = new JButton("Salva Modifiche");
17 pnlSud.add(btnAnnulla);
18 pnlSud.add(btnConferma);
19
20 // Ancoriamo il pannello alla regione SOUTH del frame
```

```
20 cp.add(pnlSud, BorderLayout.SOUTH);
```

4.6 L'Evoluzione dei Listener: Da Classi Esterne a Method Reference

Nel corso della storia di Java, il modo in cui il programmatore collega un'azione a un pulsante ha subito una straordinaria evoluzione sintattica e concettuale, culminata con l'avvento delle lambda expression.

Esaminiamo i 5 approcci storici allo stesso problema: un pulsante "Incrementa" che aumenta un contatore intero visualizzato su una `JLabel`:

4.6.1 1. Classe Esterna Separata

Creazione di un file autonomo `GestoreClick.java`. Richiede di passare al costruttore i riferimenti al modello o ai componenti della GUI, frammentando il codice e aumentando l'accoppiamento.

4.6.2 2. Classe Interna (Inner Class)

La classe che implementa `ActionListener` viene definita come membro interno del frame. Ha accesso diretto ai campi privati della finestra (la label e il contatore), eliminando i costruttori complessi.

4.6.3 3. Classe Anonima Inline (Pre-Java 8)

L'approccio dominante fino a Java 7: la classe viene istanziata al volo nel punto esatto di registrazione del listener:

Ascoltatore con Classe Anonima (Java <= 7)

```
1 btnIncrementa.addActionListener(new ActionListener() {
2     @Override
3     public void actionPerformed(ActionEvent e) {
4         contatore++;
5         lblTesto.setText("Valore: " + contatore);
6     }
7 });
```

4.6.4 4. Lambda Expression (Java 8+)

Poiché `ActionListener` è un'interfaccia funzionale dotata dell'unico metodo astratto `actionPerformed()`, l'intera classe anonima collassa in una concisa espressione lambda:

Ascoltatore con Lambda Expression

```
1 btnIncrementa.addActionListener(e -> {
2     contatore++;
3     lblTesto.setText("Valore: " + contatore);
4 });
```

4.6.5 5. Method Reference su Metodo Dedicato

Se la logica di reazione supera le due o tre righe, l'inserimento di un blocco lambda corposo all'interno dell'albero di costruzione della GUI riduce la leggibilità. La best practice moderna consiste nel fattorizzare la logica in un metodo di istanza privato e associarlo tramite **Method Reference**:

Associazione Tramite Method Reference

```
1 // Nel costruttore del Frame:
2 btnIncrementa.addActionListener(this::onIncrementaClick);
3
4 // Metodo di gestione privato e isolato:
5 private void onIncrementaClick(ActionEvent event) {
6     this.contatore++;
7     this.lblTesto.setText("Valore: " + this.contatore);
8 }
```

4.7 Progettazione di Componenti Custom: Il Caso del Tastierino Numerico (PnlDigits)

Uno dei contributi metodologici più rilevanti della lezione del 30/09 è la progettazione di un **componente grafico composito personalizzato**: il tastierino numerico telefonico o da calcolatrice (PnlDigits).

Componente Incapsulato: PnlDigits



- Incapsulamento totale
- I bottoni interni sono privati
- Espone: `addActionListener()`
- Gestione con `EventListenerList`

4.7.1 Il Principio di Incapsulamento Rigoroso

L'errore architetturale più grave compiuto dai programmatori inesperti è creare un pannello con bottoni e poi esporre i bottoni all'esterno tramite metodi getter pubblici:

```
1 // ERRORE ARCHITETTURALE: espone l'implementazione interna
2 public JButton getBtn1() { return this.btn1; }
```

Se facciamo ciò, distruggiamo l'incapsulamento del componente: la classe cliente (il frame) deve conoscere l'esistenza dei 10 bottoni singoli e deve registrare 10 listener diversi. Se un domani volessimo sostituire i pulsanti con una griglia disegnata su canvas o con un display touch virtuale, il codice cliente collasserebbe.

4.7.2 Il Componente come Proxy di Eventi

La soluzione architetturale corretta consiste nel trattare PnlDigits come una scatola nera:

1. I 10 pulsanti interni (0–9) sono **private**;
2. PnlDigits registra un listener interno su ciascun pulsante;
3. Quando l'utente preme un tasto numerico, PnlDigits intercetta il click e notifica i propri ascoltatori esterni emettendo un nuovo `ActionEvent` in cui la sorgente è PnlDigits stesso e il comando è la cifra digitata;
4. Il componente adotta `EventListenerList` di Swing per garantire sicurezza nei thread e flessibilità multicast.

Ecco l'implementazione completa e robusta del componente:

Implementazione del Componente: PnlDigits.java

```
1 import java.awt.GridLayout;
2 import java.awt.event.ActionEvent;
3 import java.awt.event.ActionListener;
4 import javax.swing.JButton;
5 import javax.swing.JPanel;
6 import javax.swing.event.EventListenerList;
7
8 public class PnlDigits extends JPanel {
9     // Lista sicura e thread-safe di ascoltatori registrati
10    private final EventListenerList listenerList = new EventListenerList();
11
12    public PnlDigits() {
13        // Griglia 4 righe x 3 colonne
14        super(new GridLayout(4, 3, 4, 4));
15
16        // Listener interno comune a tutti i pulsanti della tastiera
17        ActionListener internalListener = e -> {
18            String cifra = e.getActionCommand();
19            fireActionPerformed(new ActionEvent(this, ActionEvent.ACTION_PERFORMED,
20                cifra));
21        };
22    }
```

```

22     // Generazione dei tasti da 1 a 9
23     for (int i = 1; i <= 9; i++) {
24         JButton btn = new JButton(String.valueOf(i));
25         btn.addActionListener(internalListener);
26         this.add(btn);
27     }
28
29     // Ultima riga: spazio vuoto, tasto 0, spazio vuoto
30     this.add(new JPanel()); // segnaposto trasparente
31     JButton btnZero = new JButton("0");
32     btnZero.addActionListener(internalListener);
33     this.add(btnZero);
34     this.add(new JPanel());
35 }
36
37 // Registrazione di un ascoltatore esterno (Multicast Event)
38 public void addActionListener(ActionListener listener) {
39     listenerList.add(ActionListener.class, listener);
40 }
41
42 // Deregistrazione dell'ascoltatore esterno
43 public void removeActionListener(ActionListener listener) {
44     listenerList.remove(ActionListener.class, listener);
45 }
46
47 // Propagazione dell'evento a tutti gli ascoltatori registrati
48 protected void fireActionPerformed(ActionEvent event) {
49     Object[] listeners = listenerList.getListenerList();
50     // L'array di EventListenerList e' strutturato a coppie: [TipoClasse,
51     // IstanzaListener]
52     for (int i = listeners.length - 2; i >= 0; i -= 2) {
53         if (listeners[i] == ActionListener.class) {
54             ((ActionListener) listeners[i + 1]).actionPerformed(event);
55         }
56     }
57 }

```

4.7.3 Integrazione del Componente nel Frame Applicativo

Grazie a questa architettura a scatola nera, la finestra che include il tastierino registra **un unico listener** su tutto il pannello:

Utilizzo Pulito del Componente PnlDigits

```

1  JFrame frame = new JFrame("Display Calcolatrice");
2  JTextField display = new JTextField(12);
3  display.setEditable(false);
4
5  PnlDigits tastierino = new PnlDigits();
6
7  // La finestra si iscrive agli eventi del pannello senza sapere come e' fatto dentro:
8  tastierino.addActionListener(e -> {
9      String cifraPremuta = e.getActionCommand();
10     display.setText(display.getText() + cifraPremuta);
11 });
12
13 frame.getContentPane().add(display, BorderLayout.NORTH);
14 frame.getContentPane().add(tastierino, BorderLayout.CENTER);

```

Avvertenza d'Esame: Limiti degli Editor Visuali (WindowBuilder) e Codifica Manuale

Durante le esercitazioni e lo studio individuale, gli studenti incontrano spesso strumenti di disegno visuale come **Eclipse WindowBuilder** o il GUI Designer di IntelliJ IDEA, che permettono di assemblare finestre tramite drag-and-drop WYSIWYG (*What You See Is What You Get*).

Il docente ha ribadito con estrema fermezza che **nella prova d'esame e nella pratica dell'ingegneria del software seria, il codice delle interfacce grafiche deve essere scritto rigorosamente a mano**:

1. **Proliferazione di Codice Monolitico e Incontrollabile**: gli editor visuali rigenerano continuamente l'albero di sintassi astratta (AST) accumulando centinaia di righe di codice spazzatura all'interno di un unico gigantesco metodo `initialize()`, violando il principio di singola responsabilità e rendendo il refactoring quasi impossibile.
2. **La Trappola del Layout Assoluto (null layout)**: per consentire il trascinamento libero dei componenti con il mouse, i visual builder impostano spesso posizionamenti rigidi a pixel fissi (`setBounds(x, y, w, h)`). Come dimostrato nella Sezione 4.5, questo codice collassa non appena l'applicazione viene eseguita su un display con diversa risoluzione DPI o su un sistema operativo differente.
3. **Incapacità di Progettare Componenti Incapsulati**: un editor visuale è incapace di concepire l'architettura a scatola nera di un componente personalizzato come `PnlDigits`. L'editor si limita ad appiattire tutti i bottoni nel frame principale, costringendo il programmatore a un design procedurale degradato.

Lo studente deve padroneggiare la composizione a strati dei pannelli (`JPanel`), la corretta gerarchia dei Layout Manager e la gestione rigorosa della concorrenza sull'Event Dispatch Thread.

4.8 Concorrenza in Swing: L'Event Dispatch Thread (EDT) e il Ponte verso i Background Thread

Il framework Swing adotta un'architettura fondamentale nota come **Single-Threaded Subsystem**: per massimizzare la reattività e azzerare i costosi lock di sincronizzazione nel rendering dei pixel, tutti i componenti grafici di Swing **non sono thread-safe**.

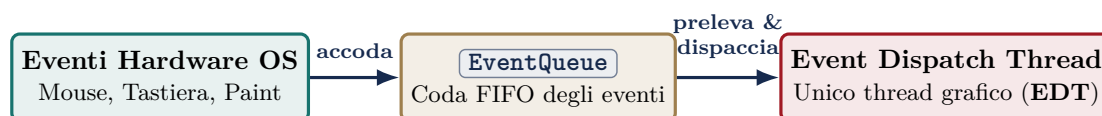


Figura 4.1: Architettura dell'Event Dispatch Thread: dalla coda degli eventi al ridisegno dello schermo.

Principio Architettureale: La Regola Aurea del Thread di Swing

Qualsiasi interazione con un componente grafico visibile (creazione di finestre, modifica di testi con `setText()`, aggiornamento di tabelle o repaint) **deve avvenire tassativamente all'interno dell'Event Dispatch Thread (EDT)**.

Per avviare un'applicazione grafica rispettando questa regola, l'entry point delega l'istanziatura della finestra tramite la coda di sistema con `SwingUtilities.invokeLater()`:

```

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        FinestraCalcolatrice gui = new FinestraCalcolatrice();
        gui.setVisible(true);
    });
}
    
```

Il Pericolo del Congelamento dell'Interfaccia (GUI Freezing)

Se un listener su un pulsante esegue un'operazione pesante (un loop intensivo di calcolo matematico, un trasferimento di rete HTTP o una lettura di un file da 1 GB), tale codice viene eseguito **direttamente sull'EDT**.

Poiché l'EDT è un unico thread, esso rimane intrappolato nell'elaborazione del task: non può prelevare nuovi eventi dalla coda, non può processare i click del mouse né inviare le istruzioni di ridisegno dello schermo al sistema operativo. L'utente percepisce la finestra come completamente bloccata o congelata ("L'applicazione non risponde").

La regola ingegneristica per risolvere questo stallo è immediata:

“Lavora fuori dall’EDT, entra nell’EDT solo per aggiornare i pixel.”

I calcoli intensivi devono essere delegati a thread operai di background. Questa esigenza introduce naturalmente la teoria e la pratica della programmazione concorrente e multi-thread, trattata capillarmente nel capitolo successivo.

Esercitazioni di Laboratorio su Swing ed Eventi

Per implementare finestre complete, analizzare sperimentalmente il comportamento dei diversi Layout Manager, realizzare componenti custom incapsulati e gestire l’interazione tra thread e interfaccia grafica, si rimanda agli esercizi del **Capitolo A** (dall’Esercizio A.4 all’Esercizio A.4).

Capitolo 5

Thread, Concorrenza, Executor e Virtual Threads

Nei sistemi software moderni, l'esecuzione sequenziale di un'unica istruzione alla volta rappresenta un limite insormontabile. I moderni microprocessori integrano molteplici core fisici e logici capaci di eseguire flussi computazionali in autentico parallelismo hardware. Parallelamente, le applicazioni moderne trascorrono gran parte del loro tempo in attesa di operazioni di Input/Output (letture su disco, interrogazioni a database, chiamate di rete tramite socket HTTP o elaborazione di eventi grafici).

Questo capitolo, basato sui moduli teorici *PAJC-26-D04* e *PAJC-26-L04a*, approfondisce in modo rigoroso l'evoluzione del modello di concorrenza in Java: dall'astrazione di base dei thread del sistema operativo e delle primitive di sincronizzazione intrinseca, fino al framework `java.util.concurrent`, all'interazione tra thread e interfaccia grafica Swing, e alla rivoluzione architetturale dei **Virtual Threads** (Project Loom) introdotta in Java 21.

5.1 Processi, Thread e il Modello di Memoria di Java

Per comprendere il funzionamento della concorrenza, è indispensabile distinguere formalmente le due unità fondamentali di esecuzione gestite dal sistema operativo: il **Processo** e il **Thread**.

Definizione: Processo vs Thread

- Un **Processo** è un'istanza di un programma in esecuzione a cui il sistema operativo assegna uno **spazio di indirizzamento di memoria virtuale privato e isolato**, un Process Control Block (PCB), descrittori di file e credenziali di sicurezza. Due processi distinti non possono accedere direttamente alla memoria l'uno dell'altro se non tramite espliciti meccanismi di comunicazione inter-processo (IPC come Shared Memory, Named Pipes o Socket). Il cambio di contesto (*context switch*) tra processi è un'operazione onerosa per la CPU, richiedendo l'invalidazione dei buffer di traslazione della memoria virtuale (TLB) e l'intervento del kernel.
- Un **Thread** (o *flusso di controllo leggero*) è l'unità elementare di schedulazione della CPU all'interno di un processo. Tutti i thread generati all'interno del medesimo processo **condividono lo stesso spazio di memoria (lo Heap)** e i medesimi descrittori di risorse, ma ciascun thread possiede un proprio **Stack privato**, un Program Counter (PC) e un insieme di registri di lavoro.

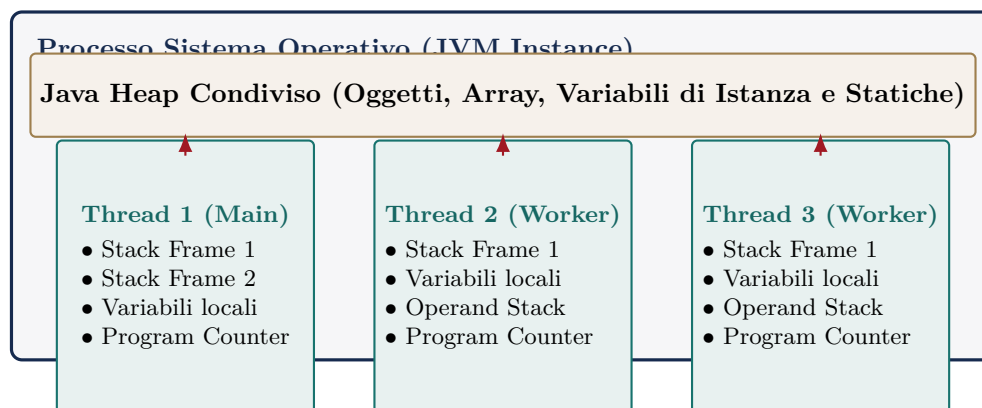


Figura 5.1: Modello di Memoria dei Thread in Java: Stack privati per-thread e Heap comune condiviso.

Principio Architetturale: Il Paradosso della Memoria Condivisa

La condivisione dello Heap tra thread offre una velocità di comunicazione impareggiabile: un thread può scrivere un riferimento nello heap e un altro thread può leggerlo istantaneamente a costo computazionale zero.

Tuttavia, questo vantaggio rappresenta anche la principale fonte di vulnerabilità dei sistemi concorrenti: se più thread tentano di leggere e modificare la medesima struttura dati contemporaneamente senza un'adeguata sincronizzazione, si generano **corruzioni di memoria irreversibili**, inconsistenze di stato e **Race Conditions**.

Al contrario, le variabili allocate all'interno dei metodi risiedono nello ****Stack privato**** del thread che sta eseguendo quel metodo: le variabili locali e i parametri primitivi sono pertanto **intrinsecamente thread-safe**.

5.2 Creazione e Ciclo di Vita dei Thread: `Thread` vs `Runnable`

Nel runtime classico di Java, l'unità di esecuzione concorrente è incapsulata dalla classe `java.lang.Thread`.

5.2.1 I Due Meccanismi di Creazione

Java mette a disposizione due modalità per definire il corpo di un thread:

1. **Ereditarietà (`extends Thread`)**: si crea una sottoclasse di `Thread` e si sovrascrive il metodo `run()`. Questo approccio è sconsigliato nei progetti reali perché Java supporta solo l'ereditarietà singola, impedendo alla classe di estendere altre gerarchie;
2. **Composizione (`implements Runnable`)**: si implementa l'interfaccia funzionale `Runnable` e la si passa come parametro al costruttore di `Thread`. Questo approccio rispetta il principio di separazione tra il compito computazionale (*cosa fare*) e il meccanismo di esecuzione (*chi lo esegue*).

Definizione di un Thread con Runnable e Lambda (HelloThread.java)

```

1 package it.unibs.eps;
2
3 public class HelloThread {
4     public static void main(String[] args) {
5         // 1. Definizione del task tramite interfaccia funzionale Runnable
6         Runnable task = () -> {
7             String nomeThread = Thread.currentThread().getName();
8             System.out.printf("[%s] Esecuzione in background avviata...\n",
9                 nomeThread);
10        };
11
12        // 2. Creazione dell'istanza del thread e assegnazione di un identificativo
13        Thread worker = new Thread(task, "Worker-Alpha");
14
15        // 3. Avvio concorrente
16        worker.start();
17
18        System.out.println("[main] Thread principale continua in parallelo.");
19    }
20 }

```

Principio Architeturale: L'Errore Mortale del Principiante: `start()` vs `run()`

Come evidenziato dal docente durante le lezioni, la differenza tra `start()` e `run()` è categorica:

- **`worker.start()`**: invoca una chiamata nativa della JVM (`private native void start0()`) che richiede al kernel del sistema operativo di allocare un nuovo stack frame e avviare un **nuovo thread parallelo**. Il metodo `start()` ritorna immediatamente e il codice prosegue;
- **`worker.run()`**: **NON crea alcun nuovo thread!** Si limita a eseguire il metodo `run()` come una comunissima funzione sequenziale **sul thread chiamante corrente** (main), bloccandolo ed eliminando qualsiasi parallelismo.

5.2.2 Gli Stati del Ciclo di Vita di un Thread (`Thread.State`)

Un thread Java attraversa un ciclo di vita a 6 stati formali, interrogabili tramite `getState()`:

1. **NEW**: il thread è stato istanziato con `new`, ma non è ancora stato avviato con `start()`;

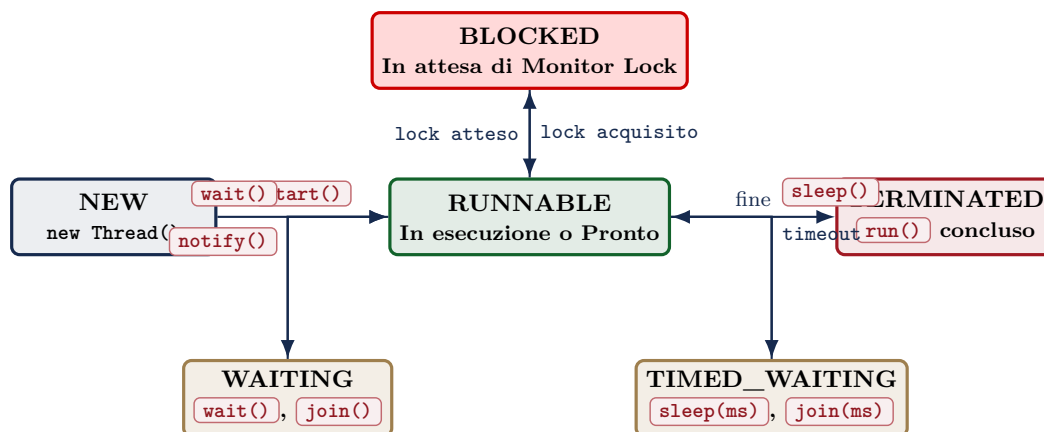


Figura 5.2: Diagramma degli stati del ciclo di vita di un Thread Java (`Thread.State`).

- RUNNABLE**: il thread è in esecuzione sulla CPU oppure è pronto nella coda di scheduling del sistema operativo in attesa di un quanto di tempo (*time slice*);
- BLOCKED**: il thread è sospeso in attesa di acquisire un lock intrinseco (`synchronized`) occupato da un altro thread;
- WAITING**: il thread è in attesa indefinita di un segnale da parte di un altro thread (`Object.wait()`, `Thread.join()`);
- TIMED_WAITING**: il thread è in attesa con timeout prefissato (`Thread.sleep(millis)`, `join(millis)`);
- TERMINATED**: il metodo `run()` ha concluso la propria esecuzione (normalmente o per eccezione non catturata).

5.2.3 Attesa di Terminazione (`join()`) e Sospensione (`sleep()`)

Per coordinare l'esecuzione ed attendere che un thread completi il proprio lavoro prima di leggerne il risultato, si impiega il metodo `join()`:

Sincronizzazione mediante `join()` e `sleep()`

```

1 Thread worker = new Thread(() -> {
2     try {
3         System.out.println("Elaborazione intensiva in corso...");
4         Thread.sleep(2000); // TIMED_WAITING per 2 secondi
5     } catch (InterruptedException e) {
6         System.out.println("Thread interrotto prematuramente!");
7     }
8 });
9
10 worker.start();
11 System.out.println("Attendo la conclusione del worker...");
12 worker.join(); // Il thread chiamante (main) si blocca in WAITING finche' worker non
    termina
13 System.out.println("Worker concluso. Procedo con l'aggregazione.");
    
```

5.2.4 La Politica Cooperativa di Interruzione (`interrupt()`)

Nel linguaggio Java, i metodi violenti per terminare un thread (`stop()`, `destroy()`) sono considerati deprecati e rimossi: essi terminavano il thread all'istante, lasciando i lock acquisiti aperti e corrompendo la memoria della JVM.

Java adotta una ****politica di cancellazione rigorosamente cooperativa****:

- Invocando `t.interrupt()`, non si uccide il thread, ma si imposta a `true` un flag booleano interno (*interruption status*);
- Se il thread si trova in uno stato di attesa bloccante (`sleep()`, `wait()`, `join()`), l'operazione viene interrotta immediatamente e viene scatenata un'`InterruptedException`, resettando il flag a `false`;
- Se il thread si trova in un loop di computazione pura, ha la responsabilità etica di controllare periodicamente `Thread.currentThread().isInterrupted()` e terminare ordinatamente.

Principio Architetture: La Regola d'Oro per la Gestione di InterruptedException

Quando si cattura un'InterruptedException, non bisogna mai sopprimerla silenziosamente:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // REGOLA AUREA: ripristinare il flag per consentire al chiamante di accorgersene!
    Thread.currentThread().interrupt();
    return; // Uscire ordinatamente dal ciclo di esecuzione
}
```

5.3 Il Framework Executor: Disaccoppiamento tra Compito ed Esecuzione

Nei sistemi industriali, la creazione manuale di thread con `new Thread(r).start()` è un grave anti-pattern per tre motivi:

1. **Costo di Allocazione Elevato:** la creazione di un thread di sistema operativo richiede l'allocazione di ≈ 1 MB di stack riservato e una costosa system call al kernel del sistema operativo;
2. **Pericolo di Thread Explosion:** se un server web avvia un nuovo thread nativo per ogni richiesta HTTP in arrivo, con 10.000 richieste concorrenti la memoria fisica collassa per `OutOfMemoryError`;
3. **Mancanza di Controllo:** non esiste una coda centralizzata per limitare il numero di compiti contemporanei.

A partire da Java 5, il package `java.util.concurrent` introduce il **Framework Executor**, che disaccoppia la sottomissione dei compiti dalla gestione fisica del pool di thread.

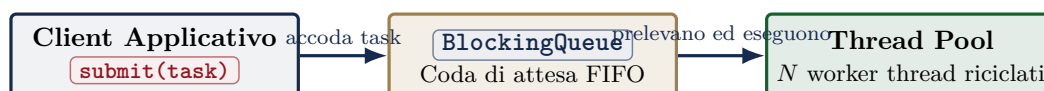


Figura 5.3: Architettura a Thread Pool: disaccoppiamento tra accodamento compiti e riutilizzo dei thread.

5.3.1 Tipi di Thread Pool Standard (`java.util.concurrent.Executors`)

La factory `Executors` offre diverse configurazioni di pool:

- `Executors.newFixedThreadPool(int nThreads)`: istanzia un numero rigorosamente fisso di thread operai. I task in eccesso attendono in una coda illimitata (`LinkedBlockingQueue`). È ideale per compiti a intensità di calcolo prevedibile, dimensionando `nThreads` pari al numero di core fisici disponibili:

$$N_{\text{threads}} = \text{Runtime.getRuntime().availableProcessors}()$$

- `Executors.newCachedThreadPool()`: crea nuovi thread su richiesta se tutti sono occupati e riutilizza quelli disponibili. I thread inattivi per oltre 60 secondi vengono distrutti. Ideale per molti task brevi e leggeri I/O-bound, ma rischioso in caso di carico improvviso;
- `Executors.newSingleThreadExecutor()`: impiega un unico worker thread, garantendo che tutti i task sottomessi vengano eseguiti in modo rigorosamente sequenziale nell'ordine di inserimento;
- `Executors.newScheduledThreadPool(int corePoolSize)`: consente di pianificare l'esecuzione di task a intervalli temporali periodici o con ritardo prefissato (`scheduleAtFixedRate()`).

5.3.2 Ciclo di Vita e Chiusura Ordinata (`shutdown()`)

Un'istanza di `ExecutorService` mantiene i thread operai attivi anche se la coda dei compiti è vuota. Se il programmatore non chiude esplicitamente il pool, la JVM non terminerà mai!

```
1 ExecutorService pool = Executors.newFixedThreadPool(4);
2 // Sottomissione task...
3 pool.shutdown(); // Rifiuta nuovi task, ma completa tutti quelli già accodati
4 try {
5     if (!pool.awaitTermination(60, TimeUnit.SECONDS)) {
6         pool.shutdownNow(); // Forza l'interruzione se supera il timeout
7     }
8 } catch (InterruptedException ex) {
```

```

9     pool.shutdownNow();
10    Thread.currentThread().interrupt();
11 }
    
```

5.4 Calcolo Asincrono con Valore: `Callable<V>` e `Future<V>`

L'interfaccia classica `Runnable` presenta due gravi limitazioni didattiche e architetturali:

- Non può restituire alcun risultato computazionale (`void run()`);
- Non può sollevare alcuna eccezione controllata (*checked exception*).

Per superare queste carenze, Java introduce l'interfaccia funzionale tipizzata `Callable<V>`:

Interfaccia `Callable<V>` (`java.util.concurrent`)

```

1  @FunctionalInterface
2  public interface Callable<V> {
3      V call() throws Exception; // Restituisce un tipo parametrico V e ammette
        eccezioni
4  }
    
```

Quando un task `Callable<V>` viene sottomesso a un `ExecutorService` tramite `submit()`, il metodo restituisce immediatamente un **segnaposto asincrono** che implementa l'interfaccia `Future<V>`:

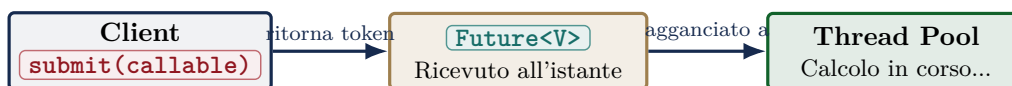


Figura 5.4: Funzionamento del token asincrono `Future<V>`.

I metodi principali di `Future<V>`:

- `V get()`: blocca il thread chiamante fino a quando il calcolo non è completato, restituendo il valore `V` o rilanciando l'eventuale eccezione incapsulata in una `ExecutionException`;
- `V get(long timeout, TimeUnit unit)`: attende con un tempo limite prefissato, sollevando `TimeoutException` se il calcolo impiega troppo tempo;
- `boolean isDone()`: verifica senza bloccare se il task ha terminato l'esecuzione.

5.4.1 Caso di Studio d'Aula: Conteggio Parallelo dei Numeri Primi

Il benchmark canonico introdotto nelle lezioni del corso confronta il conteggio dei numeri primi compresi tra 1 e 10^8 in modalità sequenziale contro la versione concorrente basata su `ExecutorService`:

Conteggio Parallelo con `FixedThreadPool` e `Future` (`PrimeCounter.java`)

```

1  package it.unibs.eps;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.concurrent.*;
6
7  public class PrimeCounter {
8      // Predicato matematico di primalità deterministico
9      public static boolean isPrime(long n) {
10         if (n <= 1) return false;
11         if (n <= 3) return true;
12         if (n % 2 == 0 || n % 3 == 0) return false;
13         for (long i = 5; i * i <= n; i += 6) {
14             if (n % i == 0 || n % (i + 2) == 0) return false;
15         }
16         return true;
17     }
18
19     public static void main(String[] args) throws Exception {
20         long limiteMax = 10_000_000L;
21         int numCore = Runtime.getRuntime().availableProcessors();
22         System.out.printf("Avvio conteggio fino a %d su %d core logici...\n",
            limiteMax, numCore);
    }
    
```

```

23
24     ExecutorService pool = Executors.newFixedThreadPool(numCore);
25     List<Future<Long>> parziali = new ArrayList<>();
26     long chunkSize = limiteMax / numCore;
27
28     long t0 = System.currentTimeMillis();
29
30     // Suddivisione dell'intervallo [1, limiteMax] in chunk omogenei
31     for (int i = 0; i < numCore; i++) {
32         final long start = i * chunkSize + 1;
33         final long end = (i == numCore - 1) ? limiteMax : (start + chunkSize - 1);
34
35         Callable<Long> task = () -> {
36             long count = 0;
37             for (long n = start; n <= end; n++) {
38                 if (isPrime(n)) count++;
39             }
40             return count;
41         };
42         parziali.add(pool.submit(task));
43     }
44
45     // Aggregazione finale bloccante sui risultati
46     long totalePrimi = 0;
47     for (Future<Long> f : parziali) {
48         totalePrimi += f.get(); // Attende il completamento del singolo chunk
49     }
50
51     long t1 = System.currentTimeMillis();
52     pool.shutdown();
53
54     System.out.printf("Trovati %d numeri primi in %d ms.\n", totalePrimi, (t1 -
55     t0));
56 }
    
```

5.5 Problemi di Sincronizzazione: Race Conditions e Visibilità

Quando due o più thread accedono contemporaneamente alla medesima variabile nello heap e almeno uno di essi vi scrive, senza opportuni meccanismi di sincronizzazione si manifesta una **Race Condition** (*Corsa Critica*).

5.5.1 L'Anatomia di una Race Condition a Livello di Bytecode

Consideriamo l'operazione apparentemente banale: `count++`. Un programmatore inesperto potrebbe pensare che l'operatore `++` sia un'istruzione atomica.

Analizzando il codice disassemblato generato da `javap -c`, scopriamo che `count++` si traduce in tre istruzioni distinte del bytecode della JVM:

```

1  aload_0           // 1. Carica il riferimento a 'this' nello stack
2  getfield #count   // 2. Legge il valore attuale di 'count' dallo Heap
3  iconst_1          // 3. Carica la costante 1
4  iadd              // 4. Somma 1 nello stack di operandi
5  putfield #count   // 5. Scrive il nuovo valore nel campo 'count' nello Heap
    
```

L'Interlacciamento Distruttivo delle Istruzioni

Se due thread T_1 e T_2 eseguono contemporaneamente `count++` partendo dal valore iniziale 10:

1. T_1 esegue `getfield` e legge 10;
2. Il sistema operativo sospende T_1 e assegna il quanto di tempo a T_2 ;
3. T_2 esegue `getfield` (legge 10), calcola $10 + 1 = 11$ e scrive 11 con `putfield`;
4. T_1 si risveglia: il suo stack possedeva già il vecchio valore 10, calcola $10 + 1 = 11$ e sovrascrive lo heap con 11!

Due incrementi distinti hanno prodotto un aumento di 1 sola unità: l'incremento di T_2 è andato perso per

sempre. Se due thread eseguono ciascuno 100.000 incrementi, il risultato finale sarà compreso in modo imprevedibile tra 100.000 e 200.000!

5.5.2 Il Problema della Visibilità e la Keyword `volatile`

Oltre all'atomicità, esiste il problema della **Visibilità della Memoria**:

- Per massimizzare le prestazioni, le moderne CPU mantengono copie locali delle variabili nelle velocissime cache hardware (L1, L2, L3) e nei write-buffer locali del core;
- Se il thread T_1 modifica un flag booleano `running = false`, tale modifica potrebbe rimanere confinata nella cache del Core 1 e non essere riversata immediatamente nella memoria principale RAM;
- Se il thread T_2 gira sul Core 2, continuerà a leggere il vecchio valore `running = true` dalla propria cache L1, rimanendo intrappolato in un loop infinito!

Definizione: Parola Chiave

La dichiarazione `volatile boolean running = true;` impone al compilatore e alla JVM l'inserimento di **Memory Barrier** (barriere di memoria hardware):

- Ogni operazione di **scrittura** su una variabile **volatile** viene forzata immediatamente nella memoria RAM principale e le cache degli altri core vengono invalidate;
- Ogni operazione di **lettura** legge direttamente il valore fresco dalla RAM;
- Impedisce il riordinamento (*instruction reordering*) delle istruzioni da parte del compilatore e della CPU.

ATTENZIONE CRITICA: `volatile` garantisce unicamente la **visibilità immediata**, ma **NON garantisce l'atomicità** delle operazioni composte (`count++` rimane corrotto anche se `count` è `volatile`!).

5.6 Sincronizzazione Intrinseca: La Parola Chiave `synchronized`

Per garantire contemporaneamente sia la mutua esclusione (atomicità) sia la visibilità della memoria, Java integra nativamente il concetto di **Monitor Lock** (*Intrinsic Lock*).

Ogni singolo oggetto residente nello heap della JVM possiede associato un lock intrinseco. Quando un thread esegue un blocco **synchronized**:

1. Acquisisce il lock dell'oggetto: nessun altro thread può entrare in blocchi protetti dallo stesso lock;
2. Gli altri thread concorrenti vengono sospesi nello stato `BLOCKED`;
3. Al termine del blocco (anche in caso di eccezione imprevista), il lock viene rilasciato automaticamente e le modifiche vengono riversate nella RAM.

5.6.1 Le Tre Modalità di Sincronizzazione

Modalità di Sincronizzazione in Java (ContatoreSincronizzato.java)

```

1 package it.unibs.eps;
2
3 public class ContatoreSincronizzato {
4     private int count = 0;
5     private final Object lockAusiliario = new Object(); // Lock dedicato
6
7     // 1. Metodo di istanza sincronizzato: lock acquisito su 'this'
8     public synchronized void incrementa() {
9         this.count++;
10    }
11
12    // 2. Blocco sincronizzato fine-grained: lock su oggetto dedicato
13    public void incrementaFineGrained() {
14        // Operazioni preliminari non sincronizzate...
15        synchronized (this.lockAusiliario) {
16            this.count++;
17        }
18        // Operazioni successive non sincronizzate...
19    }

```

```

20
21 // 3. Metodo statico sincronizzato: lock acquisito sull'oggetto Class
    (ContatoreSincronizzato.class)
22 public static synchronized void logGlobale(String msg) {
23     System.out.println("[AUDIT LOG] " + msg);
24 }
25 }
    
```

5.6.2 Coordinamento Avanzato con `wait()`, `notify()` e `notifyAll()`

Quando un thread ha acquisito un lock ma non può proseguire finché non si verifica una determinata condizione logica (ad esempio, una coda vuota da cui non si può prelevare), può cedere volontariamente il passo:

Principio Architetture: Il Contratto di `wait()` e `notify()`

I metodi `wait()`, `notify()` e `notifyAll()` appartengono alla classe radice `Object` e possono essere invocati **esclusivamente all'interno di un blocco sincronizzato sul medesimo oggetto**:

- `lock.wait()`: rilascia atomica-mente il lock intrinseco e pone il thread corrente nello stato WAITING;
- `lock.notifyAll()`: risveglia tutti i thread in attesa sul lock. Ciascun thread risvegliato compete per riacquisire il lock e riprendere l'esecuzione.

La Regola del While Loop contro i Risvegli Spuri (*Spurious Wakeups*)

Per motivi di architettura hardware delle CPU e dell'OS, un thread in attesa su `wait()` può occasionalmente svegliarsi anche in assenza di una notifica (*spurious wakeup*). Inoltre, tra il momento del risveglio e la riacquisizione del lock, un altro thread concorrente potrebbe aver consumato la risorsa.

Di conseguenza, è una ****regola tassativa di programmazione**** verificare la condizione di attesa all'interno di un ciclo **while**, MAI con un semplice **if**:

```

1 synchronized (lock) {
2     // CORRETTO: riesamina SEMPRE la condizione dopo il risveglio!
3     while (!condizioneValida) {
4         lock.wait();
5     }
6     // Procedi con l'azione protetta
7 }
    
```

5.7 Deadlock e le Quattro Condizioni di Coffman

L'utilizzo indiscriminato di lock annidati espone l'architettura al rischio mortale del ****Deadlock**** (*Stallo Cieco*): una situazione in cui due o più thread rimangono bloccati a tempo indefinito, ciascuno in attesa di un lock detenuto dall'altro.

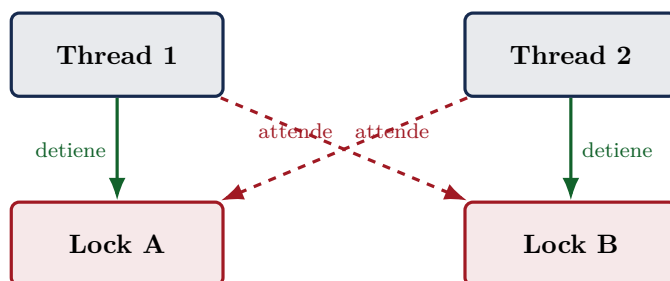


Figura 5.5: Lo stallo circolare del Deadlock: dipendenza ciclica tra lock e thread.

Definizione: Le Quattro Condizioni di Coffman (1971)

Affinché si verifichi un Deadlock, devono coesistere contemporaneamente quattro condizioni necessarie e sufficienti:

1. **Mutua Esclusione**: almeno una risorsa deve essere detenuta in modo non condiviso (un solo thread alla volta);

2. **Possesso e Attesa (*Hold and Wait*)**: un thread detiene almeno una risorsa e ne richiede contestualmente un'altra attualmente occupata da un altro thread;
3. **Assenza di Prelazione (*No Preemption*)**: le risorse non possono essere sottratte con la forza a un thread; possono essere rilasciate unicamente dal detentore;
4. **Attesa Circolare (*Circular Wait*)**: esiste una catena chiusa di thread $\{T_0, T_1, \dots, T_n\}$ tale che T_0 attende una risorsa detenuta da T_1 , T_1 attende T_2 , $\dots$, e T_n attende T_0 .

Principio Architeturale: Tecnica Architeturale di Prevenzione: Ordinamento Globale dei Lock

Per distruggere a priori la quarta condizione di Coffman (l'attesa circolare), l'ingegnere del software adotta la strategia del ****Global Lock Ordering****:

Tutti i thread dell'applicazione devono acquisire i lock sempre e solo nel medesimo ordine canonico predefinito.

Se due thread devono acquisire sia il conto A che il conto B per effettuare un bonifico, si ordina l'acquisizione in base all'ID univoco o a `System.identityHashCode()`: il lock dell'account con ID minore viene acquisito sempre per primo, eliminando alla radice ogni possibilità di ciclo.

5.8 Utilità Concorrenti Avanzate del Package `java.util.concurrent`

Per superare la rigidità dei blocchi `synchronized`, Java 5 mette a disposizione strutture dati concorrenti altamente ottimizzate:

5.8.1 Primitive Atomiche Lock-Free (`java.util.concurrent.atomic`)

Le classi `AtomicInteger`, `AtomicLong` e `AtomicBoolean` permettono di eseguire operazioni aritmetiche composte (`incrementAndGet()`, `compareAndSet()`) **senza usare lock bloccanti**. Sotto il cofano, sfruttano l'istruzione hardware ****CAS**** (*Compare-And-Swap*) implementata nativamente dalle CPU moderne:

CAS(indirizzo, valoreAtteso, nuovoValore)

La CPU confronta il valore in memoria e lo aggiorna in un unico ciclo di clock hardware solo se corrisponde a quello atteso, ritentando automaticamente in un loop ottimistico qualora un altro thread abbia modificato il valore nel frattempo.

5.8.2 Lock Espliciti `ReentrantLock` e Interfacce `Condition`

La classe `ReentrantLock` sostituisce `synchronized` offrendo controlli avanzati:

- `tryLock(long timeout, TimeUnit unit)`: tenta di acquisire il lock senza rimanere bloccato all'infinito, prevenendo i deadlock;
- Politica di equità (*Fairness*): permette di configurare una politica FIFO per l'acquisizione del lock;
- Interfacce `Condition`: consentono di creare molteplici code di attesa separate per lo stesso lock (es. `nonPieno` e `nonVuoto`), evitando il risveglio indiscriminato di tutti i thread.

5.8.3 Il Pattern Produttore-Consumatore con `BlockingQueue`

L'implementazione del pattern Produttore-Consumatore con `ArrayBlockingQueue` o `LinkedBlockingQueue` condensa decine di righe di `wait()` e `notifyAll()` in un'unica, elegantissima struttura thread-safe con metodi bloccanti `put()` e `take()`:

Produttore-Consumatore Pulito con `BlockingQueue`

```

1  BlockingQueue<String> coda = new ArrayBlockingQueue<>(10);
2
3  // Thread Produttore
4  new Thread(() -> {
5      try {
6          while (true) {
7              String item = generaMessaggio();
8              coda.put(item); // Blocca automaticamente se la coda e' piena (capienza
9              }
10         }

```

```

0    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
1  }).start();
2
3  // Thread Consumatore
4  new Thread(() -> {
5      try {
6          while (true) {
7              String item = coda.take(); // Blocca automaticamente se la coda e' vuota
8              elabora(item);
9          }
10     } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
11 }).start();

```

5.9 Concorrenza in Ambienti Grafici: Swing, EDT e `SwingWorker`

Ricollegandoci all'analisi dell'Event Dispatch Thread (EDT) affrontata nel Capitolo 4, quando un'applicazione desktop Swing deve eseguire un'elaborazione prolungata (es. una scansione della rete o il calcolo dei numeri primi), è vietato bloccare l'EDT.

Java risolve questa esigenza tramite la classe astratta tipizzata `javax.swing.SwingWorker<T, V>`:

- `T doInBackground()`: eseguito su un thread di background del pool interno di Swing. Non deve toccare alcun componente grafico visibile;
- `publish(V... chunks)`: invocato dall'interno di `doInBackground()` per trasmettere risultati intermedi;
- `process(List<V> chunks)`: eseguito ****automaticamente sull'EDT**** per aggiornare barre di avanzamento (`JProgressBar`) o etichette grafiche;
- `done()`: eseguito ****sull'EDT**** al termine del calcolo. Permette di recuperare il risultato finale tramite `get()` e riabilitare i pulsanti dell'interfaccia.

Esempio Canonico di `SwingWorker` per GUI Reattiva

```

1  public class CalcoloPrimiWorker extends SwingWorker<Long, Integer> {
2      private final long max;
3      private final JProgressBar barra;
4
5      public CalcoloPrimiWorker(long max, JProgressBar barra) {
6          this.max = max;
7          this.barra = barra;
8      }
9
10     @Override
11     protected Long doInBackground() throws Exception {
12         long contatore = 0;
13         for (long i = 1; i <= max; i++) {
14             if (PrimeCounter.isPrime(i)) contatore++;
15             if (i % 100_000 == 0) {
16                 int percentuale = (int) ((i * 100) / max);
17                 publish(percentuale); // Invia dato parziale
18             }
19         }
20         return contatore;
21     }
22
23     @Override
24     protected void process(List<Integer> chunks) {
25         // ESEGUITO SULL'EDT: sicuro per aggiornare Swing!
26         int ultimoValore = chunks.get(chunks.size() - 1);
27         barra.setValue(ultimoValore);
28     }
29
30     @Override
31     protected void done() {
32         // ESEGUITO SULL'EDT: notifica finale

```

```

33     try {
34         long risultato = get();
35         JOptionPane.showMessageDialog(null, "Calcolo completato: " + risultato +
36         " primi.");
37     } catch (Exception ex) {
38         ex.printStackTrace();
39     }
40 }

```

5.10 La Nuova Era di Java: Virtual Threads (Java 21, Project Loom)

Con il rilascio di **Java 21** a supporto esteso (LTS), la piattaforma Java ha subito la mutazione architetturale più radicale della sua storia con l'introduzione dei **Virtual Threads** (formalizzati da *JEP 444* all'interno del progetto *Loom*).

5.10.1 Il Limite Strutturale dei Platform Threads (1:1 con l'OS)

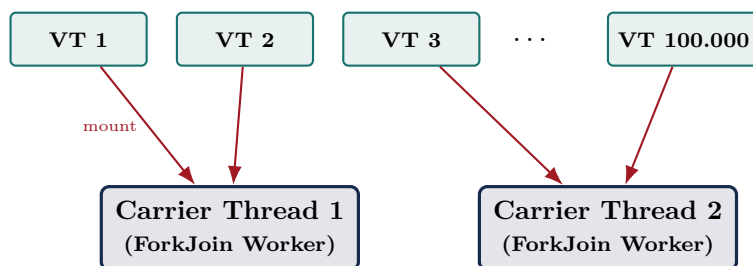
Fin dalle origini, i thread Java standard (*Platform Threads*) operavano secondo una mappatura 1:1 con i thread del kernel del sistema operativo (pthreads su Linux, thread Win32 su Windows). Questo approccio comportava colli di bottiglia insormontabili:

- Ogni platform thread riserva ≈ 1 MB di memoria stack fisica;
- Una macchina hardware con 16 GB di RAM non può fisicamente reggere più di 3.000-5.000 thread contemporanei senza collassare;
- Quando un server adotta il modello classico *“un thread per connessione”* (Thread-Per-Request), la scalabilità si arresta non per mancanza di CPU, ma per saturazione della memoria e overhead dei context-switch kernel-mode.

5.10.2 L'Architettura dei Virtual Threads (M:N User-Mode)

I **Virtual Threads** sono flussi di esecuzione ultra-leggeri gestiti interamente a livello utente dal runtime della JVM, slegati dalla mappatura 1:1 del sistema operativo:

Milioni di Virtual Threads nello Heap ($\approx$ poche centinaia di byte)



Pochi Platform Threads del Sistema Operativo (uguali al numero di Core CPU)

Figura 5.6: Architettura M:N dei Virtual Threads: milioni di task virtuali montati dinamicamente su pochi Carrier Threads fisici.

Il meccanismo di funzionamento è straordinario:

1. Un Virtual Thread occupa inizialmente solo **poche centinaia di byte** nello heap della JVM (non 1 MB!);
2. La JVM esegue i Virtual Thread montandoli (*mount*) su un piccolo pool di platform thread del sistema operativo detti **Carrier Threads** (pari al numero di core CPU);
3. Quando il Virtual Thread esegue un'operazione bloccante (lettura da socket di rete, accesso a file, interrogazione database JDBC o `Thread.sleep()`):

La JVM effettua l'*unmount* istantaneo del Virtual Thread dal Carrier Thread!

Lo stato di esecuzione del virtual thread viene congelato nello heap, e il Carrier Thread fisico del sistema operativo viene immediatamente riutilizzato per eseguire un altro Virtual Thread!

4. Quando l'operazione di I/O si sblocca, la JVM *rimonta* il Virtual Thread su un Carrier Thread disponibile e il codice riprende esattamente da dove si era interrotto.

Creazione e Scalabilità Massiva con Virtual Threads (Java 21)

```

1 package it.unibs.eps;
2
3 import java.time.Duration;
4 import java.util.concurrent.Executors;
5
6 public class VirtualThreadDemo {
7     public static void main(String[] args) throws Exception {
8         // Creazione di un Executor basato su Virtual Threads
9         // NON si usano piu' pool fissi: per ogni task si genera un nuovo Virtual
10        Thread!
11        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
12
13            // Possiamo lanciare tranquillamente 100.000 thread senza esaurire la
14            memoria!
15            for (int i = 0; i < 100_000; i++) {
16                final int id = i;
17                executor.submit(() -> {
18                    // Simulazione di una chiamata di rete o I/O bloccante
19                    Thread.sleep(Duration.ofSeconds(1));
20                    if (id % 20_000 == 0) {
21                        System.out.printf("Virtual Thread #%d completato con
22                        successo.\n", id);
23                    }
24                    return id;
25                });
26            }
27            // Il try-with-resources attende automaticamente la conclusione di tutti i
28            100.000 thread!
29
30            System.out.println("Tutti i 100.000 task asincroni sono terminati con
31            successo.");
32        }
33    }
34 }

```

Principio Architetture: Impatto Ingegneristico dei Virtual Threads

I Virtual Threads mandano in pensione la complessità del Reactive Programming (*Reactive Streams*, *WebFlux*, *CompletableFuture* con callback hell):

- Gli sviluppatori possono tornare a scrivere codice ****semplice, lineare, imperativo e sincrono**** (`String res = client.get();`);
- Le chiamate bloccanti non costano nulla perché non bloccano più i thread del sistema operativo;
- Gli stack trace delle eccezioni tornano a essere chiari, leggibili e sequenziali, semplificando drasticamente il debugging e il monitoraggio forense.

Consultazione Esercizi di Laboratorio su Thread e Concorrenza

Per approfondire l'implementazione pratica della concorrenza, il calcolo numerico parallelo, il benchmarking empirico delle race conditions, il pattern Produttore-Consumatore e l'impiego dei Virtual Threads di Java 21, si consultino gli esercizi dettagliati nell'**Appendice A**.

Appendice A

Eserciziario e Tracce di Laboratorio

Il presente capitolo raccoglie organicamente tutte le tracce, le specifiche di collaudo e le consegne degli esercizi del corso, integrando le attività svolte durante le lezioni frontali, i compiti assegnati per casa (*homework*) e le simulazioni di progetti complessi.

Conformemente all'impostazione metodologica e al rigore architetturale del corso, ciascun esercizio è catalogato formalmente secondo **tre livelli progressivi di difficoltà**:

- **Livello 1 (Base – Sintassi, API e Strutture Dati)**: focalizzato sulla padronanza delle firme dei metodi, dei tipi primitivi e wrapper, e delle classi core del runtime Java.
- **Livello 2 (Intermedio – Algoritmico e Problem Solving)**: richiede la combinazione di più principi (incapsulamento, polimorfismo, programmazione funzionale, gestione dell'I/O ed eccezioni).
- **Livello 3 (Avanzato – Architettura ad Oggetti Multi-file)**: progetti d'ingegneria del software su larga scala articolati su 4-8+ classi e interfacce interconnesse, con contratti formali, thread-safety, asserzioni TDD e assenza totale di codice simulato.

A.1 Capitolo 1: Warm-up e Fondamenti della Programmazione a Oggetti

Fascia 1: Base (Sintassi, API e Strutture Dati)

Esercizio 1.1: StringMagic – Manipolazione Testuale, Incapsulamento e TDD

Livello: *Livello 1 – Base.*

Argomento del Task: Creazione di classi con stato immutabile/incapsulato, costruttori, scansione algoritmica di sequenze di caratteri e collaudo guidato dai test (*Test-Driven Development*).

Riferimento Didattico: Slide 41 (*PAJC_L01_Warm-up.pdf* – Sezione “*HOMEWORK*”).

Consegna Operativa: Si richiede di realizzare una classe denominata `StringMagic` per la rappresentazione incapsulata di un testo, esponendo:

1. Costruttore esplicito che riceve la stringa base;
2. Metodo `reverse()` che restituisce una nuova stringa con i caratteri invertiti;
3. Metodo `maskVowel()` che sostituisce tutte le vocali (minuscole e maiuscole) con il carattere asterisco `'*'`;
4. Metodo `freq(char c)` che restituisce il conteggio esatto delle occorrenze del carattere `c`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: `StringMagicTest.java`

```
1 StringMagic s = new StringMagic("un seme magico");
2 assert "ocigam emes nu".equals(s.reverse()) : "Errore reverse!";
3 assert "*n s*m* m*g*c*".equals(s.maskVowel()) : "Errore maskVowel!";
4 assert s.freq('m') == 2 : "Errore frequenza di 'm'!";
5 assert s.freq('z') == 0 : "Errore frequenza carattere assente!";
```

Esercizio 1.2: FileUtil – Ispezione del Filesystem, Visualizzazione Numerata e Hex Dump

Livello: *Livello 1 – Base.*

Argomento del Task: Gestione degli stream di I/O a caratteri e a byte (`BufferedReader`, `FileInputStream`), gestione deterministica delle risorse (`try-with-resources`) ed esame forense esadecimale.

Riferimento Didattico: Slide 56 (*PAJC_L01_Warm-up.pdf* – Sezione “*COMPITO*”).

Consegna Operativa: Progettare una classe di utilità `FileUtil` dotata dei seguenti metodi statici:

1. `print(String filename)`: legge un file di testo e ne visualizza le righe precedute dal numero progressivo allineato a destra ("%3d | %s\n");
2. `dump(String filename)`: apre un flusso a byte e visualizza l'esadecimale a blocchi di 16 byte per riga, stampando offset a 8 cifre (00000000:), byte esadecimali separati da spazi e caratteri ASCII stampabili (sostituendo con '.' i caratteri di controllo non stampabili).

Esercizio 1.3: CollectionStats – Statistiche descrittive su Liste e Insiemi

Livello: *Livello 1 – Base.*

Argomento del Task: Utilizzo di collezioni generiche (`List`, `Set`, `Map`), calcolo di frequenze, moda statistica e rilevazione duplicati.

Riferimento Didattico: Esercitazione Warm-up sui generics e collections.

Consegna Operativa: Realizzare una classe generica `CollectionStats<T>` che incapsula una collezione di elementi omogenei e fornisce i seguenti metodi analitici:

1. `distinctCount()`: restituisce il numero di elementi distinti (sfruttando un `HashSet`);
2. `frequencyMap()`: genera una `Map<T, Integer>` contenente il conteggio delle occorrenze di ciascun elemento;
3. `mode()`: restituisce l'elemento con la frequenza più elevata (in caso di parità, il primo incontrato).

Fascia 2: Intermedio (Algoritmico e Problem Solving)

Esercizio 1.4: Gerarchia Geometrica vs Protocollo Visivo – Disaccoppiamento Ortogonale

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Polimorfismo, principio di singola responsabilità, separazione tra tassonomia (`extends`) e contratti comportamentali (`implements`).

Riferimento Didattico: Slide 14–39 (*PAJC_L01_Warm-up.pdf*).

Consegna Operativa:

1. Dichiarare l'interfaccia `Shape2d` con i metodi `double perimeter()` e `double area()`;
2. Implementare le classi concrete `Rectangle`, `Square`, `Circle` ed `Ellipse`;
3. Introdurre l'interfaccia ortogonale `Drawable` con il metodo `void draw()`;
4. Creare una classe `Spaceship` che implementa `Drawable` pur non essendo una forma geometrica, evidenziando il polimorfismo orizzontale;
5. Popolare una lista polimorfa `List<Drawable>` e iterare invocando `draw()`.

Esercizio 1.5: Macchina a Stati Finiti (FSM) tramite Enum con Metodi Polimorfi

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Pattern State implementato mediante enumerativi Java avanzati, metodi astratti in `enum` e transizioni di stato deterministiche.

Riferimento Didattico: Slide 60–61 (*PAJC_L01_Warm-up.pdf*).

Consegna Operativa: Modellare una FSM per un distributore automatico o processo di validazione con 4 stati: `IDLE` → `AUTHENTICATING` → `PROCESSING` → `COMPLETED`. Ciascun valore dell'`enum StatoWorkflow` deve:

1. Possedere un campo `String descrizione` e il rispettivo getter;
2. Implementare il metodo astratto `public abstract StatoWorkflow nextStep(boolean esito)`;
3. Gestire l'eventuale transizione verso lo stato di errore `FAILED` in caso di `esito == false`.

Esercizio 1.6: Polinomio Incapsulato con Algebra Simbolica e Derivata**Livello:** *Livello 2 – Intermedio.***Argomento del Task:** Incapsulamento difensivo, aritmetica su mappe sparse (`Map<Integer, Double>` grado-coefficiente), uguaglianza formale (`equals` e `hashCode`).**Consegna Operativa:** Progettare la classe `Polinomio` che rappresenta un polinomio a coefficienti reali a una variabile:

1. Metodo `aggiungiTermine(int grado, double coefficiente)`;
2. Metodo `valuta(double x)`: calcola il valore del polinomio in x con il metodo di Horner o esponenziazione diretta;
3. Metodo `derivata()`: restituisce un nuovo `Polinomio` rappresentante la derivata prima;
4. Metodo `somma(Polinomio altro)`: restituisce un nuovo polinomio somma.

Fascia 3: Avanzato (Architettura ad Oggetti Multi-file)**Esercizio 1.7: MiniFileSystemSimulator – Architettura Multi-file Composita****Livello:** *Livello 3 – Avanzato (Architettura a 5+ Classi).***Argomento del Task:** Design Pattern Composite, gestione della gerarchia ad albero, percorsi assoluti e relativi, permessi Unix-like e navigazione ricorsiva.**Consegna Operativa:** Progettare una suite di classi interconnesse per la simulazione di un filesystem in memoria:

1. **Classe astratta** `FSNode`: radice comune con nome, nodo genitore (`DirectoryNode`) e timestamp di creazione;
2. **Classe** `FileNode`: estende `FSNode`, contiene un payload di byte e il calcolo della dimensione fisica;
3. **Classe** `DirectoryNode`: estende `FSNode`, contiene una collezione di figli (`Map<String, FSNode>`), metodi per aggiungere, rimuovere, cercare per nome e calcolare ricorsivamente la dimensione totale;
4. **Classe** `FileSystemEngine`: gestisce il nodo root (`"/`), il nodo corrente (*working directory*) e simula i comandi tipici: `cd(String path)`, `ls()`, `mkdir(String name)`, `touch(String name, byte[] data)` e `pwd()`.

Esercizio 1.8: VehicleFleetManager – Telemetria, Contratti di Manutenzione ed Emissioni**Livello:** *Livello 3 – Avanzato (Architettura a 6+ Classi).***Argomento del Task:** Gerarchia polimorfa profonda, interfacce di certificazione, storicizzazione cronologica degli eventi e calcolo energetico.**Consegna Operativa:** Realizzare un sistema per la gestione di una flotta aziendale di veicoli:

1. Interfaccia `Veicolo`: metodi `targa()`, `chilometraggio()`, `consumoMedioPerKm()`;
2. Sottoclassi concrete: `AutoElettrica` (capacità batteria, degrado celle), `CamionDiesel` (portata merci, classe Euro), `FurgoneIbrido`;
3. Interfaccia `SottoponibileARRevisione` con contratto formale per il controllo periodico;
4. Classe `RegistroManutenzioni` che registra interventi, costi e notifiche di scadenza chilometrica;
5. Classe `FlottaAziendale` per il calcolo dell'impronta carbonica totale e ordinamento per efficienza energetica.

Progetti Evolutivi Modulari Interconnessi

Esercizio 1.9: [Evolutivo Filone A – Parte 1] InventoryStore: Modello Prodotti, Scorte e Persistenza su File

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 1).*

Argomento del Task: Incapsulamento OOP, gerarchia di prodotti, gestione eccezioni checked/unchecked, I/O binario/testuale e validazione integrità con checksum.

Riferimento Didattico: Slide 44–58 (*PAJC_L01_Warm-up.pdf*) e §1.9.

Consegna Operativa: Si richiede di progettare il nucleo di un sistema di gestione magazzino ed e-commerce che verrà progressivamente esteso nei capitoli successivi:

1. Progettare la classe astratta `Articolo` (codice SKU alfanumerico univoco, nome, prezzo base `double`, quantità a magazzino `int`) e le sottoclassi concrete `ProdottoFisico` (con peso in kg e costo di spedizione) e `ProdottoDigitale` (con URL di download e dimensione in MB);
2. Applicare l'incapsulamento difensivo: l'inventario non può accettare prezzi negativi o scorte inferiori a zero (sollevare `IllegalArgumentOutOfRangeException`);
3. Realizzare la classe `Magazzino` che mantiene una collezione indicizzata `Map<String, Articolo>` ed espone metodi per carico merce, scarico merce (sollevando una checked exception custom `ScorteInsufficientiException`) e ricerca per codice;
4. Implementare la persistenza dello stato su file mediante due metodi:
 - `salvaSuFile(String filename)`: serializza l'inventario in un file di testo formattato CSV, calcolando e accodando all'ultima riga un checksum (es. somma hash o SHA256 dei record);
 - `caricaDaFile(String filename)`: legge e ricostruisce il catalogo verificando tassativamente il checksum (sollevare `DatiCorrottiException` se il file è manomesso).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: InventoryStoreTest.java

```
1  Magazzino m = new Magazzino();
2  m.aggiungi(new ProdottoFisico("SKU-01", "Tastiera Meccanica", 89.90, 15, 1.2));
3  m.aggiungi(new ProdottoDigitale("SKU-02", "E-Book Java Pro", 29.99, 100,
4      "https://store/ebook.pdf", 14.5));
5
6  assert m.cerca("SKU-01").getPrezzoBase() == 89.90 : "Errore ricerca!";
7  m.preleva("SKU-01", 5);
8  assert m.cerca("SKU-01").getQuantita() == 10 : "Errore scarico scorte!";
9
10 m.salvaSuFile("inventario.csv");
11 Magazzino ripristinato = Magazzino.caricaDaFile("inventario.csv");
12 assert ripristinato.cerca("SKU-02").getNome().equals("E-Book Java Pro") : "Errore
    ripristino!";
```

Esercizio 1.10: [Evolutivo Filone B – Parte 1] UrbanRoadNetwork: Grafo Stradale, Veicoli e Topologia da File

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 1).*

Argomento del Task: Strutture dati a grafo (liste di adiacenza), manipolazione stringhe con tokenizzatore, lettura file di configurazione e modellazione OOP di entità semantiche.

Riferimento Didattico: §1.4 e §1.9.

Consegna Operativa: Costruire il modello geometrico e topologico di una rete stradale cittadina:

1. Definire la classe `Incrocio` (id univoco, coordinate x, y , lista di strade incidenti) e la classe `TrattaStradale` (nodo sorgente, nodo destinazione, lunghezza in metri, limite di velocità e capienza massima veicoli);
2. Definire la gerarchia di veicoli con classe astratta `Veicolo` (targa, lunghezza fisica in metri, velocità

- attuale) e sottoclassi concrete `Automobile`, `Autobus` e `Ambulanza` (con priorità di emergenza);
- Progettare la classe `ReteStradale` che consente di caricare la topologia della città da un file di configurazione strutturato a sezioni (`[INCROCI]` e `[STRADE]`);
 - Implementare il metodo `inserisciVeicolo(Veicolo v, String idStrada)` che verifica la capienza fisica residua della tratta, sollevando un'eccezione checked `StradaSaturaException` qualora lo spazio occupato ecceda la lunghezza totale della carreggiata.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: UrbanRoadNetworkTest.java

```

1 ReteStradale rete = new ReteStradale();
2 rete.caricaConfigurazione("mappa_citta.txt");
3
4 assert rete.getIncroci().size() >= 2 : "Incroci non caricati!";
5 TrattaStradale viaRoma = rete.getTratta("TR-01");
6 rete.inserisciVeicolo(new Automobile("AB123CD", 4.2, 50.0), "TR-01");
7 assert viaRoma.getVeicoliPresenti().size() == 1 : "Veicolo non inserito!";
    
```

Sfida Iper-Difficile (High-Performance Systems & Zero-GC HFT Engine)

Esercizio 1.11: BinaryTimeseriesWAL & Zero-GC RingBuffer: Motore di Storage Binario ad Alto Throughput

Livello: *Livello 3 – Iper-Difficile (Systems Programming & Zero-GC Architecture).*

Argomento del Task: Accesso I/O a byte grezzi a bassissimo livello (`RandomAccessFile`, `ByteBuffer`), layout di memoria contiguo, bitwise CRC32, buffer circolare fisso senza allocazione dinamica di oggetti nel ciclo critico (*Zero-GC*).

Riferimento Didattico: §1.9 (Architettura degli stream a basso livello).

Consegna Operativa: Nei motori di contrattazione finanziaria ad alta frequenza (HFT) e nei database timeseries, la creazione continua di oggetti Java nello heap distrugge le prestazioni scatenando pause imprevedibili del Garbage Collector (Stop-The-World). Si richiede di ingegnerizzare un motore di persistenza su file Write-Ahead Log (WAL) conforme ai seguenti vincoli tassativi:

- Record Binario Fisso (32 Byte Contigui):** Ciascun tick di mercato deve occupare esattamente 32 byte:
 - `long` timestamp epoch in nanosecondi (8 byte);
 - `int` ID numerico dello strumento finanziario (4 byte);
 - `double` prezzo di scambio (8 byte);
 - `int` volume di titoli scambiati (4 byte);
 - `int` flag binari di mercato compressi a livello di bit (4 byte);
 - `int` checksum di controllo CRC32 calcolato manualmente bit-a-bit (4 byte).
- Struttura Dati Zero-GC:** Progettare la classe `ZeroGcRingBuffer` basata su un array di byte preallocato contiguo (`byte[]`) di dimensione 2^N (con maschera bitwise per l'indice circolare: `index & (capacity - 1)`).
- Divieto Assoluto di Allocazione:** Nel ciclo critico di inserimento e scarico è vietata qualsiasi invocazione a `new`. Il passaggio dei dati avviene tramite un'unica istanza riutilizzabile di una struttura mutable `MarketTickRecord` passata come cursore (*Flyweight Pattern*).
- Flushing Sequenziale ad Alto Throughput:** Implementare il metodo `flushToDisk(RandomAccessFile raf)` che trasferisce blocchi contigui da 64 KB direttamente su disco, garantendo oltre 2.000.000 di inserimenti al secondo su benchmark locale.
- Verifica Forense:** Implementare la procedura di ripristino `recover(RandomAccessFile raf)` che scandisce l'intero file, ricalcola il checksum CRC32 per ogni blocco da 32 byte e isola i record corrotti senza interrompere il flusso.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: BinaryWalBenchmarkTest.java

```

1 ZeroGcRingBuffer buffer = new ZeroGcRingBuffer(65536); // 64K slot preallocati
2 MarketTickRecord cursore = new MarketTickRecord();
3
4 long t0 = System.nanoTime();
5 for (int i = 0; i < 1_000_000; i++) {
6     cursore.set(System.nanoTime(), 101, 150.25 + (i % 10), 100, 0x01);
7     buffer.append(cursore); // Nessuna allocazione heap nell'hot loop!
8 }
9 long t1 = System.nanoTime();
10 System.out.printf("Throughput inserimento: %.2f M op/sec\n", 1000.0 / ((t1 - t0)
    / 1e6));
11
12 RandomAccessFile raf = new RandomAccessFile("ticks.wal", "rw");
13 buffer.flushToDisk(raf);
14 assert raf.length() == 1_000_000L * 32 : "Dimensione file WAL non
    corrispondente!";
15 raf.close();

```

A.2 Capitolo 2: Classi Anonime, Lambda, Interfacce Moderne e Sealed

Fascia 1: Base (Sintassi e Refactoring)

Esercizio 2.1: Refactoring Progressivo: Da Classe Anonima a Lambda e Method Reference

Livello: *Livello 1 – Base.*

Argomento del Task: Contrazione sintattica, eliminazione del boilerplate, target typing e mapping delle 4 categorie di method reference.

Riferimento Didattico: Dispensa D02 (*Esercizio 2.1*) e Slide 10–13 (*PAJC-26-L02.pdf*).

Consegna Operativa: Riscrivere tre classi anonime tradizionali (`Runnable`, `Comparator<Integer>`, `Function<String, String>`) prima in forma di lambda expression concisa e successivamente (ove lecito) in forma di method reference, giustificando la non-equivalenza semantica di `System.out::println` per `Runnable`.

Esercizio 2.2: Audit di Conformità e Certificazione SAM (@FunctionalInterface)

Livello: *Livello 1 – Base.*

Argomento del Task: Definizione formale di interfaccia funzionale, regole di conteggio dei metodi astratti, esclusione di default/statici e metodi di `Object`.

Riferimento Didattico: Dispensa D02 (*Esercizio 2.2*) e Slide 18–19 (*PAJC-26-L02.pdf*).

Consegna Operativa: Analizzare cinque interfacce con diverse combinazioni di metodi astratti, metodi default e firme di `Object` (`equals`), determinando quali sono conformi a `@FunctionalInterface` e dimostrando il rigetto delle non conformi da parte del compilatore.

Esercizio 2.3: Analisi Forense della Cattura di Stato: Il Contatore non Effectively Final

Livello: *Livello 1 – Base.*

Argomento del Task: Modello di memoria Stack vs Heap, ciclo di vita dei record di attivazione e vincolo di immutabilità **effectively final**.

Riferimento Didattico: Dispensa D02 (*Esercizio 2.3*) e Slide 8 (*PAJC-26-L02.pdf*).

Consegna Operativa: Dato un ciclo che tenta di incrementare una variabile locale all'interno di una lambda, spiegare l'errore del compilatore e implementare le due soluzioni: (A) contenitore Heap invariante (`int[]` o `AtomicInteger`) e (B) redesign funzionale puro senza effetti collaterali.

Fascia 2: Intermedio (Contratti e Funzionale)

Esercizio 2.4: Architettura a Pipeline con Interfaccia Funzionale Personalizzata

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Progettazione di interfacce SAM custom, metodi di ordine superiore e composizione di trasformazioni in cascata.

Riferimento Didattico: Dispensa D02 (*Esercizio 2.4*).

Consegna Operativa: Dichiarare `@FunctionalInterface` `interface Trasformazione { String applica(String s); }` e realizzare il metodo statico `applicaTutte(String, List<Trasformazione>)` collaudandolo con `trim`, `toUpperCase` e aggiunta di punteggiatura.

Esercizio 2.5: Estensione del Motore della Calcolatrice ed Esportazione Immutabile

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Operatori matematici standard (`Math`), method references statici e principio di incapsulamento difensivo delle collezioni (*Defensive Copying*).

Riferimento Didattico: Dispensa D02 (*Esercizio 2.5*) e Slide 29–30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Estendere `Motore` registrando operatori `"%"`, `"max"` e `"min"`, ed esporre `operatoriDisponibili()` proteggendo la mappa interna da modifiche client tramite `Set.copyOf()`.

Esercizio 2.6: FileUtil con Iniezione di Comportamento e Callback di Colorazione

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Inversione del controllo su I/O, pattern callback mediante `BiConsumer<Integer, String>` e formattazione dinamica delle righe.

Riferimento Didattico: Slide 30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Aggiungere a `FileUtil` il metodo `print(filename, lineDecorator)` con parametro `BiConsumer<Integer, String>` che delega la formattazione di ogni riga a una funzione esterna (es. righe alternate colorate o filtraggio righe vuote).

Esercizio 2.7: Matrix<T> Generica e MatrixOperator Funzionale ad Alto Ordine

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Generics, interfacce funzionali ad alto ordine e trasformazione di matrici bidimensionali cella per cella.

Riferimento Didattico: Slide 30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Realizzare `Matrix<T>` con il metodo `map(MatrixOperator<T> op)` che restituisce una nuova matrice applicando la trasformazione ad ogni cella (r, c) .

Fascia 3: Avanzato (Sealed Hierarchies, Modern Interfaces e Pattern Matching)

Esercizio 2.8: Catalogo Prodotti con Comparatori Combinabili Multipli e Null-Safe

Livello: *Livello 3 – Avanzato.*

Argomento del Task: Integrazione tra `Comparable` e factory moderne di `Comparator` (`comparing`, `thenComparing`, `reversed`, `nullsLast`), coerenza con `equals`.

Riferimento Didattico: Dispensa D02b (*PAJC-26-D02b.pdf*).

Consegna Operativa: Progettare una classe immutabile `Prodotto` (con codice SKU, categoria, prezzo `double`, recensione media `Double` opzionale):

1. Implementare l'ordinamento naturale con `Comparable<Prodotto>` basato unicamente su SKU, garantendo perfetta coerenza con `equals()` e `hashCode()`;
2. Progettare comparator statici combinabili complessi:

- Ordinamento per categoria (alfabetico), poi per prezzo decrescente, e infine per recensione con valori nulli posizionati in fondo (`Comparator.nullsLast`);
3. Dimostrare in un test suite che l'ordinamento rispetta le proprietà matematiche di antisimmetria e transitività.

Esercizio 2.9: Albero delle Espressioni Matematiche con Gerarchia Sealed e Pattern Matching

Livello: *Livello 3 – Avanzato (Java 17–21 Standard).*

Argomento del Task: Gerarchie chiuse con `sealed` e `permits`, pattern matching per `switch` sui tipi, valutazione ricorsiva e derivazione simbolica.

Riferimento Didattico: Dispensa D02b (*PAJC-26-D02b.pdf* – Sezione Sealed Hierarchies).

Consegna Operativa: Costruire un motore di calcolo simbolico per espressioni algebriche:

1. Dichiarare l'interfaccia sigillata:


```
public sealed interface Expr permits Costante, Variabile, Somma, Prodotto {}
```
2. Definire i quattro tipi concreti come `record` immutabili (`final`);
3. Implementare il metodo puro di valutazione:


```
public static double eval(Expr e, Map<String, Double> ambiente)
```

 utilizzando il pattern matching per lo `switch` sui tipi con verifica di esaustività del compilatore (senza clausola `default`);
4. Implementare il metodo di derivazione analitica `public static Expr derivata(Expr e, String var)` applicando simbolicamente la regola della somma e del prodotto di Leibniz ($D(f \cdot g) = f' \cdot g + f \cdot g'$).

Progetti Evolutivi Modulari Interconnessi

Esercizio 2.10: [Evolutivo Filone A – Parte 2] FunctionalDiscountEngine & ChainedComparatorCart

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 2).*

Argomento del Task: Espansione del modulo `Magazzino` (Es. A.1) mediante programmazione funzionale: comportamento iniettabile con `Function`, regole di sconto con `Predicate`, composizione di comparatori (`Comparator.comparing`) e copie difensive immutabili.

Riferimento Didattico: §2.8 e §2.13.2.

Consegna Operativa: Riprendere ed estendere l'architettura dell'esercizio A.1:

1. Progettare la classe `Carrello` contenente una collezione ordinata di articoli selezionati dall'utente e la rispettiva quantità;
2. Dichiarare l'interfaccia funzionale `RegolaSconto`:


```
@FunctionalInterface public interface RegolaSconto { double calcolaSconto(Carrello c); }
```
3. Implementare factory statiche di sconti mediante lambda expression:
 - Sconto percentuale sul totale se l'importo supera una determinata soglia;
 - Coupon a codice promozionale fisso (sottrazione diretta);
 - Sconto categoria (es. 20% sui soli `ProdottoDigitale`);
4. Progettare la classe `MotorePromozioni` che riceve una lista di `RegolaSconto` e le applica in cascata combinandole con composizione funzionale, assicurando che lo sconto totale non superi il valore del carrello;
5. Implementare nel `Carrello` metodi di ordinamento che accettano comparatori combinati: ordinare gli articoli per prezzo decrescente, e a parità di prezzo per nome alfabetico (`Comparator.comparing(Articolo::getPrezzo).reversed().thenComparing(Articolo::getNome)`).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: FunctionalDiscountEngineTest.java

```

1 Carrello cart = new Carrello();
2 cart.aggiungi(new ProdottoFisico("SKU-01", "Monitor 4K", 300.0, 5, 4.0), 1);
3 cart.aggiungi(new ProdottoDigitale("SKU-02", "Corso Java", 100.0, 50, "url",
    1.0), 2); // 2 x 100 = 200
4
5 MotorePromozioni promo = new MotorePromozioni();
6 // Regola: 10% se totale > 400
7 promo.registraRegola(c -> c.totaleLordo() > 400 ? c.totaleLordo() * 0.10 : 0.0);
8
9 assert cart.totaleLordo() == 500.0 : "Totale errato!";
10 assert promo.calcolaScontoTotale(cart) == 50.0 : "Sconto errato!";
11 assert promo.totaleNetto(cart) == 450.0 : "Netto errato!";

```

Esercizio 2.11: [Evolutivo Filone B – Parte 2] TrafficLightFSM & FunctionalDriverBehavior

Livello: Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 2).

Argomento del Task: Espansione del simulatore di traffico (Es. A.1) con macchine a stati a semaforo (**enum**), strategie di guida funzionali iniettate tramite `Predicate<Veicolo>` e `Function<Veicolo, Velocita>`.

Riferimento Didattico: §1.10 e §2.9.

Consegna Operativa: Evolvere il modello della rete stradale dell'esercizio A.1 trasformando il sistema da statico a dinamico:

- Modellare il semaforo cittadino mediante un **enum** evoluto `StatoSemaforo`:
 - Valori: ROSSO, GIALLO, VERDE;
 - Ciascun valore definisce un metodo astratto `public abstract StatoSemaforo transizione();`
 - Ciascun valore possiede un predicato associato `public abstract boolean consenteTransito(Veicolo v);` l'ambulanza in emergenza deve poter transitare anche con il rosso!
- Associare a ciascun incrocio una collezione di semafori regolati temporalmente;
- Definire la politica di accelerazione/decelerazione del veicolo tramite un'interfaccia funzionale `ComportamentoGuida`:


```
@FunctionalInterface public interface ComportamentoGuida { double
        calcolaVelocita(Veicolo v, double distanza0stacolo); }
```
- Creare tre stili di guida predefiniti con lambda: *Prudente* (decelera gradualmente a partire da 50 metri), *Aggressivo* (mantiene la velocità massima finché non è a 15 metri dall'ostacolo) ed *Eco-Friendly*;
- Implementare il metodo `simulaPassoTemporale(double deltaSecondi)` che aggiorna la posizione di tutti i veicoli in base allo stato dei semafori e alle rispettive funzioni di comportamento.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficSimulationStepTest.java

```

1 Automobile auto = new Automobile("MI999ZZ", 4.0, 50.0);
2 auto.setComportamento((v, dist) -> dist < 20.0 ? 0.0 : v.getVelocitaMassima());
3
4 Ambulanza ambulanza = new Ambulanza("EMERG-1", 5.5, 90.0, true);
5
6 StatoSemaforo rosso = StatoSemaforo.ROSSO;
7 assert !rosso.consentireTransito(auto) : "L'auto non puo' passare con il rosso!";
8 assert rosso.consentireTransito(ambulanza) : "L'ambulanza deve poter transitare con
    il rosso!";

```

Sfida Iper-Difficile (High-Performance Systems & Zero-GC HFT Engine)

Esercizio 2.12: LockFreeDisruptorPipeline & SealedASTMatcher: Event Dispatching Sequenziale Senza Mutex

Livello: *Livello 3 – Iper-Difficile (Lock-Free Concurrency & Functional Type Matching).*

Argomento del Task: Primitive atomiche (`AtomicLong`), barriere di memoria (`volatile`), code SPSC ad alte prestazioni, interfacce funzionali primitive specializzate e compilazione di query con gerarchia `sealed` e `switch` pattern matching.

Riferimento Didattico: §2.14 e §2.9.

Consegna Operativa: Nelle architetture di trading e brokeraggio, l'utilizzo di `synchronized` o `ReentrantLock` nel percorso critico introduce latenze inaccettabili a causa del context switch del sistema operativo. Si richiede di progettare un bus di messaggi lock-free basato sul pattern **LMAX Disruptor**:

1. **Coda Circolare SPSC (Single Producer Single Consumer):** Implementare una coda lock-free su array fisso di dimensione 2^K . I cursori di scrittura (`writeSequence`) e di lettura (`readSequence`) devono essere gestiti con variabili `volatile` o registri atomici (`AtomicLong`) con padding esplicito per prevenire il *false sharing* tra le linee di cache della CPU L1/L2 (64 byte cache-line);
2. **Interfaccia di Elaborazione a Zero-Boxing:** Definire l'interfaccia funzionale `LongEventConsumer` con metodo `void onEvent(long sequence, long payloadData)`;
3. **Albero di Predicati Sealed (Query AST):** Modellare i filtri di matching degli eventi attraverso una gerarchia sigillata:


```
public sealed interface EventFilter permits ValueEquals, RangeBetween,
      BitmaskMatches, LogicalAnd, LogicalOr {}
```
4. **Valutazione JIT Esaustiva:** Implementare il matcher degli eventi con pattern matching per lo `switch` sui tipi (senza clausola `default`), verificando che il compilatore garantisca l'eshaustività a compile-time;
5. **Benchmark di Concorrenza:** Dimostrare in un test con due thread indipendenti (Producer ed Event Processor) una capacità di dispatching superiore a 10.000.000 di eventi al secondo con latenza sub-microsecondo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: DisruptorPipelineBenchmarkTest.java

```
1 LockFreeDisruptor disruptor = new LockFreeDisruptor(1024 * 64);
2 EventFilter filter = new RangeBetween(1000, 2000);
3
4 AtomicLong eventiElaborati = new AtomicLong(0);
5 disruptor.subscribe((seq, data) -> {
6     if (filter.matches(data)) {
7         eventiElaborati.incrementAndGet();
8     }
9 });
10
11 Thread producer = new Thread(() -> {
12     for (long i = 0; i < 5_000_000; i++) {
13         disruptor.publish(i % 10_000);
14     }
15 });
16 producer.start();
17 producer.join();
18
19 assert eventiElaborati.get() > 0 : "Nessun evento processato dal matcher!";
```

A.3 Capitolo 3: Stream Computazionali, Pipeline e Collettori

Fascia 1: Base (Pipeline e Metodi Core)

Esercizio 3.1: Filtro, Mappatura e Statistiche su Flussi di Stringhe

Livello: *Livello 1 – Base.*

Argomento del Task: Pipeline lineari con `filter`, `map`, `distinct`, `sorted` e raccolta con `toList()`.

Riferimento Didattico: Dispensa D03 (*PAJC-26-D03.pdf* – §1–2) e Slide L03.

Consegna Operativa: Data una lista di stringhe rappresentanti titoli di libri (contenenti spazi e lettere miste):

1. Rimuovere gli spazi superflui (`trim()`) e scartare i titoli vuoti o più corti di 3 caratteri;
2. Convertire tutti i titoli in formato maiuscolo;
3. Eliminare i duplicati (`distinct()`);
4. Ordinare alfabeticamente e raccogliere i primi 5 risultati in una lista immutabile.

Esercizio 3.2: IntStream e Analisi di Primalità Funzionale

Livello: *Livello 1 – Base.*

Argomento del Task: Flussi primitivi specializzati, azzeramento del boxing, `IntStream.rangeClosed()` e `summaryStatistics()`.

Riferimento Didattico: Dispensa D03 (*PAJC-26-D03.pdf* – §4).

Consegna Operativa:

1. Realizzare il predicato `public static boolean isPrimo(int n)` usando un'unica pipeline funzionale: `n > 1 && IntStream.rangeClosed(2, (int)Math.sqrt(n)).noneMatch(d -> n % d == 0)`;
2. Generare tutti i numeri primi compresi tra 1 e 10.000;
3. Calcolarne in un unico passaggio la somma totale, il valore massimo e la media sfruttando `IntSummaryStatistics`.

Esercizio 3.3: Generatore di Fibonacci e Sequenze Geometriche con Stream.iterate

Livello: *Livello 1 – Base.*

Argomento del Task: Flussi infiniti, iterazione algoritmica mediante funzioni unarie e troncamento con `limit()`.

Riferimento Didattico: Slide L03 e Appunti del 30/09.

Consegna Operativa:

1. Generare i primi N termini della sequenza di Fibonacci operando su array di coppie `long[] {a, b}`;
2. Generare la sequenza infinita delle potenze di due $(1, 2, 4, 8, \dots)$, scartare le prime 5 potenze con `skip(5)` ed estrarre le successive 10 convertendole in stringa formattata.

Fascia 2: Intermedio (Laziness, Factory e Collettori)

Esercizio 3.4: Rilevazione Sperimentale della Laziness e dell'Ordine di Esecuzione

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Studio forense dell'elaborazione verticale vs orizzontale, intercettazione con `peek()` e short-circuit.

Riferimento Didattico: Lezione 30/09 (trascrizione e note d'aula).

Consegna Operativa: Scrivere un programma di benchmarking didattico che dimostra:

1. Che l'assenza di un'operazione terminale produce zero computazione;
2. Che scambiando l'ordine di `filter()` e `map()` su un insieme di 100.000 stringhe, il numero totale di invocazioni della funzione di mappatura crolla proporzionalmente alla selettività del filtro;

- Misurare e stampare il numero esatto di passaggi eseguiti nei due casi mediante un contatore atomico `AtomicInteger`.

Esercizio 3.5: Risoluzione del Single-Use Stream tramite Supplier Factory

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Gestione dell'`IllegalStateException`, pattern Factory per flussi ripetibili e concorrenza.

Riferimento Didattico: Dispensa D03 (§3).

Consegna Operativa: Progettare una classe `StreamQuery<T>` che incapsula una sorgente dati e una catena di filtri racchiusa in un `Supplier<Stream<T>`:

- Esporre metodi riutilizzabili: `count()`, `first()`, `toList()` e `anyMatch()`;
- Dimostrare che il client può invocare ripetutamente questi metodi in qualsiasi ordine senza mai incorrere in `IllegalStateException`;
- Validare il comportamento in un contesto multi-thread con 4 worker concorrenti.

Esercizio 3.6: Censimento e Partizionamento del Bimondo Geometrico

Livello: *Livello 2 – Intermedio.*

Argomento del Task: `Collectors.groupingBy`, aggregazioni downstream con `counting()` e `partitioningBy`.

Riferimento Didattico: Lezione 30/09 (Esercizio Bimondo in una riga).

Consegna Operativa: A partire da una lista eterogenea di figure `Shape2d` del Capitolo 1:

- Raggruppare le forme per classe concreta calcolando il conteggio istanze in un'unica espressione (`groupingBy(Object::getClass, counting())`);
- Partizionare le forme in base all'area (superiore o inferiore a 50.0);
- Calcolare per ciascun gruppo la somma totale dei perimetri tramite downstream collector (`Collectors.summingDouble(Shape2d::perimeter)`).

Esercizio 3.7: LogAnalyzer – Parsing e Aggregazione di Log Server con Files.lines

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Stream I/O ad alte prestazioni con `Files.lines()`, espressioni regolari e raggruppamenti statistici.

Consegna Operativa: Dato un file di log web server nel formato standard "DATA_ORA [LIVELLO] IP CODICE_HTTP MESSAGGIO":

- Utilizzare `Files.lines(Path)` in un blocco `try-with-resources` per garantire il rilascio immediato del file descriptor del sistema operativo;
- Estrarre la lista degli indirizzi IP unici che hanno generato errori HTTP ≥ 400 ;
- Generare una mappa `Map<Integer, Long>` con il conteggio degli accessi per ciascun codice di stato HTTP;
- Identificare i primi 3 IP con il maggior volume di richieste complessive.

Fascia 3: Avanzato (Architettura Multi-file Funzionale)

Esercizio 3.8: FinancialPortfolioAnalyzer – Analisi Portafoglio con FlatMap e Custom Collectors

Livello: *Livello 3 – Avanzato (Architettura a 5 Classi).*

Argomento del Task: Trasformazioni complesse 1-a-molti con `flatMap()`, collettori personalizzati, raggruppamenti multidimensionali e calcolo del rischio.

Consegna Operativa: Realizzare un motore finanziario articolato sulle seguenti classi:

1. `TitoloAzionario`: ticker, settore industriale, prezzo corrente;
2. `Transazione`: data, tipo (ACQUISTO/VENDITA), quantità, prezzo unitario;
3. `PosizionePortafoglio`: titolo, lista cronologica di transazioni;
4. `Portafoglio`: intestatario, lista di posizioni;
5. Progettare una suite di metodi analitici basati esclusivamente su Stream:
 - Calcolo del valore di mercato totale del portafoglio;
 - Ripartizione percentuale del capitale investito per settore industriale (`Map<String, Double>`);
 - Estrazione delle transazioni storiche con volume superiore a soglia mediante `flatMap()`.

Esercizio 3.9: FullTextSearchEngine – Motore di Ricerca ad Indice Inverso Funzionale

Livello: *Livello 3 – Avanzato (Architettura a 6 Classi).*

Argomento del Task: Elaborazione del linguaggio naturale (NLP) funzionale, rimozione stop-words, indicizzazione inversa con Stream paralleli e ranking TF-IDF.

Consegna Operativa: Costruire un motore di ricerca per documenti testuali:

1. Classe immutabile `Documento` (id, titolo, testo integrale);
2. Pipeline di tokenizzazione: divide il testo in token alfanumerici, converte in minuscolo, elimina caratteri non ASCII e filtra un set predefinito di stop-words;
3. Costruzione dell'**Indice Inverso** (`Map<String, Set<Integer>`) che mappa ciascun termine all'insieme di ID dei documenti in cui compare, generata con un singolo `collect(groupingBy(...))`;
4. Algoritmo di interrogazione booleana (AND, OR) su query multi-parola con calcolo di rilevanza TF-IDF e ordinamento dei documenti più pertinenti.

Progetti Evolutivi Modulari Interconnessi

Esercizio 3.10: [Evolutivo Filone A – Parte 3] SalesAnalyticsEngine: Reportistica Vendite e Collettori Downstream

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 3).*

Argomento del Task: Espansione del modulo vendite (Es. A.2) con pipeline Stream avanzate: trasformazione 1-a-molti con `flatMap`, aggregazioni complesse con `Collectors.groupingBy`, downstream collectors multipli e partizionamento.

Riferimento Didattico: §3.8 e §B.2.

Consegna Operativa: Integrare nel sistema e-commerce un motore analitico basato interamente su Stream per supportare il Business Intelligence aziendale:

1. Progettare la classe `Ordine` (id, data, cliente, lista di articoli acquistati con quantità, stato: IN_ATTESA, EVASO, ANNULLATO);
2. Implementare la classe `RegistroVendite` contenente lo storico di tutti gli ordini;
3. Realizzare una suite di metodi analitici puri tramite un'unica pipeline di Stream per ciascun requisito:
 - `fatturatoPerCategoria()`: calcola il ricavo totale generato raggruppato per categoria merceologica (`Map<String, Double>`) combinando `flatMap`, `groupingBy` e `summingDouble()`;
 - `clientiTopSpender(int limit)`: estrae i primi *N* clienti con il maggior volume di spesa complessivo negli ordini evasi;
 - `partizionaPerFasciaDiSpesa(double soglia)`: partiziona gli ordini in due liste (`Map<Boolean, List<Ordine>`) con valore totale superiore o inferiore alla soglia specificata;
 - `articoloPiuVenduto()`: individua l'articolo con la massima somma di quantità acquistate in assoluto, restituito in forma di `Optional<Articolo>`.
4. Dimostrare che tutte le interrogazioni sono thread-safe ed operano su copie immutabili o flussi paralleli (`parallelStream()`) senza corruzione dei dati.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: SalesAnalyticsEngineTest.java

```

1 RegistroVendite registro = new RegistroVendite();
2 // Popolamento con ordini di prova...
3 Map<String, Double> fatturato = registro.fatturatoPerCategoria();
4 assert fatturato.containsKey("Hardware") : "Categoria assente!";
5 assert fatturato.get("Hardware") > 0.0 : "Fatturato non positivo!";
6
7 List<Cliente> top3 = registro.clientiTopSpender(3);
8 assert top3.size() == 3 : "Top 3 non corretta!";

```

Esercizio 3.11: [Evolutivo Filone B – Parte 3] CongestionAnalyzer & FlowMetricsSlidingWindow

Livello: Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 3).

Argomento del Task: Espansione del simulatore di traffico (Es. A.2) tramite analisi real-time dei sensori di flusso con Stream: calcolo velocità medie, partizionamento tratte congestionate e sliding window temporale.

Riferimento Didattico: §3.6 e §3.8.

Consegna Operativa: Aggiungere al simulatore stradale cittadino una centralina analitica che elabora i passaggi dei veicoli rilevati dalle telecamere:

1. Modellare l'evento di transito `TransitoRecord` (idSensore, targaVeicolo, timestampMillis, velocitaRilevata);
2. Implementare la classe `AnalizzatoreTraffico` alimentata da un flusso continuo o da una lista di eventi;
3. Realizzare le seguenti metriche analitiche tramite Stream:
 - `velocitaMediaPerTratta()`: mappa ciascuna strada alla velocità media registrata nell'ultimo intervallo (Map<String, Double>);
 - `tratteCongestionate(double velocitaCritica)`: estrae l'insieme delle tratte stradali la cui velocità media è inferiore alla soglia critica, ordinandole per severità del blocco;
 - `rilevazioneInfrazioniVelocita(double limite)`: filtra i veicoli che hanno ecceduto il limite consentito ed emette una stringa CSV formattata con `Collectors.joining("\n")`;
4. Implementare un meccanismo a **Finestra Scorrevole (Sliding Window)** che conserva solo gli eventi degli ultimi 60 secondi ed elimina automaticamente i transiti obsoleti mediante `removeIf()` prima di ogni calcolo statistico.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficAnalyticsTest.java

```

1 AnalizzatoreTraffico analyzer = new AnalizzatoreTraffico();
2 analyzer.registraTransito(new TransitoRecord("TR-01", "MI111AA",
3     System.currentTimeMillis(), 25.0));
4 analyzer.registraTransito(new TransitoRecord("TR-01", "MI222BB",
5     System.currentTimeMillis(), 15.0));
6
7 Map<String, Double> medie = analyzer.velocitaMediaPerTratta();
8 assert medie.get("TR-01") == 20.0 : "Media calcolata errata!";
9 assert analyzer.tratteCongestionate(30.0).contains("TR-01") : "Tratta non
10     rilevata come congestionata!";

```

Sfida Iper-Difficile (High-Performance Systems & Zero-GC HFT Engine)

Esercizio 3.12: MultiGigabyteLogSpliterator & OffHeapPrimitiveHashCollector: Streaming di File Enormi a Basso Livello

Livello: *Livello 3 – Iper-Difficile (Big Data Processing & Custom Spliterator).*

Argomento del Task: Implementazione da zero di `Spliterator<String>` per partizionare file binari/testuali da decine di gigabyte su blocchi fisici di byte senza spezzare le righe, parallelismo trasparente con `StreamSupport.stream()` e collettore personalizzato su array primitivi aperti.

Riferimento Didattico: §3.4 e §B.2.

Consegna Operativa: Quando si elaborano file di log web da 50 o 100 GB, l'utilizzo di `Files.lines()` genera un overhead insostenibile a causa della decodifica UTF-8 riga per riga su un unico thread o dell'impossibilità di partizionare il file in modo perfettamente bilanciato su più core della CPU. Si richiede di costruire un motore di elaborazione parallelo personalizzato a basso livello:

1. **Custom File Spliterator** (`ByteBlockSpliterator`): Implementare l'interfaccia `Spliterator<String>`:
 - Divide il file di input su disco in N segmenti contigui di byte (ad es. 64 MB per blocco);
 - Nel metodo `trySplit()`, partiziona lo spazio di byte e riallinea il cursore al primo carattere di newline ('`\n`') successivo all'offset, assicurando che nessuna riga venga tagliata o elaborata due volte dai thread concorrenti;
 - Implementa `tryAdvance(Consumer<? super String> action)` leggendo direttamente con buffer di byte;
2. **Stream Parallelo:** Costruire lo stream tramite:


```
Stream<String> logStream = StreamSupport.stream(new ByteBlockSpliterator(file), true);
```
3. **Collettore Primitivo a Zero Overhead:** Progettare un'implementazione personalizzata di `Collector` che aggrega le frequenze di 1.000.000 di codici evento su una tabella hash ad indirizzamento aperto basata su array primitivi (`long[] keys`, `int[] counts`) invece di `Map<Long, Integer>`, azzerando l'impronta del Garbage Collector;
4. **Scalabilità Lineare:** Dimostrare su un file di test da 2 GB che il tempo di scansione su una CPU octa-core si riduce di almeno 6× rispetto all'elaborazione sequenziale standard.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: SpliteratorBigDataTest.java

```
1 File logFile = new File("access_massive.log");
2 // Creazione dello stream parallelo a blocchi contigui di byte:
3 Spliterator<String> spliterator = new ByteBlockSpliterator(logFile, 1024 * 1024 *
4     32); // 32MB chunks
5
6 Stream<String> parallelLogs = StreamSupport.stream(spliterator, true);
7
8 long t0 = System.currentTimeMillis();
9 long righeValide = parallelLogs
10     .filter(r -> r.contains(" 500 ")) // Errori server
11     .count();
12 long t1 = System.currentTimeMillis();
13
14 System.out.printf("Righe elaborate: %d in %d ms (Zero-Crash)\n", righeValide, (t1
15     - t0));
16
17 assert righeValide >= 0 : "Conteggio errato!";
```

A.4 Capitolo 4: Interfacce Grafiche Swing ed Architettura ad Eventi

Fascia 1: Base (Finestre e Componenti Atomici)

Esercizio 4.1: Prima Finestra con Swing: Avvio Sicuro su EDT e Interazione Base

Livello: *Livello 1 – Base.*

Argomento del Task: Inizializzazione sicura della GUI su Event Dispatch Thread (`EventQueue.invokeLater()`), `JFrame`, `JButton`, `JLabel`.

Riferimento Didattico: Slide L04 (*PAJC_L04_Eventi&GUI.pdf*) e Lezione 30/09.

Consegna Operativa: Realizzare una finestra Swing con le seguenti specifiche:

1. Avvio tassativamente incapsulato in `EventQueue.invokeLater()`;
2. Titolo della finestra: "Benvenuto in Swing";
3. Chiusura su crocetta impostata con `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`;
4. Contenuto: una `JLabel` centrale con testo "Stato: In Attesa" e un `JButton` con testo "Attiva";
5. Alla pressione del pulsante, il testo della label deve aggiornarsi in "Stato: Operazione Completata con Successo!".

Esercizio 4.2: Studio Comparativo dei Layout Manager al Ridimensionamento

Livello: *Livello 1 – Base.*

Argomento del Task: Verifica empirica della fragilità di `AbsoluteLayout` rispetto alla resilienza di `BorderLayout` e `FlowLayout`.

Riferimento Didattico: Slide L04 e §4.5.

Consegna Operativa: Creare un'applicazione di test contenente tre finestre distinte con 4 pulsanti ciascuna:

1. **Finestra A:** utilizza `setLayout(null)` con coordinate `setBounds()` fisse;
2. **Finestra B:** utilizza `FlowLayout`;
3. **Finestra C:** utilizza `BorderLayout`;
4. Documentare formalmente il comportamento visivo di ciascuna finestra quando l'utente la raddoppia in larghezza o la riduce al 50%.

Fascia 2: Intermedio (Interattività e Listener)

Esercizio 4.3: Contatore Interattivo a Due Vie con Limiti di Soglia

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Sincronizzazione bidirezionale modello-vista, pulsanti di incremento/decremento, validazione input su `JTextField`.

Consegna Operativa: Costruire una GUI per la gestione di un contatore intero:

1. Display centrale costituito da un `JTextField` che visualizza il valore corrente (modificabile anche manualmente dall'utente);
2. Due pulsanti laterali: "-" (decrementa) e "+" (incrementa);
3. Un pulsante inferiore "Reset" che azzerà il contatore;
4. Vincolo di sicurezza: il valore non può scendere sotto zero né superare 100. Se l'utente digita caratteri non numerici, la pressione di Invio deve ripristinare il valore precedente ed emettere un segnale acustico (`Toolkit.getDefaultToolkit().beep()`).

Esercizio 4.4: Convertitore di Temperatura e Valuta con Dialoghi di Validazione

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Gestione eventi semantici, menu a scelta singola (`JComboBox` o radio buttons), gestione dialoghi modali (`JOptionPane`).

Consegna Operativa: Progettare una calcolatrice di conversione unità:

1. Consente di convertire temperature da Celsius a Fahrenheit e Kelvin;
2. Disposizione grafica con `BorderLayout` per la struttura e un `GridLayout(3, 2, 5, 5)` per i campi input/output;
3. Gestione di `NumberFormatException`: se il campo è vuoto o contiene testo alfabetico, intercettare l'errore e mostrare un messaggio di allerta con `JOptionPane.showMessageDialog()`.

Esercizio 4.5: Calcolatrice Grafica Desktop con Integrazione del Motore Funzionale

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Integrazione architetturale MVC, collegamento del `Motore` (Capitolo 2) con un'interfaccia a griglia `GridLayout`.

Consegna Operativa: Assemblare una calcolatrice grafica completa:

1. **Modello:** istanza della classe `Motore` per l'esecuzione dei calcoli aritmetici;
2. **Vista:** `JTextField` superiore per la visualizzazione dell'espressione e griglia 4×4 contenente cifre numeriche (0–9) e simboli (+, -, *, /, =, C);
3. **Controller:** gestione degli stati (primo operando, operatore, secondo operando) evitando azzeramenti imprevisti.

Fascia 3: Avanzato (Componenti Custom e Architettura Event-Driven Multi-file)

Esercizio 4.6: Il Componente Custom PnlKeypad con Incapsulamento ed EventListenerList

Livello: *Livello 3 – Avanzato (Pattern Componente Riutilizzabile).*

Argomento del Task: Componenti compositi che estendono `JPanel`, incapsulamento totale dei widget interni, pattern proxy di eventi, gestione multicast con `EventListenerList`.

Riferimento Didattico: Lezione 30/09 e §4.7.

Consegna Operativa: Reimplementare e raffinare il componente `PnlDigits` presentato a lezione:

1. La classe `PnlKeypad` deve estendere `JPanel` e configurare una griglia 4×3 ;
2. **Incapsulamento rigido:** i 12 pulsanti interni (0–9, "*", "#") devono essere tassativamente privati e inaccessibili all'esterno;
3. Implementare la registrazione multicast esponendo `addActionListener(ActionListener)` e `removeActionListener(ActionListener)`;
4. Utilizzare `EventListenerList` per memorizzare gli ascoltatori ed emettere un nuovo `ActionEvent` avente `PnlKeypad` come sorgente e il tasto premuto come `actionCommand`;
5. Integrare il componente in una finestra che simula l'inserimento di un codice PIN segreto con mascheramento a video.

Esercizio 4.7: SensorTelemetryDashboard – Monitoraggio Concorrente Multi-file con EDT

Livello: *Livello 3 – Avanzato (Architettura a 6 Classi).*

Argomento del Task: Concorrenza tra thread di simulazione in background ed Event Dispatch Thread, disaccoppiamento MVC, listener custom e rendering dinamico.

Consegna Operativa: Costruire una dashboard di controllo per una sala server con monitoraggio di temperatura e carico CPU:

1. `Sensore`: classe eseguibile su thread autonomo (`Runnable`) che genera ciclicamente valori di telemetria con jitter gaussiano;
2. `SensorListener`: interfaccia custom con callback `onTelemetryReceived(SensorEvent e)`;
3. `PnlMetricCard`: componente grafico personalizzato che mostra valore corrente, indicatore colorato (verde per normale, giallo per attenzione, rosso per allarme);

4. **Vincolo di Sicurezza sui Thread:** le notifiche provenienti dal thread del sensore devono essere iniettate sull'EDT esclusivamente tramite `SwingUtilities.invokeLater()` prima di toccare i componenti grafici;
5. Pulsanti di start/stop per ciascun sensore e pannello di riepilogo con log scorrevole (`JScrollPane`).

Esercizio 4.8: VectorPaintStudio – Canvas Grafico Interattivo con PaintComponent e Undo/Redo

Livello: *Livello 3 – Avanzato (Architettura a 7 Classi).*

Argomento del Task: Rendering custom 2D sovrascrivendo `paintComponent(Graphics g)`, gestione eventi a basso livello (`MouseListener`, `MouseMotionListener`), pattern Command con stack di Undo/Redo.

Consegna Operativa: Realizzare un editor vettoriale bidimensionale:

1. `CanvasPanel`: estende `JPanel` e sovrascrive `paintComponent(Graphics g)`, attivando l'antialiasing tramite `Graphics2D`;
2. Toolbar superiore con pulsanti per selezionare la forma da disegnare (Rettangolo, Cerchio, Linea) e il colore corrente (`JColorChooser`);
3. Intercettazione del trascinamento del mouse (*drag*): preview dinamica durante il movimento e memorizzazione della figura al rilascio;
4. Gestione dello storico con due `Deque` di comandi per supportare le operazioni di **Annulla (Undo)** e **Ripristina (Redo)** da tastiera (combinazioni `Ctrl+Z` e `Ctrl+Y`).

Progetti Evolutivi Modulari Interconnessi

Esercizio 4.9: [Evolutivo Filone A – Parte 4] StoreManagerDesktopApp: GUI Swing con JTable, Filtri e Carrello Asincrono

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 4).*

Argomento del Task: Integrazione completa del sistema e-commerce (Es. A.1, Es. A.2, Es. A.3) in un'applicazione grafica desktop Swing con tabella dati (`JTable`), modello personalizzato (`AbstractTableModel`), ordinamento/filtraggio dinamico con Stream/Regex, gestione del carrello acquisti e checkout non bloccante su EDT.

Riferimento Didattico: Slide L04, §4.5, §B.2 e §B.2.

Consegna Operativa: Si richiede di dotare il sistema di vendita realizzato nei capitoli precedenti di una postazione desktop completa per l'addetto alle vendite:

1. **Modello della Tabella** (`ArticoloTableModel`): Estendere `AbstractTableModel` per collegare la lista di articoli dell'`InventoryStore` a una `JTable`. Le colonne devono essere: Codice, Descrizione, Categoria, Prezzo Base (€), Giacenza, Stato. Rendere non modificabili le celle e tipizzare correttamente le classi delle colonne tramite `getColumnClass()`;
2. **Pannello Superiore di Ricerca e Filtraggio:** Inserire un `JTextField` e un selettore di categoria `JComboBox<String>`. Collegare un `TableRowSorter<ArticoloTableModel>` alla tabella per filtrare istantaneamente i record digitati dall'operatore senza ricaricare la vista;
3. **Pannello Laterale del Carrello** (`PnlCartView`): Costruire un componente grafico composito per la gestione del carrello dell'utente in tempo reale:
 - `JList<RigaCarrello>` o mini-tabella con le voci selezionate e relative quantità;
 - Pulsanti "Aggiungi", "Rimuovi" e "Svuota";
 - Selettore profilo cliente (Standard, VIP, Studente) per l'attivazione delle strategie di sconto definite in Es. A.2;
 - Riepilogo contabile dinamico: visualizzazione di sub-totale, sconti applicati dal `FunctionalDiscountEngine` e prezzo finale calcolato;
4. **Checkout Asincrono e Aggiornamento Scorte:** Alla pressione del pulsante "Completa Ordine":

- Verificare la giacenza effettiva nel catalogo;
- Decrementare le quantità in magazzino;
- Registrare la transazione nel `RegistroVendite` (Es. A.3);
- Notificare la tabella con `fireTableDataChanged()`;
- Mostrare una ricevuta riassuntiva in un `JOptionPane.showMessageDialog()` modale. Garantire che l'aggiornamento grafico avvenga interamente nel rispetto del thread EDT (`SwingUtilities.invokeLater()`).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: StoreManagerAppTest.java

```

1 InventoryStore store = new InventoryStore();
2 store.aggiungi(new Articolo("ART-01", "Monitor 4K", "Hardware", 350.0, 10));
3 ArticoloTableModel tableModel = new ArticoloTableModel(store);
4
5 assert tableModel.getRowCount() == 1 : "Conteggio righe errato!";
6 assert tableModel.getValueAt(0, 0).equals("ART-01") : "ID articolo non
   corrispondente!";
7
8 Carrello carrello = new Carrello();
9 carrello.aggiungi(store.trova("ART-01").orElseThrow(), 2);
10 double totale = carrello.calcolaTotale(new DiscountEngineV2());
11 assert totale > 0.0 : "Calcolo carrello nullo o negativo!";
    
```

Esercizio 4.10: [Evolutivo Filone B – Parte 4] TrafficMonitorSimulatorCanvas: Mappa Grafica 2D Animata su Canvas e Swing Timer

Livello: Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 4).

Argomento del Task: Visualizzazione grafica dinamica e simulazione visuale del grafo stradale (Es. A.1), dei semafori a stati finiti (Es. A.2) e del monitoraggio code (Es. A.3) su canvas custom Swing con rendering `Graphics2D`, antialiasing ed animazione a 25+ FPS pilotata da `javax.swing.Timer`.

Riferimento Didattico: §B.2, §B.2 e §B.2.

Consegna Operativa: Integrare il simulatore di mobilità urbana con una dashboard geografica visuale bidimensionale interattiva:

1. **Canvas Grafico della Città** (`PnlTrafficCanvas`): Estendere `JPanel` e sovrascrivere `paintComponent(Graphics g)`:
 - Effettuare il cast a `Graphics2D` ed abilitare il rendering con antialiasing (`RenderingHints.KEY_ANTIALIASING`);
 - Disegnare gli incroci (nodi del grafo) come cerchi con identificativo e semaforo associato;
 - Il semaforo deve essere renderizzato con il colore dello stato corrente restituito dalla macchina a stati (`Rosso`, `Giallo`, `Verde`);
 - Disegnare le tratte stradali (archi del grafo) come linee o corsie collegate tra gli incroci, evidenziando in rosso le tratte congestionate rilevate da `AnalizzatoreTraffico`;
2. **Animazione dei Veicoli con Traslazione Continua:** Ciascun veicolo in transito su una tratta deve essere visualizzato come un rettangolo o cerchietto colorato che si muove dall'incrocio di partenza a quello di arrivo interpolando linearmente le coordinate ad ogni frame:

$$x(t) = x_A + t \cdot (x_B - x_A), \quad y(t) = y_A + t \cdot (y_B - y_A), \quad t \in [0.0, 1.0]$$

3. **Loop di Animazione con `javax.swing.Timer`:** Configurare un timer Swing periodico con frequenza di aggiornamento a 40 ms (≈ 25 frame al secondo) che:
 - Esegue l'avanzamento logico di un quanto temporale nel motore di simulazione;
 - Aggiorna le posizioni dei veicoli e gli stati temporizzati dei semafori;

- Richiama `repaint()` sul pannello canvas;
4. **Pannello Telemetria Laterale:** Collocare a destra (`BorderLayout.EAST`) un pannello informativo con contatori real-time dei veicoli attivi, velocità media cittadina ed un pulsante "Semaforo d'Emergenza" che forza tutti i semafori su `GIALLO_LAMPEGGIANTE`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficCanvasTest.java

```

1 ReteStradale rete = new ReteStradale();
2 Incrocio n1 = new Incrocio("INC-1", 50, 50);
3 Incrocio n2 = new Incrocio("INC-2", 250, 150);
4 rete.aggiungiTratta(new Tratta(n1, n2, 50.0));
5
6 PnlTrafficCanvas canvas = new PnlTrafficCanvas(rete);
7 canvas.setSize(800, 600);
8
9 // Verifica dell'interpolazione fisica di un veicolo:
10 VeicoloSimulato auto = new VeicoloSimulato("MI000XX", n1, n2, 100); // 100 frame
    totali
11 auto.avanzaFrame();
12 Point pos = auto.getPosizioneCorrente();
13 assert pos.x > 50 : "Avanzamento orizzontale fallito!";
14 assert pos.y > 50 : "Avanzamento verticale fallito!";

```

Sfida Iper-Difficile (High-Performance Systems & Zero-GC HFT Engine)

Esercizio 4.11: UltraLowLatencyTelemetryMonitor & DirectRasterRenderer: Oscilloscopio a 60 FPS su DataBufferInt

Livello: *Livello 3 – Iper-Difficile (High-Frequency Real-Time UI & Direct Framebuffer Rasterization).*

Argomento del Task: Progettazione di un visualizzatore grafico ultra-veloce a 60 FPS per streaming di telemetria finanziaria ad altissima frequenza (1.000.000 eventi/sec): superamento dell'overhead del pipeline standard Swing/Graphics2D mediante rasterizzazione diretta a basso livello su array primitivo `int[]` estratto da `DataBufferInt`, decrescita a zero allocazioni (Zero-GC) e blitting atomico su schermo.

Riferimento Didattico: §B.2, §B.2 e §B.2.

Consegna Operativa: Nelle sale operative di High-Frequency Trading o nei sistemi di telemetria avionica, l'aggiornamento di grafici tramite le API convenzionali di `Graphics` (es. invocando `g.drawLine()`) per centinaia di migliaia di punti al secondo o creando istanze di `GeneralPath` ed oggetti `Color` causa il collasso immediato dell'applicazione a causa dei blocchi imposti dalle pause del Garbage Collector e dall'EDT saturato. Si richiede di progettare un oscilloscopio digitale a 60 FPS stabili con politica Zero-GC nel loop di rendering:

1. **Allocazione Singola del Framebuffer Off-Screen:** Istituire un'unica istanza di `BufferedImage` a risoluzione fissa (es. 1280×720) con schema di colore `BufferedImage.TYPE_INT_RGB`. All'inizializzazione, estrarre il riferimento diretto all'array interno dei pixel:


```
int[] pixelBuffer = ((DataBufferInt) image.getRaster().getDataBuffer()).getData();
```

 È formalmente vietato creare nuovi oggetti o riallocare array all'interno del ciclo di repaint;
2. **Algoritmo di Rasterizzazione Vettoriale a Basso Livello:** Implementare un motore di disegno su raster puro:
 - Azzeramento ultra-rapido dello sfondo tramite `Arrays.fill(pixelBuffer, 0x00101820)` (sfondo scuro professionale);
 - Algoritmo di tracciamento linee di Bresenham interamente intero (oppure decrescita min/max su colonna per spettri densi) che scrive direttamente l'esadecimale del colore RGB (es. `0x0000FF00` per il verde fosforo) all'indice calcolato:

$$\text{index} = y \cdot \text{WIDTH} + x$$

con verifica preventiva dei bordi (*bounds checking*) ad altissima efficienza;

3. **Integrazione con il RingBuffer Lock-Free (Es. A.1 ed Es. A.2):** Un thread separato preleva i tick di prezzo/telemetria dal buffer circolare a 1.000.000 eventi/sec e ne calcola la decimazione aggregata su 1280 colonne;

4. **Blit su Schermo a 60 FPS senza Latenza:** All'interno di `paintComponent(Graphics g)`, limitarsi a trasferire l'intero raster con un'unica invocazione:

```
g.drawImage(image, 0, 0, null);
```

e disegnare in sovrapposizione tramite stringa statica gli indicatori prestazionali (FPS effettivi, tempo di rasterizzazione in microsecondi, consumo memoria GC pari a 0 byte/frame).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: DirectRasterRendererTest.java

```
1  int width = 800;
2  int height = 600;
3  BufferedImage img = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);
4  int[] pixels = ((DataBufferInt) img.getRaster().getDataBuffer()).getData();
5
6  assert pixels.length == width * height : "Dimensione raster non corrispondente!";
7
8  DirectOscilloscopeRenderer renderer = new DirectOscilloscopeRenderer(width,
9  height);
10 long memPrima = Runtime.getRuntime().totalMemory() -
11    Runtime.getRuntime().freeMemory();
12
13 // Esecuzione di 100 frame di rasterizzazione intensiva a 1.000.000 punti:
14 for (int i = 0; i < 100; i++) {
15     renderer.renderWaveform(pixels);
16 }
17
18 long memDopo = Runtime.getRuntime().totalMemory() -
19    Runtime.getRuntime().freeMemory();
20 System.out.printf("Delta memoria dopo 100 frame: %d byte (Target: ~0
21    allocazioni)\n", (memDopo - memPrima));
22 assert pixels[0] != 0 : "Buffer non inizializzato con colore di sfondo!";
```

A.5 Capitolo 5: Thread, Concorrenza, Executor e Virtual Threads

Fascia 1: Base (Ciclo di Vita e Sincronizzazione Elementare)

Esercizio 5.1: Calcolo Concorrente di Numeri Primi con Thread, Runnable e Partizionamento Manuale

Livello: *Livello 1 – Base.*

Argomento del Task: Creazione esplicita di thread nativi, interfaccia `Runnable`, partizionamento matematico di intervalli discreti e attesa sincronizzata con `join()`.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.1*) e Slide L04a (Slide 10–18).

Consegna Operativa: Si richiede di realizzare un'applicazione Java per il conteggio dei numeri primi compresi nell'intervallo $[1, N]$ con $N = 10.000.000$:

1. Implementare il predicato matematico `public static boolean isPrime(long n)`;
2. Suddividere l'intervallo totale in M partizioni disgiunte di ampiezza uniforme, dove $M = \text{Runtime.getRuntime().availableProcessors}()$;
3. Istanziare M thread operai passando a ciascuno un'istanza di `Runnable` o estendendo una classe dedicata che memorizza il conteggio parziale;
4. Avviare concorrentemente tutti i thread mediante `start()` (non `run()`!);
5. Nel thread principale (`main`), attendere la conclusione di tutti i worker iterando su un array di thread

ed invocando `join()`;

- Sommare i parziali, stampare il tempo di calcolo complessivo e confrontarlo con l'esecuzione a singolo thread.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: PrimeThreadTest.java

```
1 long max = 1_000_000L;
2 int threads = 4;
3 long totale = ManualThreadPrimeCounter.contaPrimi(max, threads);
4 assert totale == 78498L : "Conteggio numeri primi fino a 1.000.000 errato!";
```

Esercizio 5.2: Migrazione ad ExecutorService con Callable<Long> e Future<Long>

Livello: Livello 1 – Base.

Argomento del Task: Utilizzo del Thread Pool industriale (`FixedThreadPool`), interfaccia tipizzata `Callable<V>`, gestione token `Future<V>` e chiusura controllata del pool.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.2*) e Slide L04a (Slide 20–26).

Consegna Operativa: Rifattorizzare il calcolo dell'Esercizio A.5 eliminando la gestione manuale di `Thread`:

- Istanziare un `ExecutorService` tramite `Executors.newFixedThreadPool(core)`;
- Modellare ciascuna partizione come una lambda expression o classe anonima conforme a `Callable<Long>`;
- Sottomettere i task al pool accumulando le istanze di `Future<Long>` restituite in una `List<Future<Long>`;
- Calcolare la somma totale iterando sulla lista ed invocando `f.get()`;
- Chiudere ordinatamente il pool con `pool.shutdown()` e `pool.awaitTermination(60, TimeUnit.SECONDS)`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ExecutorPrimeTest.java

```
1 ExecutorService pool = Executors.newFixedThreadPool(4);
2 long totale = ExecutorPrimeCounter.calcola(1_000_000L, pool);
3 pool.shutdown();
4 assert totale == 78498L : "Conteggio con Executor errato!";
```

Esercizio 5.3: Dimostrazione Empirica di Race Condition e Disamina Bytecode di count++

Livello: Livello 1 – Base.

Argomento del Task: Analisi forense delle corse critiche su variabili condivise, disassemblaggio bytecode (javap), confronto prestazionale e di correttezza tra non-sincronizzato, `synchronized`, `AtomicLong` e `LongAdder`.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.3*) e Slide L04a (Slide 27–35).

Consegna Operativa:

- Progettare quattro contatori concorrenti:
 - `UnsafeCounter`: campo long count modificato con count++;
 - `SynchronizedCounter`: metodo public synchronized void increment();
 - `AtomicCounter`: basato su `AtomicLong.incrementAndGet()`;
 - `AdderCounter`: basato sulla classe ad alto throughput `LongAdder.increment()`;
- Avviare 4 thread concorrenti che eseguono ciascuno 250.000 incrementi (totale atteso: 1.000.000);
- Dimostrare sperimentalmente che `UnsafeCounter` fallisce sistematicamente producendo un valore

inferiore ad 1.000.000, e spiegare tramite le istruzioni di bytecode `getfield`, `iadd`, `putfield` il motivo esatto della perdita;

4. Misurare e confrontare il tempo di esecuzione in millisecondi dei quattro approcci.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: RaceConditionBytecodeTest.java

```

1  int threads = 4;
2  int incPerThread = 250_000;
3  long atteso = (long) threads * incPerThread; // 1.000.000
4
5  UnsafeCounter c1 = new UnsafeCounter();
6  SynchronizedCounter c2 = new SynchronizedCounter();
7  AtomicCounter c3 = new AtomicCounter();
8  AdderCounter c4 = new AdderCounter();
9
10 ExecutorService pool = Executors.newFixedThreadPool(threads);
11 CountDownLatch latch = new CountDownLatch(threads);
12
13 for (int i = 0; i < threads; i++) {
14     pool.submit(() -> {
15         for (int j = 0; j < incPerThread; j++) {
16             c1.increment();
17             c2.increment();
18             c3.increment();
19             c4.increment();
20         }
21         latch.countDown();
22     });
23 }
24
25 latch.await();
26 pool.shutdown();
27
28 System.out.printf("Esiti -> Unsafe: %d, Sync: %d, Atomic: %d, Adder: %d\n",
29     c1.get(), c2.get(), c3.get(), c4.get());
30
31 assert c1.get() < atteso : "Anomalia: UnsafeCounter non ha prodotto race
32     condition!";
33 assert c2.get() == atteso : "Errore: SynchronizedCounter ha perso incrementi!";
34 assert c3.get() == atteso : "Errore: AtomicCounter ha perso incrementi!";
35 assert c4.get() == atteso : "Errore: AdderCounter ha perso incrementi!";

```

Fascia 2: Intermedio (Pattern di Coordinamento e Utilità Concorrenti)

Esercizio 5.4: Pattern Produttore-Consumatore Canonico con `wait()` e `notifyAll()`

Livello: Livello 2 – Intermedio.

Argomento del Task: Coordinamento primitivo tra thread, monitor intrinseci, gestione della coda circolare a dimensione fissa, prevenzione dei risvegli spuri (*spurious wakeups*) con `while` e notifica a diffusione.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.4*) e Slide L04a.

Consegna Operativa: Implementare una coda bloccante generica `BoundedBuffer<T>` di capacità fissa `K`:

1. Utilizzare un array interno `Object[]` gestito con puntatori circolari di testa e coda;
2. Implementare il metodo `public synchronized void put(T item) throws InterruptedException`: se il buffer è pieno, deve attendere in un loop `while (count == capacity) wait()`;
3. Implementare il metodo `public synchronized T take() throws InterruptedException`: se il

buffer è vuoto, deve attendere in un loop while (count == 0) wait();

- Invocare tassativamente `notifyAll()` dopo ogni inserimento e prelievo per risvegliare i thread concorrenti;
- Collaudare la struttura con 3 thread produttori e 2 thread consumatori, assicurando che nessun elemento venga perso o duplicato.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: BoundedBufferTest.java

```

1 BoundedBuffer<Integer> buffer = new BoundedBuffer<>(5);
2 int totaleElementi = 100;
3 AtomicInteger consumati = new AtomicInteger(0);
4
5 Thread p1 = new Thread(() -> {
6     for (int i = 0; i < 50; i++) {
7         try { buffer.put(i); } catch (InterruptedException e) {
8             Thread.currentThread().interrupt(); }
9     };
10 Thread p2 = new Thread(() -> {
11     for (int i = 50; i < 100; i++) {
12         try { buffer.put(i); } catch (InterruptedException e) {
13             Thread.currentThread().interrupt(); }
14     };
15
16 Runnable consumerTask = () -> {
17     for (int i = 0; i < 50; i++) {
18         try { buffer.take(); consumati.incrementAndGet(); }
19         catch (InterruptedException e) { Thread.currentThread().interrupt(); }
20     };
21 };
22 Thread c1 = new Thread(consumerTask);
23 Thread c2 = new Thread(consumerTask);
24
25 p1.start(); p2.start(); c1.start(); c2.start();
26 p1.join(); p2.join(); c1.join(); c2.join();
27
28 assert consumati.get() == totaleElementi : "Elementi consumati non corrispondenti
29     al totale!";
30 assert buffer.isEmpty() : "Il buffer dovrebbe essere vuoto al termine!";

```

Esercizio 5.5: Produttore-Consumatore Industriale con ArrayBlockingQueue e ReentrantLock

Livello: Livello 2 – Intermedio.

Argomento del Task: Utilizzo avanzato di `ArrayBlockingQueue` ed implementazione con `ReentrantLock` e due code di attesa separate `Condition` (notFull, notEmpty).

Riferimento Didattico: Dispensa D04 (Esercizio 4.5).

Consegna Operativa: Riprogettare il buffer bloccante dell'Esercizio A.5 impiegando i lock espliciti di `java.util.concurrent.locks`:

- Utilizzare un'istanza di `ReentrantLock`;
- Creare due condizioni indipendenti: `Condition notFull = lock.newCondition()` e `Condition notEmpty = lock.newCondition()`;
- Nel metodo `put()`, se la coda è piena invocare `notFull.await()`, e all'inserimento segnalare selettivamente con `notEmpty.signal()`;
- Nel metodo `take()`, se la coda è vuota invocare `notEmpty.await()`, e al prelievo segnalare con `notFull.signal()`;

5. Spiegare perché le due condition distinte offrono prestazioni nettamente superiori a `notifyAll()`, eliminando il risveglio inutile di thread non interessati.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ExplicitLockQueueTest.java

```

1 LockConditionBuffer<String> queue = new LockConditionBuffer<>(10);
2 ExecutorService pool = Executors.newFixedThreadPool(6);
3 CountDownLatch latch = new CountDownLatch(1000);
4
5 for (int i = 0; i < 500; i++) {
6     final String msg = "TICKET-" + i;
7     pool.submit(() -> {
8         try { queue.put(msg); } catch (InterruptedException e) {
9             Thread.currentThread().interrupt(); }
10    });
11    pool.submit(() -> {
12        try { String res = queue.take(); assert res != null; latch.countDown(); }
13        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
14    });
15 }
16
17 boolean completato = latch.await(5, TimeUnit.SECONDS);
18 pool.shutdown();
19
20 assert completato : "Timeout: potenziale stallo o deadlock tra le condition
    notFull e notEmpty!";
21 assert queue.size() == 0 : "Coda non vuota al termine del collaudo!";

```

Esercizio 5.6: Riproduzione Deterministica di Deadlock e Risoluzione con Global Lock Ordering

Livello: *Livello 2 – Intermedio.*

Argomento del Task: Analisi formale dello stallo ciclico tra due lock concorrenti, diagnosi delle 4 condizioni di Coffman e risoluzione architetturale mediante ordinamento globale dei lock.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.6*).

Consegna Operativa:

1. Modellare la classe `ContoCorrente` (id numerico univoco, saldo in euro);
2. Scrivere un metodo ingenuo `trasferisci(ContoCorrente da, ContoCorrente a, double importo)` che acquisisce prima `synchronized(da)` e poi annidato `synchronized(a)`;
3. Avviare due thread che eseguono in contemporanea: Thread 1 trasferisce da Conto 1 a Conto 2, mentre Thread 2 trasferisce da Conto 2 a Conto 1. Verificare che l'applicazione collassa in Deadlock;
4. Riscrivere il metodo applicando il **Global Lock Ordering**: confrontare gli ID numerici dei due conti ed acquisire sempre per primo il lock del conto con ID minore, garantendo che nessun ciclo possa mai generarsi.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: GlobalLockOrderingTest.java

```

1 ContoCorrente cA = new ContoCorrente(101, 1000.0);
2 ContoCorrente cB = new ContoCorrente(102, 1000.0);
3
4 Thread t1 = new Thread(() -> {
5     for (int i = 0; i < 50_000; i++) cA.trasferisciOrdinato(cB, 1.0);
6 });
7 Thread t2 = new Thread(() -> {
8     for (int i = 0; i < 50_000; i++) cB.trasferisciOrdinato(cA, 1.0);
9 });
10
11 t1.start(); t2.start();
12 t1.join(3000); t2.join(3000);
13
14 assert !t1.isAlive() && !t2.isAlive() : "DEADLOCK: i thread risultano ancora
    bloccati in mutua attesa!";
15 assert (cA.getSaldo() + cB.getSaldo()) == 2000.0 : "Violazione dell'invariante di
    conservazione monetaria!";

```

Esercizio 5.7: GUI Reattiva con SwingWorker ed Event Dispatch Thread

Livello: Livello 2 – Intermedio.

Argomento del Task: Integrazione tra concorrenza di calcolo e grafica Swing, disaccoppiamento dei thread di background dall'EDT e aggiornamento progressivo di una `JProgressBar`.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.7*) e §5.9.

Consegna Operativa: Costruire una finestra grafica con un pulsante "Avvia Calcolo", una barra di avanzamento e una label informativa:

1. Alla pressione del pulsante, disabilitarlo ed avviare un task `SwingWorker<Long, Integer>`;
2. Il metodo `doInBackground()` scandisce 10.000.000 di numeri calcolando la primalità;
3. Ogni 100.000 iterazioni, invoca `publish(percentuale)`;
4. Il metodo `process()` aggiorna il valore della `JProgressBar` in modo sicuro sull'EDT;
5. Il metodo `done()` riabilita il pulsante e visualizza il risultato finale;
6. Dimostrare che durante l'elaborazione l'utente può ridimensionare e trascinare la finestra senza alcun blocco visivo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: PrimeSwingWorkerTest.java

```

1 JProgressBar bar = new JProgressBar(0, 100);
2 JLabel status = new JLabel("In attesa");
3 CountDownLatch doneLatch = new CountDownLatch(1);
4
5 PrimeComputationWorker worker = new PrimeComputationWorker(1_000_000L, bar,
    status, doneLatch);
6 worker.execute();
7
8 boolean completato = doneLatch.await(5, TimeUnit.SECONDS);
9 assert completato : "SwingWorker non completato entro il timeout di 5s!";
10 assert bar.getValue() == 100 : "La progress bar non ha raggiunto il 100%!";
11 assert status.getText().contains("78498") : "Risultato finale del conteggio primi
    non aggiornato su label!";

```

Fascia 3: Avanzato (High-Scale Systems & Next-Gen Java)

Esercizio 5.8: Benchmark di Scalabilità Massiva con 100.000 Virtual Threads in Java 21

Livello: *Livello 3 – Avanzato (Java 21 LTS Standard).*

Argomento del Task: Architettura M:N dei Virtual Threads (Project Loom), verifica empirica dell'unmounting su chiamate bloccanti e confronto prestazionale con i Platform Threads del kernel.

Riferimento Didattico: Dispensa D04 (*Esercizio 4.8*) e §5.10.

Consegna Operativa:

1. Scrivere un'applicazione di benchmarking che utilizza `Executors.newVirtualThreadPerTaskExecutor()`;
2. Sottomettere 100.000 task indipendenti, ciascuno dei quali simula una latenza di rete bloccante invocando `Thread.sleep(Duration.ofSeconds(1))`;
3. Utilizzare il costrutto `try-with-resources` sull'executor per attendere automaticamente la conclusione di tutti i task;
4. Misurare il tempo di esecuzione totale e la memoria fisica occupata dalla JVM;
5. Dimostrare teoricamente e sperimentalmente che il medesimo test eseguito su un pool di Platform Threads convenzionali provocherebbe il collasso immediato del sistema per esaurimento della RAM o errore di allocazione nativa del sistema operativo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: VirtualThreadScaleTest.java

```

1  long t0 = System.currentTimeMillis();
2  try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
3      for (int i = 0; i < 100_000; i++) {
4          executor.submit(() -> {
5              Thread.sleep(1000);
6              return 1;
7          });
8      }
9  }
10 long t1 = System.currentTimeMillis();
11 System.out.printf("100.000 Virtual Thread completati in: %d ms (Atteso: ~1.5 -
    2.5 secondi)\n", (t1 - t0));
12 assert (t1 - t0) < 5000 : "Esecuzione virtual thread eccessivamente lenta!";

```

Esercizio 5.9: [Evolutivo Filone A – Parte 5] ConcurrentOrderProcessor & MultiThreadInventorySync

Livello: *Livello 3 – Avanzato (Progetto Evolutivo – Filone Magazzino & E-Commerce, Parte 5).*

Argomento del Task: Elaborazione asincrona concorrente degli ordini di acquisto multithread, prevenzione di over-selling e corse critiche sulle giacenze di magazzino (`InventoryStore`), lock granulari per articolo (`ReentrantLock`), ordinamento globale dei lock contro deadlock multi-articolo, coda di spedizione (`BlockingQueue`) e tracciamento contabile con `LongAdder`.

Riferimento Didattico: §??, §?? ed evoluzione di Es. A.1, Es. A.2, Es. A.3, Es. A.4.

Consegna Operativa: Completare l'infrastruttura di gestione del negozio digitale introducendo l'elaborazione concorrente degli acquisti:

1. **Magazzino Thread-Safe con Lock Granulari:** Estendere `InventoryStore` per consentire a molteplici thread di verificare e scalare le scorte in parallelo. Associare a ciascun codice articolo un `ReentrantLock` dedicato (oppure impiegare `ConcurrentHashMap` con mutazioni atomiche su `compute()`), garantendo che ordini su articoli disgiunti non subiscano contesa né serializzazione;
2. **Elaborazione Asincrona tramite Worker Pool:** Configurare un thread pool (`ExecutorService`) con K worker che consumano ordini da una `BlockingQueue<Ordine>`: i client di checkout (Es. A.4) sottomettano ordini in modo asincrono senza bloccare l'interfaccia utente;

3. **Prenotazione Atomica Multi-Articolo e Anti-Deadlock:** Qualora un carrello contenga più articoli distinti (es. 2 unità di SKU-A e 1 unità di SKU-B), l'acquisizione dei lock di inventario deve avvenire rigorosamente secondo il **Global Lock Ordering** (ordinamento lessicografico crescente degli SKU) per prevenire deadlock tra ordini incrociati. Se un articolo risulta esaurito, l'intera transazione deve essere respinta (politica *All-or-Nothing*) rilasciando immediatamente tutti i lock già acquisiti;
4. **Metriche di Cassa e Notifiche di Esaurimento:** Aggiornare il fatturato globale concorrente mediante `LongAdder` (azzerando la contesa tra core) e notificare gli eventi di esaurimento scorte ai listener registrati tramite `CopyOnWriteArrayList`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ConcurrentOrderProcessorTest.java

```

1  InventoryStore store = new InventoryStore();
2  store.caricaArticolo("ART-01", 100); // 100 unita' disponibili
3  store.caricaArticolo("ART-02", 50);  // 50 unita' disponibili
4
5  ConcurrentOrderProcessor processor = new ConcurrentOrderProcessor(store, 4);
6  int numClienti = 80;
7  CountdownLatch latch = new CountdownLatch(numClienti);
8
9  // 80 clienti concorrenti tentano di acquistare 2 unita' di ART-01 e 1 unita' di
   ART-02:
10 for (int i = 0; i < numClienti; i++) {
11     Map<String, Integer> carrello = Map.of("ART-01", 2, "ART-02", 1);
12     processor.sottomettiOrdine(new Ordine("ORD-" + i, carrello), latch);
13 }
14
15 boolean completato = latch.await(10, TimeUnit.SECONDS);
16 processor.shutdown();
17
18 assert completato : "Timeout nell'elaborazione concorrente degli ordini!";
19 // Essendo disponibili solo 50 unita' di ART-02, esattamente 50 ordini devono
   andare a buon fine e 30 devono fallire:
20 assert processor.getOrdiniCompletati() == 50 : "Over-selling rilevato su ART-02!";
21 assert processor.getOrdiniFalliti() == 30 : "Conteggio ordini falliti errato!";
22 assert store.getGiacenza("ART-01") == 0 : "Giacenza residua di ART-01 errata
   (atteso 0: 100 - 50*2)!";
23 assert store.getGiacenza("ART-02") == 0 : "Giacenza residua di ART-02 errata!";

```

Esercizio 5.10: [Evolutivo Filone B – Parte 5] ConcurrentUrbanTrafficSimulator: Simulatore Multi-Agente con Virtual Threads & Semaphore per Corsie

Livello: Livello 3 – Avanzato (Progetto Evolutivo – Filone Smart City & Mobilità, Parte 5).

Argomento del Task: Simulazione massiva multi-agente di mobilità urbana in Java 21 LTS con 10.000 veicoli concorrenti modellati come Virtual Threads leggeri, gestione della capacità fisica delle corsie tramite `Semaphore`, regolazione degli incroci a precedenza/semaforo e telemetria non bloccante disaccoppiata verso il canvas grafico Swing.

Riferimento Didattico: §5.10, §?? ed evoluzione di Es. A.1, Es. A.2, Es. A.3, Es. A.4.

Consegna Operativa: Portare il simulatore di mobilità urbana alla massima scala computazionale mediante le funzionalità di concorrenza moderna di Java 21:

1. **Veicoli come Virtual Threads Autonomi:** Ogni veicolo immesso nella rete stradale è eseguito come un task indipendente all'interno di un `newVirtualThreadPerTaskExecutor()`; il ciclo vitale del veicolo modella la navigazione passo-passo lungo il grafo stradale, calcolando tempi di transito con attese bloccanti `Thread.sleep()` a zero consumo di thread OS del kernel;
2. **Capacità di Corsia con `Semaphore`:** Ciascuna tratta stradale dispone di una capienza fisica massima di veicoli simultanei K (es. 15 auto). L'ingresso sulla corsia impone l'acquisizione di un permesso con `corsiaSemaphore.acquire()`; il rilascio avviene solo quando il veicolo raggiunge

l'incrocio successivo;

3. **Attraversamento Incroci con Priorità Dinamica:** Negli incroci semaforizzati guidati dalla FSM (Es. A.2), i veicoli transitano solo durante la fase verde. Negli incroci a precedenza, la confluenza tra corsie è protetta da un `ReentrantLock` equo (`fair = true`) per garantire l'assenza di starvation dei flussi secondari;
4. **Telemetria Atomica verso il Canvas Grafico:** Per consentire al canvas Swing (Es. A.4) di visualizzare le posizioni delle migliaia di auto a 25 FPS senza bloccare i thread di simulazione né saturare l'EDT, i veicoli pubblicano le loro coordinate correnti su un buffer ad array atomico (`AtomicReference`), permettendo al timer Swing di estrarre lo snapshot globale in $O(1)$ senza lock di attesa.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ConcurrentTrafficSimulatorTest.java

```

1 ReteStradale rete = new ReteStradale();
2 Incrocio inA = new Incrocio("INC-A", 0, 0);
3 Incrocio inB = new Incrocio("INC-B", 100, 0);
4 TrattaAutostradale tratta = new TrattaAutostradale(inA, inB, 10); // Capienza
   max: 10 veicoli
5 rete.aggiungiTratta(tratta);
6
7 ConcurrentTrafficSimulator sim = new ConcurrentTrafficSimulator(rete);
8 int veicoliTotali = 5000;
9 CountdownLatch traguardoLatch = new CountdownLatch(veicoliTotali);
10
11 long start = System.currentTimeMillis();
12 sim.avviaSimulazioneMassiva(veicoliTotali, traguardoLatch);
13
14 boolean finiti = traguardoLatch.await(15, TimeUnit.SECONDS);
15 long elapsed = System.currentTimeMillis() - start;
16 sim.stop();
17
18 assert finiti : "Timeout: la simulazione con 5.000 veicoli non ha terminato entro
   15s!";
19 assert tratta.getMaxVeicoliRilevati() <= 10 : "Violazione di capienza della
   tratta: superato il limite del semaforo!";
20 System.out.printf("5.000 veicoli Virtual Thread simulati con successo in %d
   ms\n", elapsed);

```

Sfida Iper-Difficile (High-Performance Systems & Zero-Lock Concurrency)

Esercizio 5.11: LockFreeSPSCQueue & RingBufferMemoryBarrier: Coda Lock-Free a Singolo Produttore / Singolo Consumatore con VarHandle e Zero Allocazioni

Livello: Livello 3 – Iper-Difficile (High-Performance Systems & Zero-Lock Concurrency).

Argomento del Task: Progettazione formale da zero di una coda circolare Lock-Free Single-Producer Single-Consumer (SPSC) ad altissima velocità ($> 15.000.000$ op/sec), eliminazione dei blocchi kernel/mutex, distanziamento delle linee di cache (*Cache-Line Padding* di 64 byte contro il False Sharing), barriere di memoria acquire/release pilotate tramite `VarHandle` in Java moderno e pre-allocazione statica (Zero-GC).

Riferimento Didattico: §??, §?? ed evoluzione di Es. A.1, Es. A.2, Es. A.3, Es. A.4.

Consegna Operativa: Nelle architetture software a bassissima latenza (HFT, elaborazione flussi audio realtime, trading engine), le code bloccanti convenzionali (`ArrayBlockingQueue`, `LinkedBlockingQueue`) introducono latenze inaccettabili a causa delle chiamate di sistema del kernel (context switch) e della contesa sui monitor. Si richiede di implementare un ring buffer SPSC lock-free a zero allocazioni:

1. **Struttura Circolare a Potenza di Due:** Pre-allocare un array `Object[]` buffer con dimensione

$C = 2^k$ (es. 65.536). L'operazione di calcolo dell'indice modulo viene convertita in un'operazione bitwise a livello di registri CPU:

$$\text{index} = \text{sequence} \& (\text{CAPACITY} - 1)$$

2. **Cache-Line Padding contro il False Sharing:** I due puntatori sequenziali `head` (aggiornato dal consumatore) e `tail` (aggiornato dal produttore) risiedono tipicamente in locazioni adiacenti. Se allocati sulla medesima cache line di 64 byte della CPU, la modifica da parte di un core invalida continuamente la cache L1 dell'altro core (*False Sharing*). Inserire 7 campi `long` dummy di padding ($7 \times 8 = 56$ byte) prima e dopo ciascun indice per confinare `head` e `tail` su cache line fisicamente distinte;
3. **Semantica di Memoria con `VarHandle`:** Impiegare `VarHandle` per pilotare l'accesso agli indici con semantica acquire/release, azzerando l'overhead di `volatile` completo:
 - Il produttore inserisce l'elemento nell'array e pubblica l'indice con `VarHandle.setRelease(tail, nextTail)`;
 - Il consumatore legge l'indice con `VarHandle.getAcquire(tail)`, garantendo che la lettura del payload avvenga dopo la pubblicazione formale;
4. **Benchmark Comparativo:** Misurare il throughput trasferendo 10.000.000 di elementi tra due thread dedicati e confrontarlo con `ArrayBlockingQueue`, certificando l'assenza di lock e zero allocazioni a regime.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: LockFreeSPSCQueueTest.java

```

1  int capacity = 1024 * 64; // 65536 slot (potenza di due)
2  LockFreeSPSCQueue<Long> queue = new LockFreeSPSCQueue<>(capacity);
3  long items = 10_000_000L;
4  CountDownLatch latch = new CountDownLatch(1);
5
6  long t0 = System.nanoTime();
7  Thread consumer = Thread.ofPlatform().start(() -> {
8      long received = 0;
9      while (received < items) {
10         Long val = queue.poll();
11         if (val != null) received++;
12     }
13     latch.countDown();
14 });
15
16 Thread producer = Thread.ofPlatform().start(() -> {
17     for (long i = 0; i < items; i++) {
18         while (!queue.offer(i)) {
19             Thread.onSpinWait(); // CPU hint di pausa per pipeline out-of-order
20         }
21     }
22 });
23
24 boolean ok = latch.await(10, TimeUnit.SECONDS);
25 long t1 = System.nanoTime();
26 producer.join(); consumer.join();
27
28 assert ok : "Timeout nel consumer della coda SPSC lock-free!";
29 double sec = (t1 - t0) / 1e9;
30 double opsPerSec = items / sec;
31 System.out.printf("SPSC Lock-Free: 10M elementi elaborati in %.3f s (%.1f op/sec,
32     Zero-Lock)\n", sec, opsPerSec);
33 assert opsPerSec > 2_000_000.0 : "Throughput inferiore alla soglia minima
34     prestazionale!";

```

Appendice B

Compendio e Formulario Tecnico di Java

Guida alla Consultazione Rapida

Questo compendio costituisce il riferimento enciclopedico e operativo permanente per lo studio, lo svolgimento degli esercizi di laboratorio e la preparazione dell'esame. È strutturato in due parti complementari:

1. **Parte I – Schemi Concettuali e Principi Architettureali:** compendio teorico sui pilastri della programmazione a oggetti (UML, ereditarietà, polimorfismo, eccezioni, generics, complessità computazionale e algoritmi di ricerca/grafi).
2. **Parte II – Formulario Operativo delle API:** catalogo analitico e tabellare di tutte le classi, interfacce e metodi fondamentali di Java standard, con firme, tipi di ritorno, parametri e descrizioni operative.

B.1 PARTE I: SCHEMI CONCETTUALI E PATTERN ARCHITETTURALI

Questa prima parte raccoglie e formalizza i modelli teorici, le convenzioni grafiche di modellazione a oggetti e i criteri architetturali che governano la progettazione del software in ambiente Java. Vengono approfonditi la notazione UML, i principi cardine di ingegneria del software (incapsulamento, coesione, accoppiamento, aliasing e copie difensive), la tassonomia gerarchica delle eccezioni e l'analisi asintotica comparativa delle collezioni dati.

B.1.1 Notazione UML e Relazioni tra Classi

Nel paradigma a oggetti, il diagramma delle classi UML descrive la struttura statica del sistema specificando entità, attributi, operazioni e relazioni reciproche:

Definizione: Specificatori di Visibilità e Modificatori UML

- **+public**: visibilità totale, accessibile da qualsiasi classe e package.
- **#protected**: accessibile dalle classi dello stesso package e da tutte le sottoclassi.
- **-private**: incapsulamento stretto, accessibile esclusivamente all'interno della classe stessa.
- **~package-private**: visibilità di default (senza modificatore esplicito), riservata al medesimo package.
- **Membri Static**: sottolineati nel diagramma UML, legati alla classe e non all'istanza.
- **Costanti FINAL**: indicate in maiuscolo o con vincolo **{readOnly}**, immutabili dopo l'assegnazione.
- **Classi/Metodi Astratti**: indicati in *corsivo*, contengono firme senza implementazione diretta.

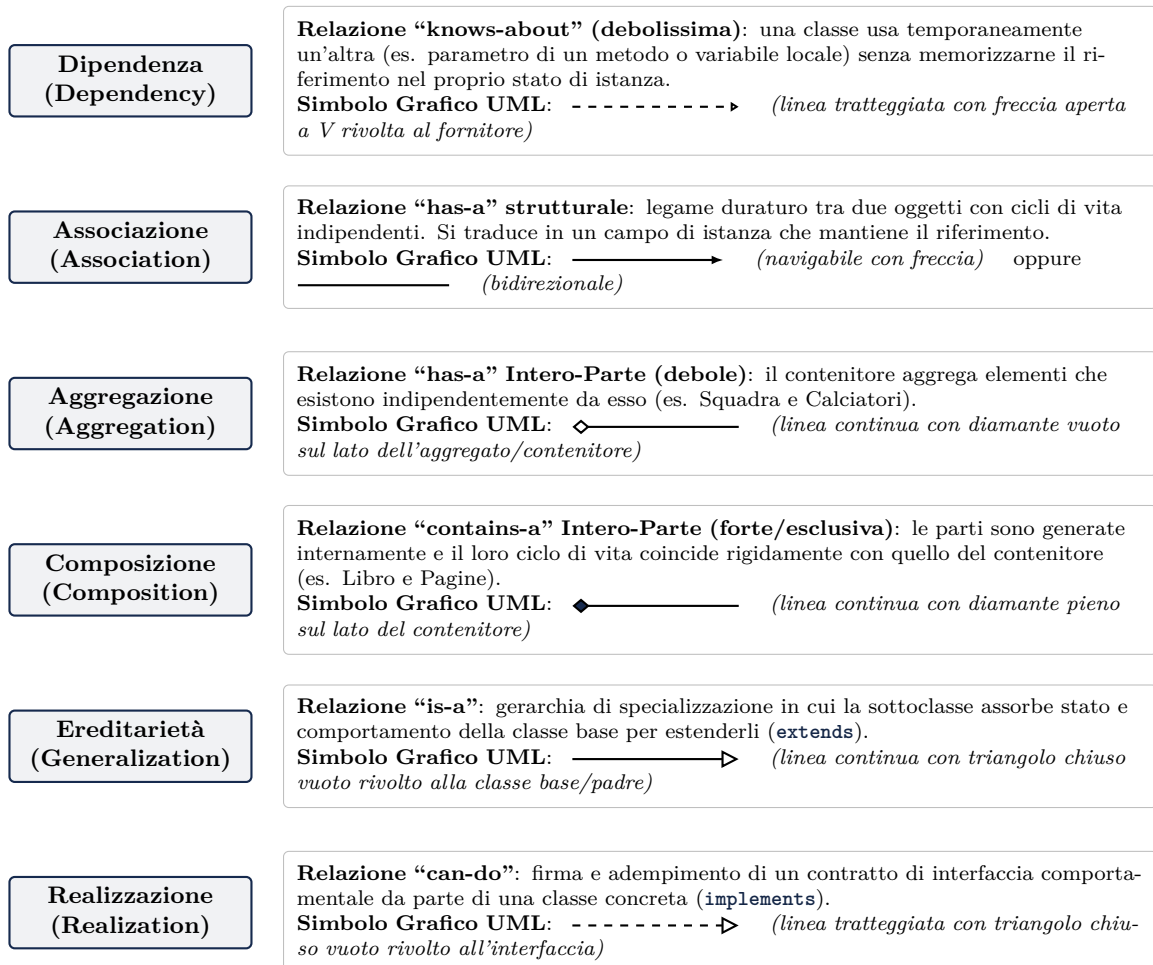


Figura B.1: Quadro sinottico delle relazioni fondamentali nel diagramma delle classi UML.

B.1.2 Principi Guida di Ingegneria del Software

- **Incapsulamento e Information Hiding**: l'accesso ai dettagli interni e alla struttura dei dati di un oggetto è precluso all'esterno; la manipolazione avviene esclusivamente attraverso l'interfaccia pubblica dei metodi, proteggendo gli invarianti di classe.
- **Coesione Elevata (High Cohesion)**: ogni classe o modulo deve perseguire una singola responsabilità logica ben delimitata (*Single Responsibility Principle*). Più una classe è coesa, più è comprensibile, manutenibile e testabile.
- **Accoppiamento Debole (Loose Coupling)**: le classi devono interagire riducendo al minimo le dipendenze reciproche. Un basso accoppiamento isola le modifiche, evitando reazioni a catena nell'intero sistema.
- **Variabili vs Oggetti**: le variabili allocate nello Stack contengono **riferimenti** (puntatori sicuri) ad aree di memoria allocate nello Heap. Un oggetto nello Heap può essere contemporaneamente referenziato da più variabili (fenomeno dell'*Aliasing*).

- **Uguaglianza: == vs equals():** l'operatore == valuta unicamente la coincidenza degli indirizzi di memoria (stessa istanza nello heap). Il metodo equals() valuta l'uguaglianza logica dello stato interno. Qualsiasi classe che sovrascrive equals() deve obbligatoriamente sovrascrivere anche hashCode() per preservare il corretto funzionamento nelle tabelle hash.
- **Shallow Copy vs Deep Copy (Copia Difensiva):**
 - *Shallow Copy*: assegna direttamente il riferimento della lista o dell'oggetto (this.dati = dati;); modifiche esterne si ripercuotono sull'oggetto interno.
 - *Deep Copy (Copia Difensiva)*: crea una struttura dati nuova e indipendente clonando gli elementi (this.dati = new ArrayList<>(dati);), isolando lo stato da interferenze esterne.

B.1.3 Ereditarietà, Polimorfismo e Classi Astratte

Caratteristica	Overloading (Sovraccarico)	Overriding (Sovrascrittura)
Ambito	All'interno della stessa classe (o sottoclasse).	Tra superclasse e sottoclasse.
Firma del metodo	Stesso nome , ma lista parametri differente per tipo o numero.	Firma identica (stesso nome, stessi parametri, tipo ritorno covariante).
Risoluzione	A tempo di compilazione (Static Binding / Early Binding).	A tempo di esecuzione (Dynamic Binding / Late Binding).
Annotazione	Nessuna.	Obbligatoria/Consigliata: @Override.
Scopo	Fornire varianti ergonomiche della stessa operazione.	Specializzare o ridefinire il comportamento di una superclasse.

Proprietà	Classe Astratta (abstract class)	Interfaccia (interface)
Semantica ontologica	Definisce un'identità (Cos'è: IS-A).	Definisce un contratto/protocollo (Cosa sa fare: CAN-DO).
Metodi con corpo	Ammessi sia concreti che astratti.	Ammessi solo se qualificati come default o static (da Java 8).
Variabili di istanza	Ammette qualsiasi campo di stato (private, protected, mutabile).	Ammette esclusivamente costanti pubbliche: implicitamente public static final .
Costruttori	Consentiti (invocati tramite super() dalle figlie).	Vietati (non possiede costruttori).
Ereditarietà	Singola: una classe può estendere (extends) una sola classe base.	Multipla: una classe può implementare (implements) un numero arbitrario di interfacce.
Risoluzione Conflitti	Regola "Le classi vincono" su metodi di default.	Se due interfacce forniscono lo stesso metodo di default, l'override è obbligatorio.

B.1.4 Gestione delle Eccezioni

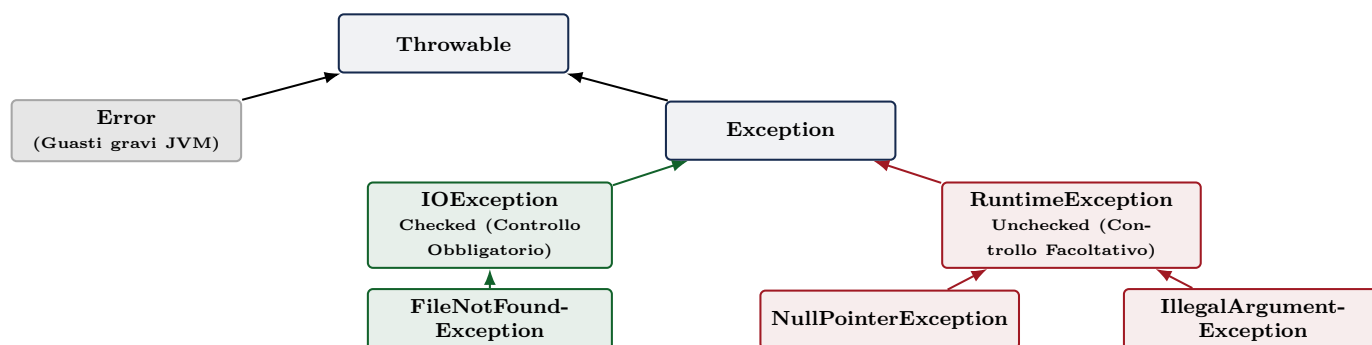


Figura B.2: Gerarchia delle eccezioni in Java: distinzione tra Checked (obbligo di gestione) e Unchecked.

- **Checked Exceptions** (derivano da Exception escludendo RuntimeException): rappresentano condizioni anomale esterne prevedibili (es. file mancante, socket interrotto). Il compilatore impone la clausola throws nella firma o l'incapsulamento in un blocco try-catch.
- **Unchecked Exceptions** (derivano da RuntimeException): rappresentano bug di programmazione (es. puntatore nullo, indice array errato). Non richiedono dichiarazione formale.
- **Clausola finally**: blocco eseguito sempre, sia in caso di successo sia in presenza di eccezione non catturata, indispensabile per il rilascio di risorse.

- **Costrutto try-with-resources:** introdotto per dichiarare risorse che implementano l'interfaccia `AutoCloseable` garantisce l'invocazione automatica di `close()` al termine del blocco.

B.1.5 Complessità Computazionale delle Strutture Dati

Il confronto asintotico che segue è il riferimento operativo per la scelta della struttura dati più adatta a un dato problema: si legge per righe, incrociando il costo delle operazioni fondamentali con il criterio di selezione riportato nella seconda tabella.

Struttura Dati	Accesso Indice	Ricerca Valore	Inserimento	Rimozione
Array primitivo (<code>T[]</code>)	$O(1)$	$O(n)$	N/A (dim. fissa)	N/A (dim. fissa)
<code>ArrayList</code>	$O(1)$	$O(n)$	$O(1)^*$ coda / $O(n)$	$O(n)$
<code>LinkedList</code>	$O(n)$	$O(n)$	$O(1)$ capi / $O(n)$	$O(1)$ capi / $O(n)$
<code>ArrayDeque</code>	$O(1)$ capi	$O(n)$	$O(1)^*$ capi	$O(1)$ capi
<code>HashSet</code> / <code>HashMap</code>	N/A (chiave)	$O(1)$ medio	$O(1)$ medio	$O(1)$ medio
<code>TreeSet</code> / <code>TreeMap</code>	N/A (chiave)	$O(\log n)$	$O(\log n)$	$O(\log n)$

Nota: $O(1)^*$ indica costo costante *ammortizzato* (incrementale su raddoppio buffer o capacità dinamica).

Criteri di Selezione Architetture

Struttura	Quando preferirla	Attenzione (trade-off critico)
Array (<code>T[]</code>)	Dimensione fissa nota a priori; calcolo intensivo su tipi primitivi, senza overhead di boxing/unboxing.	Nessun ridimensionamento: la dimensione è congelata all'istanziamento.
<code>ArrayList</code>	Scelta di default per le collezioni sequenziali; ottima per accessi casuali e append in coda.	Riallocazione a +50% con copia; inserimento/rimozione centrale $O(n)$ per lo <i>shift</i> degli elementi.
<code>LinkedList</code>	Inserimenti/rimozioni ripetuti solo agli estremi, o iterazione bidirezionale pesante con <code>ListIterator</code> .	Overhead $\sim 24-32$ B/nodo, scarsa cache locality; <code>get(i)</code> resta $O(n)$.
<code>ArrayDeque</code>	Code FIFO, pile LIFO, buffer ad anello: da preferire sempre a <code>LinkedList</code> e alla classe obsoleta <code>Stack</code> .	Nessun accesso efficiente a un indice arbitrario interno.
<code>HashSet</code> / <code>HashMap</code>	Lookup a costo pressoché costante per chiave, quando l'ordine è irrilevante.	<code>equals()</code> / <code>hashCode()</code> devono essere coerenti; <i>rehashing</i> $O(n)$ oltre il load factor 0.75.
<code>TreeSet</code> / <code>TreeMap</code>	Serve un ordinamento garantito o operazioni di range (<code>subMap()</code> , <code>headSet()</code>).	Costo $O(\log n)$ su ogni operazione; unicità basata su <code>compareTo() == 0</code> , non su <code>equals()</code> .

B.2 PARTE II: FORMULARIO OPERATIVO DELLE API DI JAVA

Classe Fondamentale `java.lang.Object`

- `boolean equals(Object obj)` → `boolean`

Verifica l'uguaglianza logica tra l'istanza corrente e l'oggetto passato.

Implementazione di default: confronta unicamente i riferimenti di memoria (`this == obj`).

Contratto formale dell'override: la relazione deve essere *riflessiva* (`x.equals(x)`), *simmetrica* (`x.equals(y) == y.equals(x)`), *transitiva*, *coerente* e *non-nulla* (`x.equals(null) == false`).

Parametri: `obj` l'oggetto di riferimento con cui confrontare l'istanza.

Esempio canonico:

```
if (this == obj) return true;
if (obj == null || getClass() != obj.getClass()) return false;
Persona other = (Persona) obj;
return eta == other.eta && Objects.equals(nome, other.nome);
```

- `int hashCode()` → `int`
Restituisce il codice hash intero associato all'oggetto, utilizzato dalle strutture dati hash (`HashMap`, `HashSet`, `Hashtable`).
Contratto cardinale: se `a.equals(b) == true`, allora tassativamente `a.hashCode() == b.hashCode()`. Due oggetti diversi possono avere lo stesso hash (collisione), ma una buona funzione deve distribuire i valori uniformemente.
Esempio: `return Objects.hash(nome, eta);`
- `String toString()` → `String`
Restituisce la rappresentazione testuale dell'oggetto. Di default: `NomeClasse@hexHashCode`. Da sovrascrivere sempre per agevolare il debug e la diagnostica nei log.
- `final Class<?> getClass()` → `Class<?>`
Restituisce l'oggetto riflettivo che descrive il tipo dinamico effettivo dell'istanza a run-time. Essendo **final**, non può essere sovrascritto.
- `protected Object clone() throws CloneNotSupportedException` → `Object`
Crea e restituisce una copia superficiale (*shallow copy*) campo a campo dell'oggetto.
Vincolo: la classe deve implementare l'interfaccia marcatore `Cloneable`, pena il sollevamento dell'eccezione checked `CloneNotSupportedException`.
- `final void wait() throws InterruptedException` → `void`
Sospende il thread corrente e rilascia temporaneamente il lock intrinseco del monitor dell'oggetto, fino a quando un altro thread non invoca `notify()` o `notifyAll()` sullo stesso monitor.
Variante con timeout: `wait(long timeoutMillis)` attende al massimo il tempo specificato. Deve essere sempre invocato all'interno di un blocco o metodo **synchronized**.
- `final void notify()` / `final void notifyAll()` → `void`
Risveglia rispettivamente un singolo thread arbitrario (`notify()`) oppure tutti i thread (`notifyAll()`) in attesa sul monitor dell'oggetto. Deve essere invocato con il lock acquisito.

Manipolazione di Stringhe: `String`, `StringBuilder` e `CharSequence`

Classe Immutabile `java.lang.String`

- `char charAt(int index)` → `char`
Restituisce il carattere presente all'indice specificato (0-based). Lancia `IndexOutOfBoundsException` se l'indice è negativo o $\geq \text{length}()$.
- `int length()` → `int`
Restituisce il numero totale di caratteri (unità di codice UTF-16) presenti nella stringa.
- `String substring(int beginIndex, int endIndex)` → `String`
Estrae la porzione di stringa nell'intervallo semichiuso `[beginIndex, endIndex)`.
Parametri: `beginIndex` indice iniziale (*incluso*); `endIndex` indice finale (*escluso*).
Esempio: `"Universita".substring(0, 3)` restituisce `"Uni"`.
Variante a 1 parametro: `substring(int beginIndex)` estrae fino al termine della stringa.
- `boolean equals(Object anObject)` / `boolean equalsIgnoreCase(String another)` → `boolean`
Confronta il contenuto testuale carattere per carattere. `equalsIgnoreCase` ignora le differenze tra maiuscole e minuscole.
- `int compareTo(String anotherString)` → `int`
Confronto lessicografico: restituisce un valore < 0 se l'istanza precede, 0 se identiche, > 0 se segue.
Variante: `compareToIgnoreCase(String str)`.
- `int indexOf(String str)` / `int lastIndexOf(String str)` → `int`
Restituisce l'indice della prima (o ultima) occorrenza della sottostringa, oppure -1 se assente.
Varianti: ammettono un secondo parametro `int fromIndex` da cui avviare la ricerca.

- `boolean contains(CharSequence s)` → `boolean`
Verifica se la sequenza di caratteri è contenuta nella stringa.
- `boolean startsWith(String prefix)` / `boolean endsWith(String suffix)` → `boolean`
Verifica se la stringa inizia o termina con la sequenza specificata.
- `boolean isEmpty()` / `boolean isBlank()` → `boolean`
`isEmpty()` ritorna `true` se `length() == 0`; `isBlank()` (Java 11) ritorna `true` anche se la stringa contiene solo caratteri di spaziatura (*whitespaces*).
- `String trim()` / `String strip()` → `String`
Rimuovono gli spazi iniziali e finali. `strip()` (Java 11) è conforme a Unicode (rimuove tutti i caratteri di spaziatura internazionali). Disponibili anche `stripLeading()` e `stripTrailing()`.
- `String toLowerCase()` / `String toUpperCase()` → `String`
Restituisce una nuova stringa con tutti i caratteri convertiti in minuscolo o maiuscolo.
- `String replace(CharSequence target, CharSequence replacement)` → `String`
Sostituisce tutte le occorrenze letterali della sequenza `target` con `replacement`.
- `String replaceAll(String regex, String replacement)` → `String`
Sostituisce tutte le corrispondenze dell'espressione regolare `regex` con la stringa di rimpiazzo.
- `String[] split(String regex)` / `String[] split(String regex, int limit)` → `String[]`
Suddivide la stringa in un array di token delimitati dalla `regex`.
- `static String join(CharSequence delimiter, CharSequence... elements)` → `String`
Concatena gli elementi interponendo il delimitatore specificato.
Esempio: `String.join("-", "2026", "10", "04")` produce `"2026-10-04"`.
- `static String format(String format, Object... args)` → `String`
Genera una stringa formattata secondo gli specificatori standard (%d intero, %s stringa, %.2f decimale).
- `String repeat(int count)` → `String`
(Java 11) Restituisce una nuova stringa composta dalla ripetizione dell'istanza per `count` volte.
- `Stream<String> lines()` → `Stream<String>`
(Java 11) Restituisce un flusso lazy di righe estratte dalla stringa, separate da `\n`, `\r` o `\r\n`.

Classe Mutabile `java.lang.StringBuilder`

- `StringBuilder append(T val)` → `StringBuilder`
Accoda la rappresentazione testuale del valore (primitivo, oggetto o stringa) al buffer corrente. Restituisce `this` per consentire l'*inlining* fluente.
- `StringBuilder insert(int offset, T val)` → `StringBuilder`
Inserisce la sequenza alla posizione `offset`, traslando a destra i caratteri successivi.
- `StringBuilder delete(int start, int end)` → `StringBuilder`
Rimuove i caratteri compresi nell'intervallo `[start, end)`.
Variante: `deleteCharAt(int index)` elimina il singolo carattere alla posizione indicata.
- `StringBuilder replace(int start, int end, String str)` → `StringBuilder`
Sostituisce i caratteri dal range `[start, end)` con la nuova stringa fornita.
- `StringBuilder reverse()` → `StringBuilder`
Inverte specularmente l'ordine di tutti i caratteri memorizzati nel buffer.
- `void setCharAt(int index, char ch)` → `void`
Sovrascrive il carattere alla posizione `index` con `ch` senza allocare una nuova stringa.
- `int length()` / `int capacity()` / `void setLength(int newLength)` → `int/void`
Rispettivamente il numero di caratteri effettivi, la capienza del buffer interno prima del raddoppio e la modifica forzata della lunghezza (tronca o riempie con null-byte).

Java I/O Classico: File, Stream e Reader/Writer (java.io)

Astrazione File System: `java.io.File`

- `boolean exists()` → `boolean`
Verifica se il file o la directory esiste fisicamente sul supporto di memorizzazione.
- `boolean isFile()` / `boolean isDirectory()` → `boolean`
Verifica se il percorso punta rispettivamente a un file regolare oppure a una directory.
- `long length()` → `long`
Restituisce la dimensione del file espressa in byte (restituisce 0L se il file non esiste).
- `String getName()` / `String getAbsolutePath()` → `String`
Restituisce rispettivamente il solo nome del file/cartella o il percorso assoluto canonico.
- `boolean createNewFile() throws IOException` → `boolean`
Crea atomicamente un nuovo file vuoto; restituisce `false` se il file esiste già.
- `boolean delete()` → `boolean`
Elimina fisicamente il file o la directory (le cartelle devono essere vuote per essere rimosse).
- `boolean mkdir()` / `boolean mkdirs()` → `boolean`
Crea la cartella singola (`mkdir`) o l'intera gerarchia di directory padre mancanti (`mkdirs`).
- `String[] list()` / `File[] listFiles()` → `String[]` / `File[]`
Restituisce i nomi o gli oggetti `File` relativi a tutti i figli contenuti nella cartella.
- `boolean renameTo(File dest)` → `boolean`
Rinomina o sposta il file verso la destinazione specificata da `dest`.
- `boolean canRead()` / `boolean canWrite()` / `boolean canExecute()` → `boolean`
Interroga i permessi del sistema operativo per l'accesso in lettura, scrittura o esecuzione.

Parsing Testuale: `java.util.Scanner`

- `String nextLine()` → `String`
Legge e consuma l'intera riga fino al terminatore di riga (attenzione ai caratteri `\n` residui!).
- `String next()` → `String`
Legge il prossimo token delimitato da spazi bianchi.
- `int nextInt()` / `double nextDouble()` → `int/double`
Converte il prossimo token nel rispettivo tipo primitivo (solleva `InputMismatchException` se non compatibile).
- `boolean hasNext()` / `boolean hasNextInt()` → `boolean`
Verifica preventivamente se è presente un token del tipo specificato prima di leggerlo.
- `Scanner useDelimiter(String pattern)` → `Scanner`
Imposta l'espressione regolare utilizzata come delimitatore tra i token.

Stream Bufferizzati e Stampa Formattata

- `BufferedReader.readLine()` → `String` (throws `IOException`)
Legge una riga di testo; restituisce `null` quando viene raggiunto l'End-Of-File (EOF).
- `BufferedWriter.write(String str)` → `void` (throws `IOException`)
Scriva la stringa nel buffer di memoria.
- `BufferedWriter.newLine()` → `void` (throws `IOException`)
Scriva il terminatore di riga nativo del sistema operativo ospitante.
- `PrintWriter.printf(String format, Object... args)` → `PrintWriter`
Scriva testo formattato su stream (file o socket) con la medesima sintassi di `System.out.printf`.

Flussi Binari, Serializzazione e Accesso Diretto

- `DataOutputStream` `.writeInt(int v)` / `DataInputStream` `.readInt()` → `void` / `int`
 Scrive e legge dati numerici primitivi in formato binario nativo (Big-Endian a 4 byte), evitando overhead testuale.
- `ObjectOutputStream` `.writeObject(Object obj)` → `void` (throws `IOException`)
 Serializza lo stato dell'oggetto e dell'intero grafo di oggetti collegati su flusso binario (`Serializable`).
- `ObjectInputStream` `.readObject()` → `Object` (throws `IOException`, `ClassNotFoundException`)
 Deserializza e ricostruisce l'oggetto dallo stream binario; richiede il cast esplicito al tipo concreto.
Modificatore transient: i campi contrassegnati con `transient` vengono ignorati e non serializzati.
- `RandomAccessFile` `.seek(long pos)` → `void` (throws `IOException`)
 Riposiziona il puntatore interno di lettura/scrittura all'offset byte assoluto `pos` (*accesso casuale*).
- `RandomAccessFile` `.getFilePointer()` → `long` (throws `IOException`)
 Restituisce la posizione corrente del puntatore all'interno del file espressa in byte.

Modern File I/O: `java.nio.file.Path` e `java.nio.file.Files` (Java 8–21)

Interfaccia di Percorso `java.nio.file.Path`

- `static Path of(String first, String... more)` → `Path`
 (Java 11) Factory method moderno raccomandato per istanziare un percorso (sostituisce `Paths.get()`).
Esempio: `Path p = Path.of("dati", "2026", "report.csv");`
- `Path getFileName()` / `Path getParent()` / `Path toAbsolutePath()` → `Path`
 Estrae rispettivamente l'ultimo elemento del percorso, la directory genitrice o la risoluzione assoluta.
- `Path resolve(String other)` / `Path resolve(Path other)` → `Path`
 Risolve e concatena un percorso relativo rispetto al path corrente.

Operazioni Massive su File: Classe di Utilità `java.nio.file.Files`

- `lines(path)` → `Stream<String>` (throws `IOException`)
Firma: `static Stream<String> lines(Path path)`
Flusso Lazy di Righe: apre il file e ne legge le righe su richiesta in uno `Stream<String>`. Consumo di memoria $O(1)$, ideale per file giganteschi.
Regola tassativa: lo stream implementa `AutoCloseable`; deve essere racchiuso in un blocco `try-with-resources` per garantire il rilascio immediato del file descriptor nel sistema operativo.
Esempio:

```
try (Stream<String> stream = Files.lines(Path.of("server.log"))) {
    long errori = stream.filter(line -> line.contains("ERROR")).count();
}
```
- `readAllLines(path)` → `List<String>` (throws `IOException`)
Firma: `static List<String> readAllLines(Path path)`
 Legge interamente il file in memoria caricando tutte le righe in una lista di stringhe ($O(n)$ in memoria RAM).
- `readString(path)` → `String` (throws `IOException`)
Firma: `static String readString(Path path)`
 (Java 11) Legge l'intero contenuto del file direttamente in una singola stringa decodificata in UTF-8.
- `writeString(path, csq, options...)` → `Path` (throws `IOException`)
Firma: `static Path writeString(Path path, CharSequence csq, OpenOption... options)`
 (Java 11) Scrive atomicamente la stringa specificata nel file. Con `StandardOpenOption.CREATE`, `StandardOpenOption.APPEND` esegue l'append in coda.
- `write(path, lines, options...)` → `Path` (throws `IOException`)

Firma: `static Path write(Path path, Iterable<? extends CharSequence> lines, OpenOption... opts)`

Scrive una sequenza iterabile di righe all'interno del file.

- `exists(path, options...)` → `boolean`

Firma: `static boolean exists(Path path, LinkOption... options)`

Verifica l'esistenza reale del file o cartella sul file system.

- `copy(source, target, options...)` → `Path` (throws `IOException`)

Firma: `static Path copy(Path source, Path target, CopyOption... options)`

Copia il file da sorgente a destinazione. Opzione chiave: `StandardCopyOption.REPLACE_EXISTING`.

- `move(source, target, options...)` → `Path` (throws `IOException`)

Firma: `static Path move(Path source, Path target, CopyOption... options)`

Sposta o rinomina atomicamente il file.

- `delete(path)` / `deleteIfExists(path)` → `void` / `boolean` (throws `IOException`)

Firma: `static void delete(Path path)`, `static boolean deleteIfExists(Path path)`

Elimina il file indicato; la variante `deleteIfExists` non lancia eccezione se il file è assente.

- `createDirectories(dir, attrs...)` → `Path` (throws `IOException`)

Firma: `static Path createDirectories(Path dir, FileAttribute<?>... attrs)`

Crea l'intera gerarchia di directory mancanti senza sollevare errori se alcune cartelle esistono già.

- `walk(start, maxDepth, options...)` → `Stream<Path>` (throws `IOException`)

Firma: `static Stream<Path> walk(Path start, int maxDepth, FileVisitOption... options)`

Attraversa ricorsivamente l'albero delle directory a partire da `start` fino a `maxDepth` livelli, restituendo un flusso computazionale di percorsi.

Interfaccia Radice `Collection<E>` e Iteratori (`java.util`)

- `boolean add(E e)` → `boolean`

Inserisce l'elemento; ritorna `true` se la collezione è stata modificata, `false` se l'elemento era già presente (negli insiemi `Set`).

Parametri: `e` l'elemento conforme al vincolo generico da inserire.

- `boolean addAll(Collection<? extends E> c)` → `boolean`

Inserisce tutti gli elementi contenuti nella collezione specificata (*unione*).

- `boolean remove(Object o)` → `boolean`

Rimuove una singola occorrenza dell'oggetto basandosi sul confronto logico `equals()`.

- `boolean removeAll(Collection<?> c)` → `boolean`

Rimuove tutti gli elementi presenti anche nella collezione passata (*differenza insiemistica*).

- `boolean retainAll(Collection<?> c)` → `boolean`

Mantiene unicamente gli elementi presenti anche nella collezione passata (*intersezione*).

- `boolean removeIf(Predicate<? super E> filter)` → `boolean`

(Java 8) Rimuove in un unico passaggio tutti gli elementi che soddisfano il predicato booleano.

Parametri: `filter` predicato funzionale che restituisce `true` per gli elementi da eliminare.

Esempio: `studenti.removeIf(s -> s.getMedia() < 18.0);`

- `boolean contains(Object o)` / `boolean containsAll(Collection<?> c)` → `boolean`

Verifica la presenza del singolo elemento o di tutti gli elementi della collezione fornita.

- `void clear()` → `void`

Svuota integralmente la collezione rimuovendo ogni riferimento.

- `boolean isEmpty()` / `int size()` → `boolean/int`

Rispettivamente verifica se la dimensione è zero e restituisce il numero di elementi contenuti.

- `Iterator<E> iterator()` → `Iterator<E>`
Restituisce l'iteratore standard per attraversare la sequenza di elementi.
- `Object[] toArray()` → `Object[]`
Converte la collezione in un array nativo di tipo `Object[]`.
- `<T> T[] toArray(T[] a)` → `T[]`
Converte la collezione in un array del tipo specificato; se `a` ha capienza sufficiente viene riutilizzato, altrimenti ne viene allocato uno nuovo della stessa dimensione.
Esempio: `String[] arr = lista.toArray(new String[0]);`
- `default <T> T[] toArray(IntFunction<T[]> generator)` → `T[]`
(Java 11) Converte la collezione in array tipizzato usando un costruttore di array come generator.
Esempio: `String[] arr = lista.toArray(String[]::new);`
- `default Stream<E> stream()` / `default Stream<E> parallelStream()` → `Stream<E>`
Crea rispettivamente uno stream sequenziale o parallelo supportato dal framework Fork/Join.

Interfaccia Posizionale `List<E>` e `ListIterator<E>`

- `E get(int index)` → `E`
Restituisce l'elemento posizionato all'indice specificato ($O(1)$ su `ArrayList`, $O(n)$ su `LinkedList`).
Parametri: `index` indice posizionale ($0 \leq \text{index} < \text{size}()$).
- `E set(int index, E element)` → `E`
Sostituisce l'elemento all'indice e restituisce il valore precedentemente memorizzato.
- `void add(int index, E element)` → `void`
Inserisce l'elemento all'indice indicato, traslando a destra di una posizione tutti gli elementi successivi.
- `E remove(int index)` → `E`
Elimina l'elemento all'indice specificato, trasla a sinistra i successivi e restituisce il valore eliminato.
- `int indexOf(Object o)` / `int lastIndexOf(Object o)` → `int`
Restituisce la posizione della prima (o ultima) occorrenza dell'oggetto, oppure -1 se assente.
- `List<E> subList(int fromIndex, int toIndex)` → `List<E>`
Restituisce una **vista** (*view*) della lista compresa nell'intervallo `[fromIndex, toIndex)`.
Attenzione: le modifiche apportate alla vista si riflettono direttamente sulla lista originale e viceversa.
- `default void sort(Comparator<? super E> c)` → `void`
(Java 8) Ordina la lista sul posto secondo il comparatore passato (passare `null` per ordine naturale).
Esempio: `lista.sort(Comparator.naturalOrder());`
- `default void replaceAll(UnaryOperator<E> operator)` → `void`
(Java 8) Sostituisce ciascun elemento della lista con il risultato dell'applicazione dell'operatore.
Esempio: `parole.replaceAll(String::toUpperCase);`
- `static <E> List<E> of(E... elements)` → `List<E>`
(Java 9) Factory method per creare una lista **immutabile e non modificabile**. Non accetta elementi `null` (solleva `NullPointerException`).
Esempio: `List<String> immutabile = List.of("A", "B", "C");`
- `static <E> List<E> copyOf(Collection<? extends E> coll)` → `List<E>`
(Java 10) Restituisce una copia immutabile indipendente della collezione fornita.

Iteratore Bidirezionale `ListIterator<E>`

- `boolean hasPrevious()` / `E previous()` → `boolean` / `E`
Verifica l'esistenza dell'elemento precedente e lo estrae muovendo il cursore a ritroso.
- `int nextIndex()` / `int previousIndex()` → `int`
Restituisce l'indice posizionale che verrebbe letto con la successiva chiamata a `next()` o `previous()`.

- `void set(E e)` / `void add(E e)` → `void`
`set` sovrascrive l'ultimo elemento restituito; `add` inserisce un elemento nella posizione corrente.

Code e Pile: `Queue<E>`, `Deque<E>` e `ArrayDeque<E>`

Operazione	Solleva Eccezione	Valore Speciale	Descrizione Comportamentale
Inserimento	<code>boolean add(E e)</code>	<code>boolean offer(E e)</code>	Inserisce in coda; restituisce <code>false</code> se la coda è piena.
Rimozione	<code>E remove()</code>	<code>E poll()</code>	Estrae dalla testa; restituisce <code>null</code> se vuota.
Ispezione	<code>E element()</code>	<code>E peek()</code>	Legge la testa senza estrarla; restituisce <code>null</code> se vuota.
Coda a Due Estremi (<code>Deque<E></code>) e Comandi Pila LIFO			
Inserimento Estremi	<code>addFirst(e)</code> / <code>addLast(e)</code>	<code>offerFirst(e)</code> / <code>offerLast(e)</code>	Inserisce rispettivamente in testa o in coda.
Rimozione Estremi	<code>removeFirst()</code> / <code>removeLast()</code>	<code>pollFirst()</code> / <code>pollLast()</code>	Estrae rispettivamente dalla testa o dalla coda.
Ispezione Estremi	<code>getFirst()</code> / <code>getLast()</code>	<code>peekFirst()</code> / <code>peekLast()</code>	Legge la testa o la coda senza estrazione.
Operazioni Pila	<code>push(E e)</code> (= <code>addFirst</code>)	<code>pop()</code> (= <code>removeFirst</code>)	Gestione a pila LIFO (<i>Last-In First-Out</i>).

Dizionari e Mappe: `Map<K, V>` e Metodi Funzionali Java 8+

- `V get(Object key)` → `V`
Restituisce il valore associato alla chiave specificata, oppure `null` se la chiave è assente.
- `V put(K key, V value)` → `V`
Associa il valore alla chiave. Se la chiave era già presente, sovrascrive il valore e restituisce il precedente valore memorizzato (oppure `null` se nuova).
- `V getOrDefault(Object key, V defaultValue)` → `V`
Restituisce il valore mappato se presente; se la chiave non esiste restituisce il valore `defaultValue`.
Esempio: `int conteggio = mappa.getOrDefault("parola", 0);`
- `V putIfAbsent(K key, V value)` → `V`
Inserisce la coppia chiave-valore solo se la chiave non è presente o è attualmente associata a `null`.
- `V remove(Object key)` / `boolean remove(Object key, Object value)` → `V` / `boolean`
La prima variante rimuove la chiave e rende il valore; la seconda variante rimuove l'entry solo se la chiave è attualmente associata esattamente a quell'oggetto `value`.
- `boolean containsKey(Object key)` / `boolean containsValue(Object value)` → `boolean`
Verifica rapida della chiave ($O(1)$ su `HashMap`) o scansione lineare dei valori ($O(n)$).
- `Set<K> keySet()` / `Collection<V> values()` / `Set<Map.Entry<K, V> entrySet()`
Restituiscono rispettivamente l'insieme delle chiavi, la raccolta dei valori e l'insieme delle coppie chiave-valore.

Metodi di Calcolo Funzionale Moderni (Java 8+)

- `V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)` → `V`
Se la chiave non è presente (o è associata a `null`), calcola il valore tramite la funzione passata e lo inserisce nella mappa. Restituisce il valore finale.
Parametri: `key` la chiave da cercare; `mappingFunction` la funzione che genera il valore a partire dalla chiave.
Esempio pratico (Creazione automatica di liste innestate):

```
Map<String, List<Studente>> perCorso = new HashMap<>();
perCorso.computeIfAbsent("Informatica", k -> new ArrayList<>()).add(studente);
```

- `V computeIfPresent(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)`**
 $\rightarrow V$
 Se la chiave è presente e non-nulla, calcola un nuovo valore usando chiave e vecchio valore. Se la funzione restituisce null, **rimuove la chiave dalla mappa**.
- `V merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)`**
 $\rightarrow V$
Algoritmo di Fusione Chiave-Valore: se la chiave non esiste o è associata a null, associa direttamente `value`. Altrimenti calcola il nuovo valore fondendo il vecchio e il nuovo tramite la funzione binaria (`remappingFunction`). Se la funzione restituisce null, elimina la chiave.
Parametri: `key` chiave; `value` valore da associare se assente; `remappingFunction` funzione $(v_{old}, v_{new}) \rightarrow v_{merged}$.
Esempio canonico (Frequenziario perfetto):

```
Map<String, Integer> frequenze = new HashMap<>();
for (String parola : testo) {
    frequenze.merge(parola, 1, Integer::sum);
}
```
- `void forEach(BiConsumer<? super K, ? super V> action)`** $\rightarrow \text{void}$
 Itera su ogni coppia della mappa invocando la lambda specificata.
Esempio: `mappa.forEach((k, v) -> System.out.println(k + " = " + v));`
- `static <K, V> Map<K, V> of(K k1, V v1, ...)`** / **`static <K, V> Map<K, V> copyOf(Map<K, V> m)`**
 $\rightarrow \text{Map<K, V>}$
 (Java 9/10) Creano mappe non modificabili e immutabili. Sollevano eccezione in presenza di chiavi duplicate o valori null.

Confronto e Ordinamento: `Comparable<T>`, `Comparator<T>` e Costruzione Fluida

Ordinamento Naturale Intrinseco: `Comparable<T>`

- `int compareTo(T other)`** $\rightarrow \text{int}$
 Definisce l'ordine canonico dell'oggetto confrontato con l'istanza `other`.
Convenzione numerica: restituisce un valore intero strettamente < 0 se `this < other`; restituisce 0 se lo stato è equivalente; restituisce > 0 se `this > other`.

Ordinamento Esterno e Combinatori Fluanti: `Comparator<T>`

- `int compare(T o1, T o2)`** $\rightarrow \text{int}$
 Metodo SAM: restituisce < 0 se $o_1 < o_2$, 0 se uguali, > 0 se $o_1 > o_2$.
- `static <T, U extends Comparable<? super U> Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)`**
 $\rightarrow \text{Comparator<T>}$
 Costruisce un comparatore estraendo una chiave ordinabile secondo il suo ordine naturale.
Esempio: `Comparator<Persona> comp = Comparator.comparing(Persona::getCognome);`
- `static <T> Comparator<T> comparingInt(ToIntFunction<? super T> keyExtractor)`** $\rightarrow \text{Comparator<T>}$
 Variante specializzata per chiavi intere primitive che azzera l'overhead di boxing/unboxing. Disponibili anche `comparingDouble` e `comparingLong`.
Esempio: `Comparator.comparingInt(Persona::getEta);`
- `default Comparator<T> thenComparing(Comparator<? super T> other)`** $\rightarrow \text{Comparator<T>}$
Criterio di spareggio (tie-breaker): se il comparatore corrente restituisce 0 (parità), valuta il secondo comparatore fornito.
Esempio combinato:

```
Comparator<Persona> comp = Comparator.comparing(Persona::getCognome)
                                     .thenComparing(Persona::getNome)
                                     .thenComparingInt(Persona::getEta);
```

- `default Comparator<T> reversed()` → `Comparator<T>`
Inverte la direzione dell'ordinamento applicato dal comparatore corrente.
- `static <T extends Comparable<? super T>> Comparator<T> naturalOrder()` /
`reverseOrder()` → `Comparator<T>`
Restituiscono comparatori basati sull'ordine naturale intrinseco o sul suo inverso.
- `static <T> Comparator<T> nullsFirst(Comparator<? super T> comparator)` /
`nullsLast(...)` → `Comparator<T>`
Adattatori sicuri per gestire valori null posizionandoli rispettivamente in testa o in coda.

Libreria Matematica: `java.lang.Math`

- `static double abs(double a)` / `static int abs(int a)` → `double` / `int`
Restituisce il valore assoluto non negativo del numero specificato (disponibile anche per `long` e `float`).
- `static double max(double a, double b)` / `static int max(int a, int b)` → `double` / `int`
Restituisce il valore maggiore tra i due argomenti passati (disponibili analoghi metodi `min`).
- `static double pow(double base, double exp)` → `double`
Calcola l'elevamento a potenza base^{exp} .
- `static double sqrt(double a)` / `static double cbrt(double a)` → `double`
Calcola rispettivamente la radice quadrata positiva ($\sqrt{a}$) o la radice cubica ($\sqrt[3]{a}$).
- `static double hypot(double x, double y)` → `double`
Restituisce l'ipotenusa $\sqrt{x^2 + y^2}$ calcolata prevenendo overflow o underflow intermedi.
- `static double ceil(double a)` / `static double floor(double a)` → `double`
Arrotonda per eccesso (il più piccolo intero $\geq a$) o per difetto (il più grande intero $\leq a$).
- `static long round(double a)` / `static int round(float a)` → `long` / `int`
Arrotonda il numero all'intero più vicino (sommando 0.5 e applicando il troncamento).
- `static double sin(double a)` / `static double cos(double a)` /
`static double tan(double a)` → `double`
Calcola le funzioni trigonometriche di un angolo espresso rigidamente in **radianti**.
- `static double toRadians(double angdeg)` / `static double toDegrees(double angrad)` → `double`
Converte la misura di un angolo da gradi sessagesimali a radianti e viceversa.
- `static double random()` → `double`
Genera un valore pseudocasuale distribuito uniformemente nell'intervallo reale $[0.0, 1.0)$.
- `static double log(double a)` / `static double log10(double a)` /
`static double exp(double a)` → `double`
Calcola il logaritmo naturale ($\ln a$), il logaritmo in base 10 ($\log_{10} a$) e l'esponenziale (e^a).
- `static int clamp(long value, int min, int max)` → `int`
(Java 21) Costringe e delimita il valore fornito all'interno dell'intervallo chiuso $[\text{min}, \text{max}]$.

Manipolazione di Array: `java.util.Arrays` e Utilità `java.util.Collections`

Classe di Utilità `java.util.Arrays`

- `sort(a)` / `sort(a, comparator)` → `void`
Firme: `static void sort(T[] a)`, `static <T> void sort(T[] a, Comparator<? super T> c)`

Ordina l'array sul posto secondo l'ordine naturale o il comparatore specificato.

- `parallelSort(a)` → `void`

Firma: `static void parallelSort(T[] a)`

Ordina l'array sfruttando il multithreading Fork/Join (consigliato per array con $> 10^5$ elementi).

- `binarySearch(a, key)` / `binarySearch(a, c, key)` → `int`

Firme: `static <T> int binarySearch(T[] a, T key)`, `static <T> int binarySearch(T[] a, Comparator<? super T> c, T key)`

Ricerca dicotomica $O(\log n)$: l'array **deve essere preventivamente ordinato**. Restituisce l'indice se trovato; se assente restituisce un indice negativo $-(\text{insertion point}) - 1$.

- `equals(a1, a2)` / `deepEquals(a1, a2)` → `boolean`

Firme: `static boolean equals(T[] a, T[] a2)`, `static boolean deepEquals(Object[] a1, Object[] a2)`

Confronta due array monodimensionali (`equals`) o multidimensionali annidati (`deepEquals`).

- `copyOf(original, newLength)` → `T[]`

Firma: `static <T> T[] copyOf(T[] original, int newLength)`

Clona l'array estendendolo o troncandolo alla nuova dimensione indicata.

- `fill(a, val)` → `void`

Firma: `static void fill(T[] a, T val)`

Assegna il valore `val` a tutti gli elementi dell'array.

- `asList(a...)` → `List<T>`

Firma: `static <T> List<T> asList(T... a)`

Crea una lista a dimensione fissa basata sull'array sottostante (modifiche ai valori si riflettono sull'array).

- `stream(array)` → `Stream` / `IntStream`

Firme: `static <T> Stream<T> stream(T[] a)`, `static IntStream stream(int[] a)`

Genera uno stream sequenziale a partire da un array di oggetti o tipi primitivi.

Classe di Utilità `java.util.Collections`

- `static void shuffle(List<?> list)` / `static void reverse(List<?> list)` → `void`

Rimescola casualmente gli elementi in ordine uniforme o inverte specularmente l'ordine della lista.

- `static <T extends Comparable<? super T>> T min(Collection<? extends T> coll)` → `T`

Restituisce il valore minimo della collezione (anche con `Comparator<? super T>`).

- `static int frequency(Collection<?> c, Object o)` → `int`

Conta quante volte l'oggetto `o` appare all'interno della collezione passata.

- `static boolean disjoint(Collection<?> c1, Collection<?> c2)` → `boolean`

Restituisce `true` se le due collezioni non hanno elementi in comune (*disgiunte*).

- `static <T> List<T> unmodifiableList(List<? extends T> list)` → `List<T>`

Restituisce una vista non modificabile della lista (disponibile anche per `unmodifiableSet`, `unmodifiableMap`).

Programmazione Funzionale: Catalogo Completo `java.util.function`

Interfacce Funzionali Fondamentali (Single Abstract Method – SAM)

- `Supplier<T>` → metodo SAM: `T get()`

Produttore puro: fornisce un valore di tipo `T` senza ricevere alcun parametro in ingresso.

Esempio: `() -> new Random(); () -> Double.valueOf(3.14).`

- `Consumer<T>` → metodo SAM: `void accept(T t)`

Consumatore: riceve un valore `T` ed esegue un'azione/effetto collaterale senza ritornare nulla.

Composizione: `Consumer<T> andThen(Consumer<? super T> after)` esegue prima l'azione corren-

te e poi quella successiva.

Esempio: `s -> System.out.println(s).`

- `BiConsumer<T, U>` → metodo SAM: `void accept(T t, U u)`
Consumatore a due argomenti: riceve due operandi ed esegue un'operazione senza ritorno.
- `Function<T, R>` → metodo SAM: `R apply(T t)`
Trasformatore: mappa un valore in ingresso di tipo T in un risultato di tipo R .
Composizione fluente: `andThen(Function after)` calcola $g(f(x))$; `compose(Function before)` calcola $f(g(x))$.
Metodo statico: `Function.identity()` restituisce l'argomento ricevuto senza alterarlo.
Esempio: `s -> s.length(); Integer::toBinaryString.`
- `BiFunction<T, U, R>` → metodo SAM: `R apply(T t, U u)`
Funzione binaria eterogenea: mappa due argomenti di tipo diverso (T, U) in un risultato R .
- `Predicate<T>` → metodo SAM: `boolean test(T t)`
Criterio di verifica booleano: valuta una condizione sull'elemento T .
Composizione logica: `and(Predicate other)`, `or(Predicate other)`, `negate()`.
Metodo statico: `Predicate.not(Predicate target)` (Java 11: nega un method reference predicato).
Esempio: `s -> s.isEmpty(); Predicate.not(String::isBlank).`
- `BiPredicate<T, U>` → metodo SAM: `boolean test(T t, U u)`
Predicato binario: valuta una condizione logica tra due argomenti (T, U) .
- `UnaryOperator<T>` → estende `Function<T, T>`, SAM: `T apply(T t)`
Operatore unario omogeneo: trasformazione chiusa sullo stesso tipo $T \rightarrow T$.
Esempio: `s -> s.toUpperCase(); x -> x * 2.`
- `BinaryOperator<T>` → estende `BiFunction<T, T, T>`, SAM: `T apply(T t1, T t2)`
Operatore binario omogeneo: calcola un valore da due operandi dello stesso tipo $(T, T) \rightarrow T$.
Metodi statici: `BinaryOperator.maxBy(Comparator)`, `BinaryOperator.minBy(Comparator)`.
Esempio: `(a, b) -> a + b; Math::max.`

Specializzazioni Primitive (Zero-Boxing Overhead)

- `IntConsumer`, `LongConsumer`, `DoubleConsumer` (`void accept(tipo v)`).
- `IntPredicate`, `LongPredicate`, `DoublePredicate` (`boolean test(tipo v)`).
- `ToIntFunction<T>` (`int applyAsInt(T value)`), `ToLongFunction<T>`, `ToDoubleFunction<T>`.
- `IntFunction<R>` (`R apply(int value)`), `IntToDoubleFunction`, `IntToLongFunction`.
- `IntUnaryOperator` (`int applyAsInt(int)`), `IntBinaryOperator` (`int applyAsInt(int, int)`).
- `ObjIntConsumer<T>` (`void accept(T t, int v)`), `ObjDoubleConsumer<T>`, `ObjLongConsumer<T>`.

Sintassi e Tassonomia Completa dei Method Reference

Le Quattro Categorie Canoniche (qualificatore::nome)

- **1. Metodo d'Istanza su Oggetto Specifico:** `istanzaEsistente::metodo`
 L'oggetto bersaglio è già vincolato e noto a priori; gli argomenti della lambda diventano argomenti del metodo.
Sintassi: `System.out::println` $\equiv$ `x -> System.out.println(x)`
- **2. Metodo Statico di Classe:** `NomeClasse::metodoStatico`
 Non vi è alcun oggetto ricevente; i parametri della lambda vengono passati direttamente al metodo statico.
Sintassi: `Math::max` $\equiv$ `(a, b) -> Math.max(a, b);` `Integer::sum` $\equiv$ `(a, b) -> Integer.sum(a, b)`

- **3. Metodo d'Istanza su Ricevente Arbitrario:** `NomeClasse::metodoDiIstanza`

Regola chiave: il **primo parametro** della lambda funge da oggetto bersaglio (*receiver*) su cui viene invocato il metodo; gli eventuali parametri successivi diventano gli argomenti.

Sintassi 1 param: `String::trim` $\equiv$ `s -> s.trim()`

Sintassi 2 param: `String::compareTo` $\equiv$ `(s, t) -> s.compareTo(t)`

- **4. Costruttore:** `NomeClasse::new`

Riferimento al costruttore per istanziazione differita del tipo specificato.

Sintassi: `ArrayList::new` $\equiv$ `() -> new ArrayList<>();` `String[]::new` $\equiv$ `size -> new String[size]`

Stream API: Pipeline, Flussi Primitivi e `Optional<T>` (`java.util.stream`)

Creazione di Stream

- `collezione.stream()` / `collezione.parallelStream()` $\rightarrow$ `Stream<E>`
Crea uno stream sequenziale o parallelo (gestito dal framework concorrente Fork/Join).
- `Arrays.stream(T[] array)` $\rightarrow$ `Stream<T>`
Crea uno stream da array (disponibile anche per primitivi: `IntStream`, `LongStream`, `DoubleStream`).
- `Stream.of(T... values)` / `Stream.empty()` $\rightarrow$ `Stream<T>`
- `Stream.iterate(T seed, UnaryOperator<T> f)` $\rightarrow$ `Stream<T>`
Genera uno stream infinito ordinato: $s_0, f(s_0), f(f(s_0)), \dots$
- `Stream.iterate(T seed, Predicate<? super T> hasNext, UnaryOperator<T> next)` $\rightarrow$ `Stream<T>`
(Java 9) Versione con predicato di arresto simile al ciclo `for(seed; hasNext; next)`.
- `Stream.generate(Supplier<? extends T> s)` $\rightarrow$ `Stream<T>`
Genera uno stream infinito non ordinato alimentato da un fornitore di valori.

Operazioni Intermedie (Lazy – Valutazione Pigra)

- `Stream<T> filter(Predicate<? super T> predicate)` $\rightarrow$ `Stream<T>`
Mantiene unicamente gli elementi che soddisfano la condizione booleana del predicato.
- `<R> Stream<R> map(Function<? super T, ? extends R> mapper)` $\rightarrow$ `Stream<R>`
Trasforma ciascun elemento da tipo T a tipo R tramite la funzione fornita (trasformazione 1-a-1).
- `IntStream mapToInt(ToIntFunction<? super T> mapper)` $\rightarrow$ `IntStream`
Trasforma lo stream in un flusso numerico primitivo a zero boxing. Analoghi: `mapToLong`, `mapToDouble`.
- `<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R> mapper)` $\rightarrow$ `Stream<R>`
Appiattisce stream annidati: mappa ciascun elemento in uno stream e fonde tutti i flussi in un unico stream piano (trasformazione 1-a-molti).
- `Stream<T> distinct()` $\rightarrow$ `Stream<T>`
Elimina i duplicati preservando l'ordine (*stateful*, si basa su `equals()` e `hashCode()`).
- `Stream<T> sorted()` / `Stream<T> sorted(Comparator<? super T> comparator)` $\rightarrow$ `Stream<T>`
Ordina gli elementi dello stream (*stateful*, non applicabile su flussi infiniti senza limit).
- `Stream<T> peek(Consumer<? super T> action)` $\rightarrow$ `Stream<T>`
Esegue un'azione non invasiva (tipicamente log o debug) su ciascun elemento senza alterare il flusso.
- `Stream<T> limit(long maxSize)` / `Stream<T> skip(long n)` $\rightarrow$ `Stream<T>`
Tronca lo stream ai primi `maxSize` elementi o scarta i primi `n` elementi.
- `Stream<T> takeWhile(Predicate<? super T> predicate)` $\rightarrow$ `Stream<T>`

(Java 9) Estrae gli elementi dall'inizio finché il predicato è `true`; al primo fallimento tronca lo stream.

- `Stream<T> dropWhile(Predicate<? super T> predicate) → Stream<T>`

(Java 9) Scarta gli elementi finché il predicato è `true`, emettendo tutti i successivi non appena fallisce.

Operazioni Terminali (Eager – Consumano e Chiudono lo Stream)

- `void forEach(Consumer<? super T> action) / void forEachOrdered(...) → void`

Esegue l'azione su ciascun elemento. `forEachOrdered` rispetta l'ordine anche in parallelo.

- `Object[] toArray() / <A> A[] toArray(IntFunction<A[]> generator) → A[]`

Raccoglie gli elementi in un array tipizzato.

Esempio: `String[] arr = stream.toArray(String[]::new);`

Focus Riduzioni: I Tre Overload Fondamentali di `reduce`

- `reduce(accumulator) → Optional<T>`

Firma formale: `Optional<T> reduce(BinaryOperator<T> accumulator)`

Esegue una riduzione associativa senza valore iniziale. Restituisce un `Optional` vuoto se lo stream è privo di elementi.

Parametri: `accumulator` operatore binario associativo $(T, T) \rightarrow T$.

Esempio: `Optional<Integer> max = numeri.stream().reduce(Integer::max);`

- `reduce(identity, accumulator) → T`

Firma formale: `T reduce(T identity, BinaryOperator<T> accumulator)`

Esegue la riduzione partendo da un valore di identità. Ritorna `identity` se lo stream è vuoto.

Parametri: `identity` elemento neutro dell'operatore ($op(identity, x) = x$); `accumulator` funzione $(T, T) \rightarrow T$.

Esempio: `int sum = numeri.stream().reduce(0, (a, b) -> a + b);`

- `reduce(identity, accumulator, combiner) → U`

Firma formale: `<U> U reduce(U identity, BiFunction<U, ? super T, U> acc, BinaryOperator<U> comb)`

Riduzione Generalizzata con Cambio di Tipo: consente di accumulare gli elementi dello stream (di tipo T) in un risultato finale di tipo completamente differente (U).

Scomposizione Analitica dei Tre Componenti:

- `identity`: valore iniziale di tipo U (restituito se lo stream è vuoto); deve fungere da elemento neutro per il combinatore: $combiner(u, identity) = u$.
- `accumulator`: funzione $(U, T) \rightarrow U$ che incorpora un elemento dello stream (T) nel valore parziale accumulato (U).
- `combiner`: funzione binaria associativa $(U, U) \rightarrow U$ invocata negli **stream paralleli** per fondere i risultati parziali prodotti concorrentemente dai vari worker thread. In esecuzione strettamente sequenziale non viene eseguita, ma è richiesta dal contratto formale della firma.

Esempio Operativo (Calcolo della lunghezza complessiva di tutte le stringhe in parallelo):

```
int totaleCaratteri = listaStringhe.parallelStream()
    .reduce(0,
        (parziale, str) -> parziale + str.length(), // Accumulatore (U, T) -> U
        Integer::sum);                             // Combinatore (U, U) -> U
```

Focus Raccoglitori: I Due Overload di `collect`

- `collect(collector) → R`

Firma formale: `<R, A> R collect(Collector<? super T, A, R> collector)`

Riduzione mutabile terminale standard delegata a un'istanza di `Collector` (vedi scheda successiva).

- `collect(supplier, accumulator, combiner) → R`

Firma formale: `<R> R collect(Supplier<R> sup, BiConsumer<R, ? super T> acc, BiConsumer<R, R> comb)`

Riduzione mutabile diretta senza istanziare formalmente un `Collector`.

Ruolo dei Tre Parametri:

- `supplier`: factory che istanzia il contenitore di accumulo mutabile (es. `ArrayList::new`).
- `accumulator`: azione che aggiunge un elemento T al contenitore mutabile R (es. `List::add`).
- `combiner`: azione che unisce due contenitori mutabili R in caso di stream parallelo (es. `List::addAll`).

Esempio: `List<String> out = stream.collect(ArrayList::new, List::add, List::addAll);`

Altre Operazioni Terminali

- `long count()` → `long`
Restituisce il numero totale di elementi transitati nello stream.
- `Optional<T> min(Comparator<? super T> comp)` / `Optional<T> max(...)` → `Optional<T>`
Estrae il valore minimo o massimo in base al comparatore fornito.
- `boolean anyMatch(Predicate)` / `allMatch(Predicate)` / `noneMatch(Predicate)` → `boolean`
Valutazioni a cortocircuito (*short-circuit*): interrompono l'elaborazione non appena l'esito è certo.
- `Optional<T> findFirst()` / `Optional<T> findAny()` → `Optional<T>`
Estrae il primo elemento (rispetta l'ordine di incontro) o un elemento qualsiasi (ottimizzato per parallelo).

Flussi Primitivi a Zero-Boxing (`IntStream`, `LongStream`, `DoubleStream`)

- `IntStream.range(int start, int end)` / `rangeClosed(start, end)` → `IntStream`
Genera intervallo numerico consecutivo escluso o incluso l'estremo superiore `end`.
- `int sum()` / `OptionalDouble average()` / `OptionalInt min()` / `OptionalInt max()`
Operazioni aritmetiche aggregate immediate senza creazione di oggetti wrapper.
- `IntSummaryStatistics summaryStatistics()` → `IntSummaryStatistics`
Calcola contemporaneamente conteggio, somma, minimo, massimo e media in un solo passaggio (`getCount()`, `getSum()`, `getAverage()`, etc.).
- `Stream<Integer> boxed()` → `Stream<Integer>`
Effettua il boxing degli elementi primitivi riconvertendoli nel rispettivo stream di wrapper.
- `<U> Stream<U> mapToObj(IntFunction<? extends U> mapper)` → `Stream<U>`
Converte il flusso primitivo in un flusso di oggetti mappando ciascun primitivo.

La Classe Contenitore Antinull: `java.util.Optional<T>`

- `of(value)` → `Optional<T>` (solleva `NullPointerException` se nullo).
- `ofNullable(value)` → `Optional<T>` (restituisce `empty()` se nullo).
- `empty()` → `Optional<T>`
- `isPresent()` / `isEmpty()` → `boolean` (Java 11: verifica presenza o assenza).
- `get()` → `T` (estrae il valore; lancia `NoSuchElementException` se vuoto: usare con cautela!).
- `ifPresent(action)` → `void` (esegue l'azione se il valore è presente).
- `ifPresentOrElse(action, emptyAction)` → `void`
(Java 9) Se presente esegue l'azione sul valore, altrimenti esegue l'azione di fallback `emptyAction`.
- `filter(predicate)` → `Optional<T>`
- `map(mapper)` → `Optional<U>`
- `flatMap(mapper)` → `Optional<U>`

Firma: `<U> Optional<U> flatMap(Function<? super T, ? extends Optional<? extends U> mapper)`

Applica la funzione evitando l'annidamento `Optional<Optional<U>`.

- `or(supplier)` → `Optional<T>`
(Java 9) Ritorna lo stesso `Optional` se presente, altrimenti restituisce l'alternativa prodotta dal supplier.
- `stream()` → `Stream<T>`
(Java 9) Converte l'optional in uno stream di 0 o 1 elemento (eccellente per `flatMap(Optional::stream)`).
- `orElse(other)` → `T` (restituisce il valore di default se vuoto).
- `orElseGet(supplier)` → `T` (valutazione lazy del valore di fallback).
- `orElseThrow()` / `orElseThrow(excSupplier)` → `T`
Firme: `T orElseThrow(), <X extends Throwable> T orElseThrow(Supplier<? extends X> exc) throws X`

Collettori Avanzati: Catalogo della Classe `java.util.stream.Collectors`

Accumulo in Collezioni e Mappe

- `toList()` / `toUnmodifiableList()` → `Collector`
Firme: `static <T> Collector<T, ?, List<T>> toList(), toUnmodifiableList()`
Accumula gli elementi in una `List` mutabile o non modificabile (Java 10).
- `toSet()` / `toUnmodifiableSet()` → `Collector`
Firme: `static <T> Collector<T, ?, Set<T>> toSet(), toUnmodifiableSet()`
Accumula gli elementi in un `Set` eliminando automaticamente i duplicati.
- `toCollection(factory)` → `Collector`
Firma: `static <T, C extends Collection<T>> Collector<T, ?, C> toCollection(Supplier<C> factory)`
Accumula in un'implementazione specifica.
Esempio: `Collectors.toCollection(TreeSet::new)`.
- `toMap(keyMapper, valueMapper)` → `Collector`
Firma: `static <T, K, U> Collector<T, ?, Map<K, U>> toMap(keyMapper, valueMapper)`
Raccoglie in `Map`. **Attenzione:** lancia `IllegalStateException` in caso di chiavi duplicate!
- `toMap(keyMapper, valueMapper, mergeFunction)` → `Collector`
Firma: `static <T, K, U> Collector<T, ?, Map<K, U>> toMap(keyM, valM, BinaryOperator<U> mergeFn)`
Risolve le collisioni di chiave applicando la funzione di merge binaria.
Esempio: `toMap(Persona::getCitta, Persona::getNome, (n1, n2) -> n1 + ", " + n2)`.
- `toMap(keyM, valM, mergeFn, mapFactory)` → `Collector`
Firma: `static <T, K, U, M extends Map<K, U>> Collector<T, ?, M> toMap(keyM, valM, mergeFn, Supplier<M> factory)`
Consente di specificare la classe concreta della mappa di destinazione (es. `TreeMap::new`).

Manipolazione Testuale

- `joining()` / `joining(del)` / `joining(del, prefix, suffix)` → `Collector`
Firme: `static Collector<CharSequence, ?, String> joining([del[, prefix, suffix]])`
Concatena le stringhe con eventuale delimitatore, prefisso e suffisso.
Esempio: `joining(", ", "[", "]")` produce `"[A, B, C]"`.

Raggruppamento e Partizionamento

- `groupingBy(classifier)` → `Collector`
Firma: `static <T, K> Collector<T, ?, Map<K, List<T>>> groupingBy(Function<? super T, ? extends K> classifier)`
Raggruppa gli elementi per chiave producendo una mappa `Map<K, List<T>>`.

- `groupingBy(classifier, downstream)` → `Collector`
Firma: `static <T, K, A, D> Collector<T, ?, Map<K, D> groupingBy(classifier, Collector downstream)`
 Raggruppa gli elementi e applica una riduzione secondaria (*downstream*) sui valori del gruppo.
Esempio (Conteggio elementi per città):

```
Map<String, Long> perCitta = persone.stream()
    .collect(Collectors.groupingBy(Persona::getCitta, Collectors.counting()));
```
- `groupingBy(classifier, mapFactory, downstream)` → `Collector`
Firma: `static <T, K, D, M extends Map<K, D> Collector<T, ?, M> groupingBy(classifier, Supplier<M> factory, downstream)`
 Specifica l'implementazione della mappa restituita (es. `TreeMap::new` per avere chiavi ordinate).
- `partitioningBy(predicate)` / `partitioningBy(predicate, downstream)` → `Collector`
Firme: `static <T> Collector<T, ?, Map<Boolean, List<T>> partitioningBy(Predicate<? super T> predicate,`
`static <T, D, A> Collector<T, ?, Map<Boolean, D> partitioningBy(predicate, Collector downstream)`
 Partiziona gli elementi in due liste in base al valore booleano del predicato (`true` o `false`) o applica un collettore downstream.

Riduzioni Downstream e Collettori Composti

- `counting()` → `Collector<T, ?, Long>`
 Conta gli elementi presenti nel gruppo.
- `summingInt(mapper)` / `summingDouble(mapper)` → `Collector`
 Somma i valori numerici estratti da ciascun elemento.
- `averagingDouble(mapper)` → `Collector`
 Calcola la media aritmetica dei valori estratti.
- `mapping(mapper, downstream)` → `Collector`
Firma: `static <T, U, A, R> Collector<T, ?, R> mapping(Function mapper, Collector downstream)`
 Applica una funzione di mapping agli elementi prima di passarli al collettore downstream.
Esempio: `groupingBy(Persona::getCitta, mapping(Persona::getNome, toList()))`.
- `filtering(predicate, downstream)` → `Collector`
Firma: `static <T, A, R> Collector<T, ?, R> filtering(Predicate predicate, Collector downstream)`
 (Java 9) Filtra gli elementi all'interno di ciascun gruppo prima dell'accumulo downstream.
- `flatMap(mapper, downstream)` → `Collector`
Firma: `static <T, U, A, R> Collector<T, ?, R> flatMapping(Function mapper, Collector downstream)`
 (Java 9) Appiattisce collezioni innestate durante il raggruppamento.
- `reducing(op)` / `reducing(identity, op)` / `reducing(identity, mapper, op)` → `Collector`
 Esegue una riduzione sugli elementi con operatore associativo.
- `teeing(c1, c2, merger)` → `Collector`
Firma: `static <T, R1, R2, R> Collector<T, ?, R> teeing(Collector c1, Collector c2, BiFunction<R1, R2, R> merger)`
Compositore a T di Collettori (Java 12): inoltra simultaneamente ciascun elemento dello stream a due collettori downstream indipendenti (`c1` e `c2`), fondendone i rispettivi risultati finali tramite la funzione `merger`.
Esempio Operativo (Calcolo congiunto di media senza ricalcolo stream):

```
double media = voti.stream().collect(
    Collectors.teeing(
```

```

        Collectors.summingDouble(v -> v), // c1: somma
        Collectors.counting(),           // c2: conteggio
        (somma, n) -> n == 0 ? 0.0 : somma / n // merger finale
    )
};

```

- `collectingAndThen(downstream, finisher)` → `Collector`

Firma: `static <T, A, R, RR> Collector<T, A, RR> collectingAndThen(Collector downstream, Function finisher)`

Applica una funzione di finitura trasformativa al termine del processo di accumulo del collettore downstream.

Java Swing, AWT, Eventi ed EDT (javax.swing & java.awt)

Gerarchia dei Componenti Grafici

- `JFrame`: `setTitle(String)`, `setSize(int w, int h)`, `setLocationRelativeTo(null)` (centra lo schermo), `pack()` (ridimensiona sui componenti), `setVisible(boolean)`, `setDefaultCloseOperation(int)` (costanti: `JFrame.EXIT_ON_CLOSE`, `DISPOSE_ON_CLOSE`).
- `JButton`: `addActionListener(ActionListener l)`, `setText(String)`, `getText()` → `String`, `setEnabled(boolean)`.
- `JTextField`: `getText()` → `String`, `setText(String)`, `setColumns(int)`, `setEditable(boolean)`.
- `JLabel`: `setText(String)`, `getText()` → `String`, `setIcon(Icon)`.
- `JCheckBox` / `JRadioButton`: `isSelected()` → `boolean`, `setSelected(boolean)`, `addItemListener(ItemListener)`. I radio button si raggruppano con `new ButtonGroup().add(radioButton);`.
- `JComboBox<E>`: `addItem(E item)`, `getSelectedItem()` → `Object`, `getSelectedIndex()` → `int`.
- `JTable` e `DefaultTableModel`: `table.setModel(model)`, `model.addRow(Object[] rowData)`, `model.getValueAt(int r, int c)`, `model.setValueAt(Object val, int r, int c)`, `model.fireTableDataChanged()`.

Gestione del Layout (`LayoutManager`)

- `BorderLayout`: 5 regioni cardinali (NORTH, SOUTH, EAST, WEST, CENTER). North/South prendono altezza preferita e tutta la larghezza; East/West larghezza preferita; Center occupa tutto lo spazio residuo. Default di `JFrame`.
Uso: `panel.add(componente, BorderLayout.CENTER);`
- `FlowLayout`: Dispone i componenti in riga da sinistra a destra con a capo automatico. Default di `JPanel`.
Costruttore: `new FlowLayout(FlowLayout.LEFT, hgap, vgap);`
- `GridLayout`: Griglia a celle di dimensioni identiche (`new GridLayout(rows, cols, hgap, vgap);`).
- `BoxLayout`: Disposizione monodimensionale su un asse (`BoxLayout.X_AXIS` o `Y_AXIS`). Supporta colle elastiche (`Box.createHorizontalGlue()`) e distanziatori fissi (`Box.createRigidArea(new Dimension(w, h))`).
- `GridBagLayout`: Griglia flessibile avanzata governata da `GridBagConstraints`: `gridx`, `gridy` (coordinate cella), `gridwidth`, `gridheight` (estensione celle), `weightx`, `weighty` (dilatazione in resize), `fill` (BOTH, HORIZONTAL, VERTICAL, NONE), `insets` (margini esterni).

Modello ad Eventi e Listener

- `ActionListener` → `void actionPerformed(ActionEvent e)`: Click pulsanti, invio su campi di testo.

- `ItemListener` → `void itemStateChanged(ItemEvent e)`: Variazione selezione su checkbox/radio button.
- `MouseListener` / `MouseAdapter` → `mouseClicked`, `mousePressed`, `mouseReleased`, `mouseEntered`, `mouseExited`.
- `WindowListener` / `WindowAdapter` → `windowClosing(WindowEvent e)`: Intercetta la chiusura della finestra.

Concorrenza Grafica ed Event Dispatch Thread (EDT)

- **Regola del Single-Thread:** Swing non è thread-safe. Tutte le modifiche ai componenti devono avvenire rigidamente sull'EDT.
- `SwingUtilities.invokeLater(Runnable r)`: Accoda un task da eseguire sull'EDT, evitando race conditions.
Esempio avvio GUI: `SwingUtilities.invokeLater(() -> new MioFrame().setVisible(true));`
- `SwingUtilities.isEventDispatchThread()` → `boolean`: Verifica se il thread corrente è l'EDT.
- `javax.swing.Timer`: Timer periodico che scatta interamente sull'EDT:
`new Timer(delayMillis, e -> /* aggiornamento periodico della GUI */ ).start();`

Task Asincroni Pesanti: `javax.swing.SwingWorker<T, V>`

- Consente di eseguire calcoli lunghi (I/O, database, simulazioni) senza congelare la UI.

Ciclo di vita strutturato:

- `T doInBackground() throws Exception`: eseguito su un **thread in background**; calcola e restituisce il risultato finale di tipo `T`.
- `void publish(V... chunks)`: invocato da `doInBackground` per trasmettere dati intermedi di tipo `V`.
- `void process(List<V> chunks)`: eseguito sull'EDT; riceve i dati intermedi e aggiorna la barra di avanzamento o la GUI.
- `void done()`: eseguito sull'EDT quando il background ha terminato; richiama `get()` per estrarre il valore finale di tipo `T` in totale sicurezza.

Disegno Grafico 2D Personalizzato

- Sovrascrittura di `protected void paintComponent(Graphics g)` in una sottoclasse di `JPanel`.
- Invocazione obbligatoria come prima istruzione di `super.paintComponent(g)`; per ripulire lo sfondo.
- Cast al contesto grafico avanzato: `Graphics2D g2 = (Graphics2D) g;`
- Attivazione dell'antialiasing: `g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);`
- Tracciamento e riempimento: `g2.setColor(Color)`, `g2.setStroke(Stroke)`, `g2.draw(Shape)`, `g2.fill(Shape)`.
- Forzatura del ridisegno da codice: `pannello.repaint()` accoda la richiesta di rendering all'EDT.